



PART 9

ADVANCED TOPICS

Chapter 24 provides further details about the index structures we covered in Chapter 14. In particular, this chapter provides detailed coverage of the LSM tree and its variants, bitmap indices, and spatial indexing, all of which were covered in brief in Chapter 14. The chapter also provides detailed coverage of dynamic hashing techniques.

Chapter 25 discusses a number of tasks involved in application development. Applications can be made to run significantly faster by performance tuning, which consists of finding and eliminating bottlenecks and adding appropriate hardware such as memory or disks. Application performance is evaluated using benchmarks, which are standardized sets of tasks that help to characterize the performance of database systems. Another important aspect of application development is testing, which requires the generation of database states and test inputs, followed by checking that the actual outputs of a query or a program on the test input match the expected outputs. Lastly, standards are very important for application development. A variety of standards have been proposed that affect database-application development. We outline several of these standards in this chapter.

Chapter 26 covers blockchain technology from a database perspective. This chapter identifies the ways in which blockchain databases differ from the traditional databases covered elsewhere in this text and shows how these distinguishing features are implemented. Although blockchain systems are often associated with Bitcoin, this chapter goes beyond Bitcoin-style algorithms and implementation to focus on alternatives that are more suited to an enterprise database environment.

CHAPTER 24



Advanced Indexing Techniques

We studied the concept of indexing, as well as a number of different index structures in Chapter 14. While some index structures, such as B⁺-trees, were covered in detail, others such as hashing, write-optimized indices, bitmap indices, and spatial indices were only briefly outlined in Chapter 14. In this chapter we provide further details of these index structures. We provide detailed coverage of the LSM tree and its variants. We then provide a detailed description of bitmap indices. Next, we provide more detailed coverage of spatial indexing, covering quad trees and R-trees in more detail. Finally, we cover hashing, with detailed coverage of dynamic hashing techniques.

24.1 Bloom Filter

A **Bloom filter** is a probabilistic data structure that can check for membership of a value in a set using very little space, but at a small risk of overestimating the set of elements that are in the set. A Bloom filter is basically a bitmap. If the set has n values, the associated bitmap has a few times n (typically $10n$) bits; the Bloom filter also has associated with it several hash functions. We assume initially that there is only one hash function $h()$.

The bits in the bitmap are all initially set to 0; subsequently, each value in the set is read, and the hash function $h(v)$ is computed on the element v , with the range of the function being 1 to $10n$. The bit at position $h(v)$ is then set to 1. This is repeated for every element v . To check if a particular value v is present in the set, the hash function $h(v)$ is computed. If bit $h(v)$ in the Bloom filter is equal to 0, we can infer that v cannot possibly be in the set. However, if bit $h(v)$ is equal to 1, v may be present in the set. Note that with some probability, the bit $h(v)$ may be 1 even if v is not present, if some other value v' , present in the set has $h(v') = h(v)$. Thus, a lookup for v results in a *false positive*.

To reduce the chance of false positives, Bloom filters use k independent hash functions $h_i()$, $i = 1..k$, for some $k > 1$; for each value v in the set, bits corresponding to $h_i(v)$, $i = 1..k$ are all set to 1 in the bitmap. When querying the Bloom filter with a

given value v the same k hash functions are used to identify k bit locations; the value v is determined to be absent if even one of these bits has a 0 value. Otherwise the value is judged to be potentially present. For example, if the bitmap has $10n$ bits, where n is the number of values in the set, and $k = 7$ hash functions are used, the false positive rate would be about 1%.

24.2 Log-Structured Merge Tree and Variants

As we saw in Section 14.8, B^+ -tree indices are not efficient for workloads with a very high number of writes, and alternative index structures have been proposed to handle such workloads. We saw a brief description of two such index structures, the *log-structured merge tree* or LSM tree and its variants, in Section 14.8.1, and the *buffer tree*, in Section 14.8.2. In this section we provide further details of the LSM tree and its variants. To help with the discussions, we repeat some of the basic material we presented in Section 14.8.1.

The key idea of the **log-structured merge tree (LSM tree)** is to replace random I/O operations during tree inserts, updates, and deletes with a smaller number of sequential I/O operations. Our initial description focuses on index inserts and lookups; we describe how to handle updates and deletes later in the section.

An LSM tree consists of several B^+ -trees, starting with an in-memory tree, called L_0 , and on-disk trees $L_1, L_2, \dots, L_k$ for some k , where k is called the level. Figure 24.1 depicts the structure of an LSM tree for $k = 3$.

An index lookup is performed by using separate lookup operations on each of the trees $L_0, \dots, L_k$, and merging the results of the lookups. (We assume here that there are no updates or deletes; we will discuss how to perform lookups in the presence of updates/deletes later.)

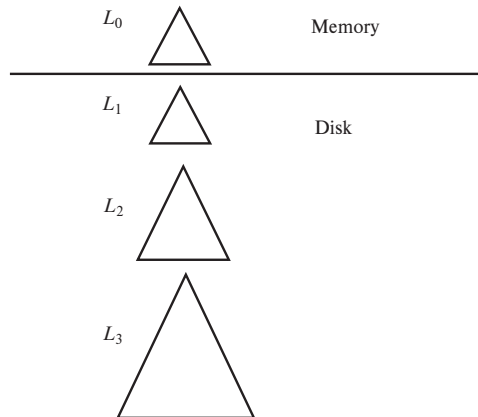


Figure 24.1 Log-structured merge tree with three levels.

24.2.1 Insertion into LSM Trees

When a record is first inserted into an LSM tree, it is inserted into the in-memory B⁺-tree structure L_0 . A fairly large amount of memory space is allocated for this tree. As the tree grows to fill the memory allocated to it, we need to move data from the in-memory structure to a B⁺-tree on disk.

If tree L_1 is empty, the entire in-memory tree L_0 is written to disk to create the initial tree L_1 . However, if L_1 is not empty, the leaf level of L_0 is scanned in increasing key order, and entries are merged with the leaf level entries of L_1 (also scanned in increasing key order). The merged entries are used to create a new B⁺-tree using the bottom-up build process. The new tree with the merged entries then replaces the old L_1 . In either case, after entries of L_0 have been moved to L_1 , all entries in L_0 are deleted. Inserts can then be made to the now empty L_0 .

Note that all entries in the leaf level of the old L_1 tree, including those in leaf nodes that do not have any updates, are copied to the new tree instead of being inserted into the existing L_1 tree node. This gives the following benefits:

- The leaves of the new tree are sequentially located, avoiding random I/O during subsequent merges.
- The leaves are full, avoiding the overhead of partially occupied leaves that can occur with page splits.

There is, however, a cost to using the LSM structure: the entire contents of the tree are copied each time a set of entries from L_0 are copied into L_1 .

If the tree structure is implemented on top of a distributed file system (Section 21.6), copying data to a new tree is often unavoidable, since most distributed file systems do not support updates to an already created block.

To ensure we get a benefit for cases where the index size on disk is much bigger than the in-memory index, the maximum size of L_1 is chosen as k times the target size of L_0 , for some k . Similarly, the maximum size of each L_{i+1} is set to k times the target size of L_i . Once a particular L_i reaches its maximum size, its entries are merged into the next component L_{i+1} . When L_{i+1} reaches its target size, its entries are in turn merged into L_{i+2} , and so on.

Note that if each leaf of L_i has m entries, m/k entries would map to a single leaf node of L_{i+1} . The value of k is chosen to ensure that m/k is some reasonable number, say 10. Let M denote the size of L_0 . Then, the size of a tree at level L_i is $k^i M$. The total number of levels r is thus roughly $\log_k(I/M)$ where I is the total size of the index entries.

Let us now consider the number of I/O operations required with a multiple-level LSM tree. At each L_i , m/k inserts are performed using only one I/O operation. On the other hand, each entry gets inserted once at each level L_i . Thus, the total number of I/O operations for each insert is $(k/m)\log_k(I/M)$. Thus, as long as the number of

levels $r = \log_k(I/M)$ is less than m/k , the overall number of I/O operations per insert is reduced by using an LSM tree as compared to direct insertion into a B^+ -tree.

If, for example, $r = 10$ and $k = 10$, the in-memory index needs to be greater than 1% of the total index size to get a benefit in terms of the number of I/O operations required for inserts. (As before, the benefit of reduced seeks is available even if the L_0 is significantly smaller.)

If the number of levels is greater than m/k , even if there is no benefit in terms of number of I/O operations, there can still be savings since sequential I/O is used instead of random I/O. In Section 24.2.4 we describe a variant of the LSM tree which further reduces the overhead on write operations, at the cost of adding overhead on read operations.

One way to avoid creating large LSM trees with many levels is to *range partition* the relation and create separate LSM trees on each partition. Such partitioning is natural in a parallel environment, as we saw earlier in Section 21.2. In particular, in such environments, a partition can be dynamically repartitioned into smaller pieces whenever it becomes too large, as we saw in Section 21.3.3. With such repartitioning, the size of each LSM tree can be kept small enough to avoid having a large number of levels. There is a price for such partitioning: each partition requires its own L_0 tree in memory. As a result, although it can be used in a centralized setting, the partitioning approach best fits a parallel environment where resources such as processing nodes can be added as the load increases.

24.2.2 Rolling Merges

We assumed for simplicity that when a particular level is full, its entries are entirely merged with the next level. This would result in more I/O load during merges with an unused I/O capacity between merges. To avoid this problem, merging is done on a continuous basis; this is called rolling merge.

With **rolling merge**, a few pages of L_i are merged into corresponding pages of L_{i+1} at a time, and removed from L_i . This is done whenever L_i becomes close to its target size, and it results in L_i shrinking a bit to return to its target size. When L_i grows again, the rolling merge restarts from a point at the leaf level of L_i just after where the earlier rolling merge stopped, so the scan is sequential. When the end of the L_i tree is reached, the scan starts again at the beginning of the tree. Such a merge is called a rolling merge since records are moved from one level to another on a continuous basis.

The number of leaves merged at a time is kept high enough to ensure that the seek time is small compared to the time to transfer data from and to disk.

24.2.3 Handling Deletes and Updates

So far we have only described inserts and lookups. Deletes are handled in an interesting manner. Instead of directly finding an index entry and deleting it, deletion results in insertion of a new **deletion entry** that indicates which index entry is to be deleted. The

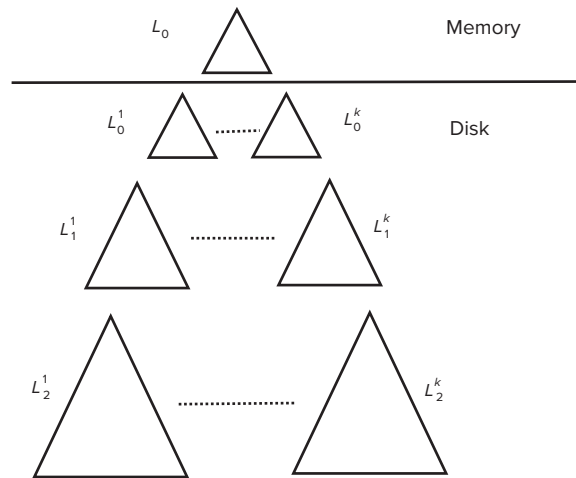


Figure 24.2 stepped-merge index

process of inserting a deletion entry is identical to the process of inserting a normal index entry.

However, lookups have to carry out an extra step. As mentioned earlier, lookups retrieve entries from all the trees and merge them in sorted order of key value. If there is a deletion entry for some entry, both of them would have the same key value. Thus, a lookup would find both the deletion entry and the original entry for that key, which is to be deleted. If a deletion entry is found, the to-be-deleted entry should be filtered out and not returned as part of the lookup result.

When trees are merged, if one of the trees contains an entry, and the other had a matching deletion entry, the entries get matched up during the merge (both would have the same key), and are both discarded.

Updates are handled in a manner similar to deletes, by inserting an update entry. Lookups need to match update entries with the original entries and return the latest value. The update is actually applied during a merge, when one tree has an entry and another has its matching update entry; the update is applied during the merge, and the update entry is discarded.

24.2.4 The Stepped-Merge Index

We now consider a variant of the LSM tree, which has multiple trees at each level instead of one tree per level and performs inserts in a slightly different manner. This structure is shown in Figure 24.2. We call the structure a **stepped-merge index**, following the terminology in an early paper that introduced it. In the developer community, the basic LSM tree, the stepped-merge index, and several other variants are all referred to as LSM trees. We use the terms *stepped-merge index* and *basic LSM tree* to clearly identify which variant we are referring to.

24.2.4.1 Insertion Algorithm

In the stepped-merge index, incoming data are initially stored in memory, in an L_0 tree, in a manner similar to the LSM tree. However, when the tree reaches its maximum size, instead of merging it into an L_1 tree, the in-memory L_0 tree is written to disk. When the in-memory tree again reaches its maximum size, it is again written to disk. Thus, we may have multiple L_0 trees on disk, which we shall refer to as L_0^1, L_0^2 and so forth. Each of the L_0^i trees is a B⁺-tree and can be written to disk using only sequential I/O operations.

If this process is repeated, after a while we would have a large number of trees, each as large as memory, stored on disk. Lookups would then have to pay a high price, since they would have to search through each of the tree structures, incurring separate I/O costs on each search.

To limit the overhead on lookups, once the number of on-disk trees at a level L_i reaches some limit k , all the trees at a level are merged together into one combined new tree structure at the next level L_{i+1} . The leaves of the trees at level L_i are read sequentially, and the keys merged in sorted order, and the level L_{i+1} tree is constructed using standard techniques for bottom-up construction of B⁺-trees. As before, the merge operation avoids random I/O operations, since it reads the individual tree structures sequentially and writes the resultant merged tree also sequentially.

Once a set of trees are merged into a single new tree, future queries can search the merged tree; the original trees can then be deleted (after ensuring any ongoing searches have completed).

The benefit of the stepped-merge index scheme as compared to the basic LSM tree is that index entries are written out only once per level. With the basic LSM tree, each time a tree at level L_i is merged into a tree at level L_{i+1} , the entire contents of the L_{i+1} tree is read and written back to disk. Thus, on average, each record is read and written back $k/2$ times at each level of an LSM tree for a total of $k \log_k(I/M)$ I/O operations. In contrast, with stepped-merge index, each record is written to disk once per layer, and read again when merging into the next layer, for a total of approximately $2 \log_k(I/M)$ I/O operations. Thus, the stepped-merge index incurs significantly less cost for updates.

The total number of bytes written (across all levels) on account of inserting an entry, divided by the size of entry, is called the **write amplification**. To calculate the write amplification of the LSM trees and the stepped-merge index, we can modify the above formulae for I/O operations by ignoring the reads. For a B⁺-tree where each leaf gets on average only one update before it is written back, the write amplification would be the size of the page divided by the size of the index entry.

For a B⁺-tree, if a page has 100 entries, the write amplification would be 100. With $k = 5$, and $I = 100M$, we would have $\log_5(100) = 3$ levels. The write amplification of an LSM tree would then be $5/2 \times 3 = 7.5$. The write amplification of the stepped-merge index would be 3. With $k = 10$, the tree would have $\log_{10}(100) = 2$ levels, leading to a write amplification of 2 for stepped-merge index, and 10 for an LSM tree.

Note that like the basic LSM tree, the stepped-merge index also requires no random I/O operations during insertion, in contrast to a B⁺-tree insertion. Thus, the performance of B⁺-trees would be worse than what the write amplification number above indicates.

Merging can be optimized as follows: While merging k trees at a particular level L_i , into a level L_{i+1} tree, trees at levels $L_j, j < i$ can also be merged in at the same time. Entries in these trees can thus entirely skip one or more levels of the stepped-merge index. Further, if the system has idle capacity, trees at a level L_i can be merged even if there are fewer than k trees at that level. In a situation where there is a long period of time with very few inserts, and the system load is light, trees across all levels could potentially get merged into a single tree at some level r .

24.2.4.2 Lookup Operations Using Bloom Filters

Lookup operations in stepped-merge index have to separately search each of the trees. Thus, compared to the basic LSM scheme, the stepped-merge index increases the burden on lookups, since in the worst case lookups need to access k trees at each level, leading to a total of $k * \log_k(I/M)$ tree lookups, instead of $\log_k(I/M)$ tree lookups in the worst case with a basic LSM tree.

For workloads with a significant fraction of reads, this overhead can be unacceptable. For example, with the stepped-merge index with $I = 100M$ and $k = 5$, a single lookup requires 15 tree traversals, while the LSM tree would require 3 tree traversals. Note that for the common case where each key value occurs in only one tree, only one of the traversals would find a given search key, while all the other traversals would fail to find the key.

To reduce the cost of point lookups (i.e., lookups of a given key value), most systems use a Bloom filter to check if a tree can possibly contain the given key value. One Bloom filter is associated with each tree, and it is built on the set of key values in the tree. To check if a particular tree may contain a search key v , the key v is looked up in the Bloom filter. If the Bloom filter indicates that the key value is absent, it is definitely not present in the tree, and lookup can skip that tree. Otherwise, the key value may be present in the tree, which must be looked up.

A Bloom filter with $10n$ bits, where the tree has n elements, and using 7 hash functions would give a false positive rate of 1 percent. Thus, for a lookup on a key that is present in the index, on average just slightly more than one tree would be accessed. Thus, lookup performance would be only slightly worse than on a regular B⁺-tree.

The Bloom filter check thus works very well for point lookups, allowing a significant fraction of the trees to be skipped, as long as sufficient memory is available to store all the Bloom filters in memory. With I key values in the index, approximately $10I$ bits of memory will be required. To reduce the main memory overhead, some of the Bloom filters may be stored on flash storage.

Note that for range lookups, the Bloom filter optimization cannot be used, since there is no unique hash value. Instead, all trees must be accessed separately.

24.2.5 LSM Trees For Flash Storage

LSM trees were initially designed to reduce the write and seek overheads of hard disks. Flash disks have a relatively low overhead for random I/O operations since they do not require seek, and thus the benefit of avoiding random I/O that LSM tree variants provide is not particularly important with flash disks.

However, recall that flash memory does not allow in-place update, and writing even a single byte to a page requires the whole page to be rewritten to a new physical location; the original location of the page needs to be erased eventually, which is a relatively expensive operation. The reduction in write amplification using LSM tree variants, as compared to traditional B⁺-trees, can provide substantial performance benefits when LSM trees are used with flash storage.

24.3 Bitmap Indices

As we saw in Section 14.9, a bitmap index is a specialized type of index designed for easy querying on multiple keys. Bitmaps work best for attributes that take only a small number of distinct values.

For bitmap indices to be used, records in a relation must be numbered sequentially, starting from, say, 0. Given a number n , it must be easy to retrieve the record numbered n . This is particularly easy to achieve if records are fixed in size and allocated on consecutive blocks of a file. The record number can then be translated easily into a block number and a number that identifies the record within the block.

Recall that *column-oriented storage*, described in Section 13.6, stores attributes in arrays, allowing efficient access of the attribute of the i th record, for any given i . Bitmap indices are thus particularly useful with columnar storage.

We use as a running example a relation *instructor_info*, which has an attribute *gender*, which can take only values **m** (male) or **f** (female), and an attribute *income_level*, where income has been broken up into 5 levels: $L1$: 0-9999, $L2$: 10,000-19,999, $L3$: 20,000-39,999, $L4$: 40,000-74,999, and $L5$: 75,000 – ∞ .

24.3.1 Bitmap Index Structure

As we saw in Section 14.9, a **bitmap** is simply an array of bits. In its simplest form, a **bitmap index** on the attribute A of relation r consists of one bitmap for each value that A can take. Each bitmap has as many bits as the number of records in the relation. The i th bit of the bitmap for value v_j is set to 1 if the record numbered i has the value v_j for attribute A . All other bits of the bitmap are set to 0.

In our example, there is one bitmap for the value **m** and one for **f**. The i th bit of the bitmap for **m** is set to 1 if the *gender* value of the record numbered i is **m**. All other bits of the bitmap for **m** are set to 0. Similarly, the bitmap for **f** has the value 1 for bits corresponding to records with the value **f** for the *gender* attribute; all other bits have

record number				Bitmaps for <i>gender</i>		Bitmaps for <i>income_level</i>	
	<i>ID</i>	<i>gender</i>	<i>income_level</i>	m	f	L1	L2
0	76766	m	L1	10010	01101	10100	01000
1	22222	f	L2			01000	00001
2	12121	f	L1			00001	00010
3	15151	m	L4			00010	00000
4	58583	f	L3			00000	

Figure 24.3 Bitmap indices on relation *instructor_info*.

the value 0. Figure 24.3 shows an example of bitmap indices on a relation *instructor_info*.

We now consider when bitmaps are useful. The simplest way of retrieving all records with value *m* (or value *f*) would be to simply read all records of the relation and select those records with value *m* (or *f*, respectively). The bitmap index doesn't really help to speed up such a selection. While it would allow us to read only those records for a specific gender, it is likely that every disk block for the file would have to be read anyway.

In fact, bitmap indices are useful for selections mainly when there are selections on multiple keys. Suppose we create a bitmap index on attribute *income_level*, which we described earlier, in addition to the bitmap index on *gender*.

Consider now a query that selects women with income in the range \$10,000 to \$19,999. This query can be expressed as

```
select *
from instructor_info
where gender = 'f' and income_level = 'L2';
```

To evaluate this selection, we fetch the bitmaps for *gender* value *f* and the bitmap for *income_level* value *L2* and perform an **intersection** (logical-and) of the two bitmaps. In other words, we compute a new bitmap where bit *i* has value 1 if the *i*th bit of the two bitmaps are both 1 and has a value 0 otherwise. In the example in Figure 24.3, the intersection of the bitmap for *gender* = *f* (01101) and the bitmap for *income_level* = *L2* (01000) gives the bitmap 01000.

Since the first attribute can take two values, and the second can take five values, we would expect only about 1 in 10 records, on an average, to satisfy a combined condition on the two attributes. If there are further conditions, the fraction of records satisfying all the conditions is likely to be quite small. The system can then compute the query result by finding all bits with value 1 in the intersection bitmap and retrieving the cor-

responding records. If the fraction is large, scanning the entire relation would remain the cheaper alternative.

Another important use of bitmaps is to count the number of tuples satisfying a given selection. Such queries are important for data analysis. For instance, if we wish to find out how many women have an income level $L2$, we compute the intersection of the two bitmaps and then count the number of bits that are 1 in the intersection bitmap. We can thus get the desired result from the bitmap index, without even accessing the relation.

Bitmap indices are generally quite small compared to the actual relation size. Records are typically at least tens of bytes to hundreds of bytes long, whereas a single bit represents the record in a bitmap. Thus, the space occupied by a single bitmap is usually less than 1 percent of the space occupied by the relation. For instance, if the record size for a given relation is 100 bytes, then the space occupied by a single bitmap will be $\frac{1}{8}$ of 1 percent of the space occupied by the relation. If an attribute A of the relation can take on only one of eight values, a bitmap index on attribute A would consist of eight bitmaps, which together occupy only 1 percent of the size of the relation.

Deletion of records creates gaps in the sequence of records, since shifting records (or record numbers) to fill gaps would be extremely expensive. To recognize deleted records, we can store an **existence bitmap**, in which bit i is 0 if record i does not exist and 1 otherwise. We shall see the need for existence bitmaps in Section 24.3.2. Insertion of records should not affect the sequence numbering of other records. Therefore, we can do insertion either by appending records to the end of the file or by replacing deleted records.

24.3.2 Efficient Implementation of Bitmap Operations

We can compute the intersection of two bitmaps easily by using a **for** loop: the i th iteration of the loop computes the **and** of the i th bits of the two bitmaps. We can speed up computation of the intersection greatly by using bit-wise **and** instructions supported by most computer instruction sets. A *word* usually consists of 32 or 64 bits, depending on the architecture of the computer. A bit-wise **and** instruction takes two words as input and outputs a word where each bit is the logical **and** of the bits in corresponding positions of the input words. What is important to note is that a single bit-wise **and** instruction can compute the intersection of 32 or 64 bits *at once*.

If a relation had 1 million records, each bitmap would contain 1 million bits, or equivalently 128 kilobytes. Only 31,250 instructions are needed to compute the intersection of two bitmaps for our relation, assuming a 32-bit word length. Thus, computing bitmap intersections is an extremely fast operation.

Just as bitmap intersection is useful for computing the **and** of two conditions, bitmap union is useful for computing the **or** of two conditions. The procedure for bitmap union is exactly the same as for intersection, except we use bit-wise **or** instructions instead of bit-wise **and** instructions.

The complement operation can be used to compute a predicate involving the negation of a condition, such as **not** (*income_level* = *L1*). The complement of a bitmap is generated by complementing every bit of the bitmap (the complement of 1 is 0 and the complement of 0 is 1). It may appear that **not** (*income_level* = *L1*) can be implemented by just computing the complement of the bitmap for income level *L1*. If some records have been deleted, however, just computing the complement of a bitmap is not sufficient. Bits corresponding to such records would be 0 in the original bitmap but would become 1 in the complement, although the records don't exist. A similar problem also arises when the value of an attribute is *null*. For instance, if the value of *income_level* is null, the bit would be 0 in the original bitmap for value *L1* and 1 in the complement bitmap.

To make sure that the bits corresponding to deleted records are set to 0 in the result, the complement bitmap must be intersected with the existence bitmap to turn off the bits for deleted records. Similarly, to handle null values, the complement bitmap must also be intersected with the complement of the bitmap for the value *null*.¹

Counting the number of bits that are 1 in a bitmap can be done quickly by a clever technique. We can maintain an array with 256 entries, where the *i*th entry stores the number of bits that are 1 in the binary representation of *i*. Set the total count initially to 0. We take each byte of the bitmap, use it to index into this array, and add the stored count to the total count. The number of addition operations is $\frac{1}{8}$ of the number of tuples, and thus the counting process is very efficient. A large array (using $2^{16} = 65,536$ entries), indexed by pairs of bytes, would give even higher speedup, but at a higher storage cost.

24.3.3 Bitmaps and B⁺-Trees

Bitmaps can be combined with regular B⁺-tree indices for relations where a few attribute values are extremely common, and other values also occur, but much less frequently. In a B⁺-tree index leaf, for each value we would normally maintain a list of all records with that value for the indexed attribute. Each element of the list would be a record identifier, consisting of at least 32 bits, and usually more. For a value that occurs in many records, we store a bitmap instead of a list of records.

Suppose a particular value v_i occurs in $\frac{1}{16}$ of the records of a relation. Let N be the number of records in the relation, and assume that a record has a 64-bit number identifying it. The bitmap needs only 1 bit per record, or N bits in total. In contrast, the list representation requires 64 bits per record where the value occurs, or $64 * N/16 = 4N$ bits. Thus, a bitmap is preferable for representing the list of records for value v_i . In our example (with a 64-bit record identifier), if fewer than 1 in 64 records have a particular value, the list representation is preferable for identifying records with that

¹Handling predicates such as **is unknown** would cause further complications, which would in general require use of an extra bitmap to track which operation results are unknown.

value, since it uses fewer bits than the bitmap representation. If more than 1 in 64 records have that value, the bitmap representation is preferable.

Thus, bitmaps can be used as a compressed storage mechanism at the leaf nodes of B⁺-trees for those values that occur very frequently.

24.4 Indexing of Spatial Data

As we saw in Section 14.10.1, indices are required for efficient access to spatial data, and such indices must efficiently support queries such as range and nearest neighbor queries. We also gave a brief overview of k-d trees, quadtrees, and R-trees; we also briefly described how to answer range queries using k-d trees. In this section we provide further details of quadtrees and R-trees.

As mentioned in Section 14.10.1, in addition to indexing of points, spatial indices must also support indexing of regions of space such as line segments, rectangles, and other polygons. There are extensions of k-d trees and quadtrees for this task. However, a line segment or polygon may cross a partitioning line. If it does, it has to be split and represented in each of the subtrees in which its pieces occur. Multiple occurrences of a line segment or polygon caused by such splits can result in inefficiencies in storage, as well as inefficiencies in querying. R-trees were developed to support efficient indexing of such structures.

24.4.1 Quadtrees

An alternative representation for two-dimensional data are a **quadtree**. An example of the division of space by a quadtree appears in Figure 24.4. Each node of a quadtree is

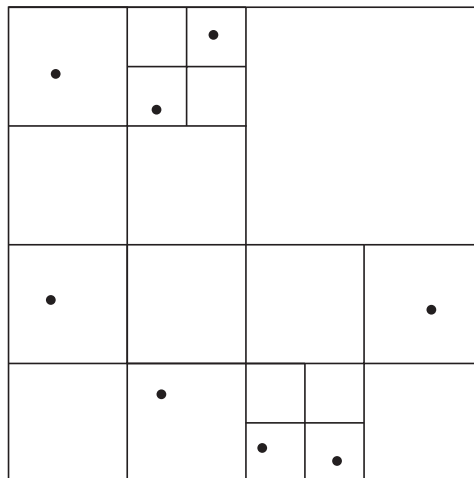


Figure 24.4 Division of space by a quadtree.

associated with a rectangular region of space. The top node is associated with the entire target space. Each nonleaf node in a quadtree divides its region into four equal-sized quadrants, and correspondingly each such node has four child nodes corresponding to the four quadrants. Leaf nodes have between zero and some fixed maximum number of points. Correspondingly, if the region corresponding to a node has more than the maximum number of points, child nodes are created for that node. In the example in Figure 24.4, the maximum number of points in a leaf node is set to 1.

This type of quadtree is called a **PR quadtree**, to indicate it stores points, and that the division of space is divided based on regions, rather than on the actual set of points stored. We can use **region quadtrees** to store array (raster) information. A node in a region quadtree is a leaf node if all the array values in the region that it covers are the same. Otherwise, it is subdivided further into four children of equal area and is therefore an internal node. Each node in the region quadtree corresponds to a subarray of values. The subarrays corresponding to leaves either contain just a single array element or have multiple array elements, all of which have the same value.

24.4.2 R-Trees

A storage structure called an **R-tree** is useful for indexing of objects such as points, line segments, rectangles, and other polygons. An R-tree is a balanced tree structure with the indexed objects stored in leaf nodes, much like a B⁺-tree. However, instead of a range of values, a rectangular **bounding box** is associated with each tree node. The bounding box of a leaf node is the smallest rectangle parallel to the axes that contains all objects stored in the leaf node. The bounding box of internal nodes is, similarly, the smallest rectangle parallel to the axes that contains the bounding boxes of its child nodes. The bounding box of an object (such as a polygon) is defined, similarly, as the smallest rectangle parallel to the axes that contains the object.

Each internal node stores the bounding boxes of the child nodes along with the pointers to the child nodes. Each leaf node stores the indexed objects and may optionally store the bounding boxes of the objects; the bounding boxes help speed up checks for overlaps of the rectangle with the indexed objects—if a query rectangle does not overlap with the bounding box of an object, it cannot overlap with the object, either. (If the indexed objects are rectangles, there is no need to store bounding boxes, since they are identical to the rectangles.)

Figure 24.5 shows an example of a set of rectangles (drawn with a solid line) and the bounding boxes (drawn with a dashed line) of the nodes of an R-tree for the set of rectangles. Note that the bounding boxes are shown with extra space inside them, to make them stand out pictorially. In reality, the boxes would be smaller and fit tightly on the objects that they contain; that is, each side of a bounding box B would touch at least one of the objects or bounding boxes that are contained in B .

The R-tree itself is at the right side of Figure 24.5. The figure refers to the coordinates of bounding box i as BB_i in the figure.

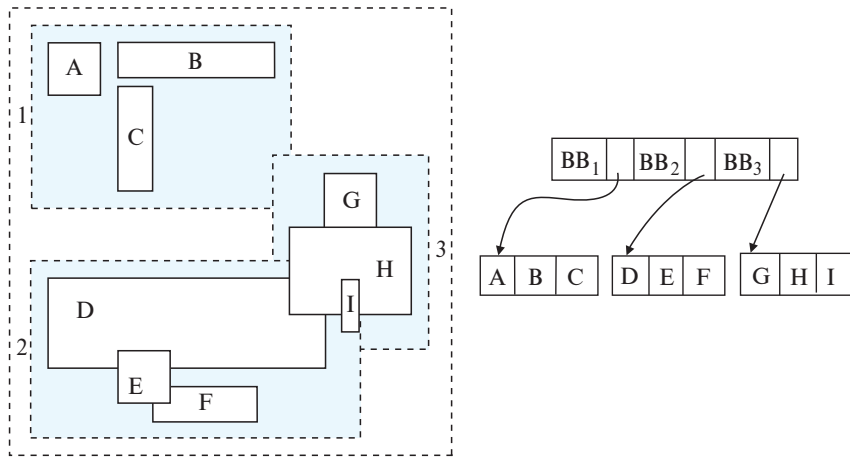


Figure 24.5 An R-tree.

We shall now see how to implement search, insert, and delete operations on an R-tree.

- **Search.** As the figure shows, the bounding boxes associated with sibling nodes may overlap; in B^+ -trees, k-d trees, and quadtrees, in contrast, the ranges do not overlap. A search for objects containing a point therefore has to follow *all* child nodes whose associated bounding boxes contain the point; as a result, multiple paths may have to be searched. Similarly, a query to find all objects that intersect a given object has to go down every node where the associated rectangle intersects the given object.
- **Insert.** When we insert an object into an R-tree, we select a leaf node to hold the object. Ideally we should pick a leaf node that has space to hold a new entry, and whose bounding box contains the bounding box of the object. However, such a node may not exist; even if it did, finding the node may be very expensive, since it is not possible to find it by a single traversal down from the root. At each internal node we may find multiple children whose bounding boxes contain the bounding box of the object, and each of these children needs to be explored. Therefore, as a heuristic, in a traversal from the root, if any of the child nodes has a bounding box containing the bounding box of the object, the R-tree algorithm chooses one of them arbitrarily. If none of the children satisfy this condition, the algorithm chooses a child node whose bounding box has the maximum overlap with the bounding box of the object for continuing the traversal.

Once the leaf node has been reached, if the node is already full, the algorithm performs node splitting (and propagates splitting upward if required) in a manner very similar to B^+ -tree insertion. Just as with B^+ -tree insertion, the R-tree insertion algorithm ensures that the tree remains balanced. Additionally, it ensures that the

bounding boxes of leaf nodes, as well as internal nodes, remain consistent; that is, bounding boxes of leaves contain all the bounding boxes of the objects stored at the leaf, while the bounding boxes for internal nodes contain all the bounding boxes of the children nodes.

The main difference of the insertion procedure from the B^+ -tree insertion procedure lies in how the node is split. In a B^+ -tree, it is possible to find a value such that half the entries are less than the midpoint and half are greater than the value. This property does not generalize beyond one dimension; that is, for more than one dimension, it is not always possible to split the entries into two sets so that their bounding boxes do not overlap. Instead, as a heuristic, the set of entries S can be split into two disjoint sets S_1 and S_2 so that the bounding boxes of S_1 and S_2 have the minimum total area; another heuristic would be to split the entries into two sets S_1 and S_2 in such a way that S_1 and S_2 have minimum overlap. The two nodes resulting from the split would contain the entries in S_1 and S_2 , respectively. The cost of finding splits with minimum total area or overlap can itself be large, so cheaper heuristics, such as the *quadratic split* heuristic, are used. (The heuristic gets its name from the fact that it takes time quadratic in the number of entries.)

The **quadratic split** heuristic works this way: First, it picks a pair of entries a and b from S such that putting them in the same node would result in a bounding box with the maximum wasted space; that is, the area of the minimum bounding box of a and b minus the sum of the areas of a and b is the largest. The heuristic places the entries a and b in sets S_1 and S_2 , respectively.

It then iteratively adds the remaining entries, one entry per iteration, to one of the two sets S_1 or S_2 . At each iteration, for each remaining entry e , let $i_{e,1}$ denote the increase in the size of the bounding box of S_1 if e is added to S_1 and let $i_{e,2}$ denote the corresponding increase for S_2 . In each iteration, the heuristic chooses one of the entries with the maximum difference of $i_{e,1}$ and $i_{e,2}$ and adds it to S_1 if $i_{e,1}$ is less than $i_{e,2}$, and to S_2 otherwise. That is, an entry with “maximum preference” for S_1 or S_2 is chosen at each iteration. The iteration stops when all entries have been assigned, or when one of the sets S_1 or S_2 has enough entries that all remaining entries have to be added to the other set so the nodes constructed from S_1 and S_2 both have the required minimum occupancy. The heuristic then adds all unassigned entries to the set with fewer entries.

- **Deletion.** Deletion can be performed like a B^+ -tree deletion, borrowing entries from sibling nodes, or merging sibling nodes if a node becomes underfull. An alternative approach redistributes all the entries of underfull nodes to sibling nodes, with the aim of improving the clustering of entries in the R-tree.

See the bibliographical references for more details on insertion and deletion operations on R-trees, as well as on variants of R-trees, called R^* -trees or R^+ -trees.

The storage efficiency of R-trees is better than that of k-d trees or quadrees, since an object is stored only once, and we can ensure easily that each node is at least half

full. However, querying may be slower, since multiple paths have to be searched. Spatial joins are simpler with quadrees than with R-trees, since all quadrees on a region are partitioned in the same manner. However, because of their better storage efficiency and their similarity to B-trees, R-trees and their variants have proved popular in database systems that support spatial data.

24.5 Hash Indices

We described the concepts of hashing and hash indices in Section 14.5. We provide further details in this section.

24.5.1 Static Hashing

As in Section 14.5, let K denote the set of all search-key values, and let B denote the set of all bucket addresses. A hash function h is a function from K to B . Let h denote a hash function. Recall that in a hash index, buckets contain index entries, with pointers to records, while in a hash file organization, actual records are stored in the buckets. All the other details remain the same, so we do not explicitly differentiate between these two versions henceforth. We use the term *hash index* to denote hash file organizations as well as secondary hash indices.

Figure 24.6 shows a secondary hash index on the *instructor* file, for the search key *ID*. The hash function in the figure computes the sum of the digits of the *ID* modulo 8. The hash index has eight buckets, each of size 2 (realistic indices would have much larger bucket sizes). One of the buckets has three keys mapped to it, so it has an overflow bucket. In this example, *ID* is a primary key for *instructor*, so each search key has only one associated pointer. In general, multiple pointers can be associated with each key.

Hash indices can efficiently answer *point queries*, which retrieve records with a specified value for a search key. However, they cannot efficiently answer *range queries*, which retrieve all records whose search key value lies in a range (lb, ub) . The difficulty arises because a good hash function assigns values randomly to buckets. Thus, there is no simple notion of “next bucket in sorted order.” The reason we cannot chain buckets together in sorted order on A_i is that each bucket is assigned many search-key values. Since values are scattered randomly by the hash function, the values in the specified range are likely to be scattered across many or all of the buckets. Therefore, we have to read all the buckets to find the required search keys.

Recall that deletion is done as follows: If the search-key value of the record to be deleted is K_i , we compute $h(K_i)$, then search the corresponding bucket for that record, and delete the record from the bucket. Deletion of a record is efficient if there are not many records with a given key value. However, in the case of a hash index on a key with many duplicates, a large number of entries with the same key value may have to be scanned to find the entry for the record that is to be deleted. The complexity can in the worst case be linear in the number of records.

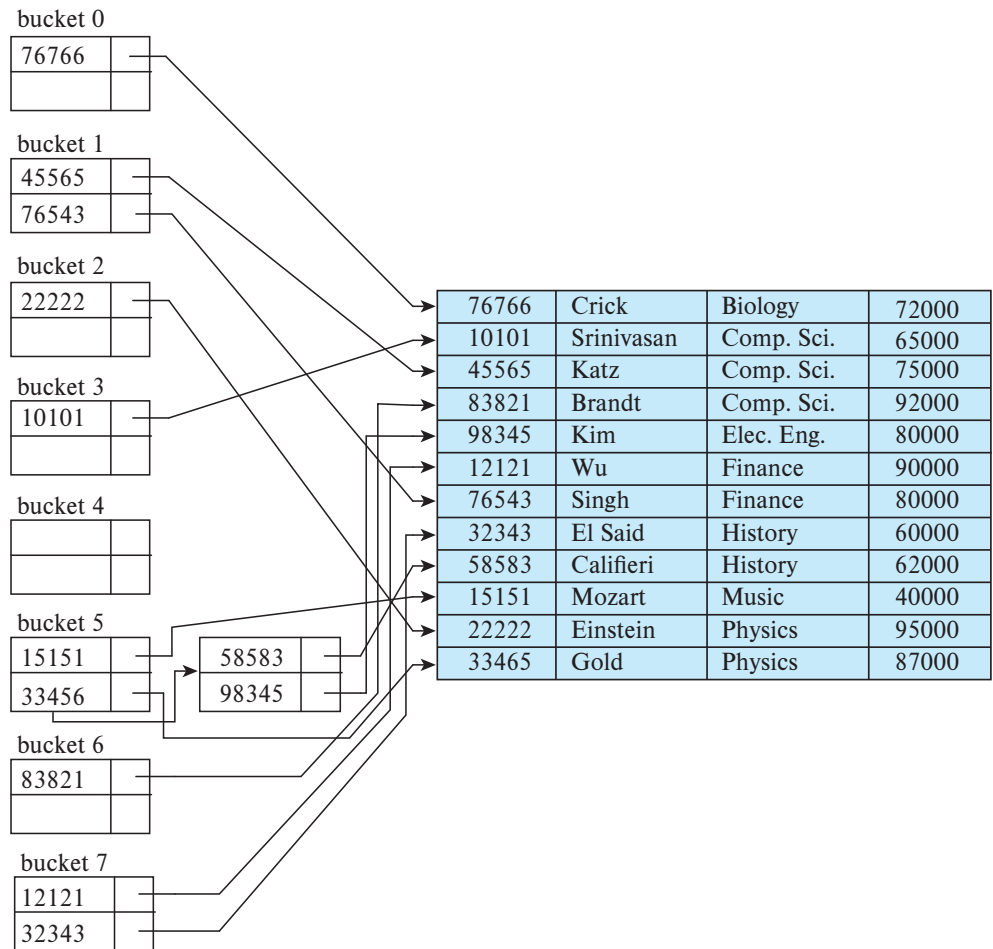


Figure 24.6 Hash index on search key *ID* of *instructor* file.

Recall also that with static hashing, the set of buckets is fixed at the time the index is created. If the relation grows far beyond the expected size, hash indices would be quite inefficient due to long overflow chains. We could rebuild the hash index using a larger number of buckets. Such rebuilding can be triggered when the number of records exceeds the estimated number by some margin, and the index is rebuilt with a number of buckets that is a multiple of the original number of buckets (say by a factor of 1.5 to 2). Such rebuilding is in fact done in many systems with in-memory hash indices.

However, doing so can cause significant disruption to normal processing with large relations, since a large number of records have to be reindexed; the disruption is particularly marked with disk-resident data. In this section we discuss dynamic hashing techniques that allow hash indices to grow gradually, without causing disruption.

24.5.1.1 Hash Functions

The worst possible hash function maps all search-key values to the same bucket. Such a function is undesirable because all the records have to be kept in the same bucket. A lookup has to examine every such record to find the one desired. An ideal hash function distributes the stored keys uniformly across all the buckets, so that every bucket has the same number of records.

Since we do not know at design time precisely which search-key values will be stored in the file, we want to choose a hash function that assigns search-key values to buckets in such a way that the distribution has these qualities:

- The distribution is *uniform*. That is, the hash function assigns each bucket the same number of search-key values from the set of *all* possible search-key values.
- The distribution is *random*. That is, in the average case, each bucket will have nearly the same number of values assigned to it, regardless of the actual distribution of search-key values. More precisely, the hash value will not be correlated to any externally visible ordering on the search-key values, such as alphabetic ordering or ordering by the length of the search keys; the hash function will appear to be random.

As an illustration of these principles, let us choose a hash function for the *instructor* file using the search key *dept_name*. The hash function that we choose must have the desirable properties not only on the example *instructor* file that we have been using, but also on an *instructor* file of realistic size for a large university with many departments.

Assume that we decide to have 26 buckets, and we define a hash function that maps names beginning with the *i*th letter of the alphabet to the *i*th bucket. This hash function has the virtue of simplicity, but it fails to provide a uniform distribution, since we expect more names to begin with such letters as B and R than Q and X, for example.

Now suppose that we want a hash function on the search key *salary*. Suppose that the minimum salary is \$30,000 and the maximum salary is \$130,000, and we use a hash function that divides the values into 10 ranges, \$30,000–\$40,000, \$40,001–\$50,000, and so on. The distribution of search-key values is uniform (since each bucket has the same number of different *salary* values) but is not random. Records with salaries between \$60,001 and \$70,000 are far more common than are records with salaries between \$30,001 and \$40,000. As a result, the distribution of records is not uniform—some buckets receive more records than others do. If the function has a random distribution, even if there are such correlations in the search keys, the randomness of the distribution will make it very likely that all buckets will have roughly the same number of records, as long as each search key occurs in only a small fraction of the records. (If a single search key occurs in a large fraction of the records, the bucket containing it is likely to have more records than other buckets, regardless of the hash function used.)

bucket 0			

bucket 1			
15151	Mozart	Music	40000

bucket 2			
32343	El Said	History	80000
58583	Califieri	History	60000

bucket 3			
22222	Einstein	Physics	95000
33456	Gold	Physics	87000
98345	Kim	Elec. Eng.	80000

bucket 4			
12121	Wu	Finance	90000
76543	Singh	Finance	80000

bucket 5			
76766	Crick	Biology	72000

bucket 6			
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

bucket 7			

Figure 24.7 Hash organization of *instructor* file, with *dept_name* as the key.

Typical hash functions perform computation on the internal binary machine representation of characters in the search key. A simple hash function of this type first computes the sum of the binary representations of the characters of a key, then returns the sum modulo the number of buckets.

Figure 24.7 shows the application of such a scheme, with eight buckets, to the *instructor* file, under the assumption that the *i*th letter in the alphabet is represented by the integer *i*.

The following hash function is a better alternative for hashing strings. Let *s* be a string of length *n*, and let *s*[*i*] denote the *i*th byte of the string. The hash function is defined as:

$$s[0] * 31^{(n-1)} + s[1] * 31^{(n-2)} + \dots + s[n-1]$$

The function can be implemented efficiently by setting the hash result initially to 0 and iterating from the first to the last character of the string, at each step multiplying the hash value by 31 and then adding the next character (treated as an integer). The above expression would appear to result in a very large number, but it is actually computed with fixed-size positive integers; the result of each multiplication and addition is thus automatically computed modulo the largest possible integer value plus 1. The result of the above function modulo the number of buckets can then be used for indexing.

Hash functions require careful design. A bad hash function may result in lookup taking time proportional to the number of search keys in the file. A well-designed function gives an average-case lookup time that is a (small) constant, independent of the number of search keys in the file.

24.5.1.2 Handling of Bucket Overflows

So far, we have assumed that, when a record is inserted, the bucket to which it is mapped has space to store the record. If the bucket does not have enough space, a **bucket overflow** is said to occur. Bucket overflow can occur for several reasons, as we outlined in Section 14.5.

- **Insufficient buckets.** The number of buckets, which we denote n_B , must be chosen such that $n_B > n_r/f_r$, where n_r denotes the total number of records that will be stored and f_r denotes the number of records that will fit in a bucket. This designation assumes that the total number of records is known when the hash function is chosen.
- **Skew.** Some buckets are assigned more records than are others, so a bucket may overflow even when other buckets still have space. This situation is called **bucket skew**. Skew can occur for two reasons:
 1. Multiple records may have the same search key.
 2. The chosen hash function may result in nonuniform distribution of search keys.

So that the probability of bucket overflow is reduced, the number of buckets is chosen to be $(n_r/f_r) * (1 + d)$, where d is a fudge factor, typically around 0.2. Some space is wasted: About 20 percent of the space in the buckets will be empty. But the benefit is that the probability of overflow is reduced.

Despite allocation of a few more buckets than required, bucket overflow can still occur. As we saw in Section 14.5, we handle bucket overflow by using overflow buckets. We must also change the lookup algorithm slightly to handle overflow chaining, to look at the overflow buckets in addition to the main bucket.

The form of hash structure that we have just described is called **closed addressing** (or, less commonly, **closed hashing**). Under an alternative approach called **open addressing** (or, less commonly, **open hashing**), the set of buckets is fixed, and there are no overflow chains. Instead, if a bucket is full, the system inserts records in some other bucket in the initial set of buckets B . One policy is to use the next bucket (in cyclic order) that has space; this policy is called *linear probing*. Other policies, such as computing further hash functions, are also used. Open addressing has been used to construct symbol tables for compilers and assemblers, but closed addressing is preferable for database systems. The reason is that deletion under open addressing is troublesome. Usually, compilers and assemblers perform only lookup and insertion operations on

their symbol tables. However, in a database system, it is important to be able to handle deletion as well as insertion. Thus, open addressing is of only minor importance in database implementation.

An important drawback to the form of hashing that we have described is that we must choose the hash function when we implement the system, and it cannot be changed easily thereafter if the file being indexed grows or shrinks. Since the function h maps search-key values to a fixed set B of bucket addresses, we waste space if B is made large to handle future growth of the file. If B is too small, the buckets contain records of many different search-key values, and bucket overflows can occur. As the file grows, performance suffers. We study in Section 24.5.2 how the number of buckets and the hash function can be changed dynamically.

24.5.2 Dynamic Hashing

As we have seen, the need to fix the set B of bucket addresses presents a serious problem with the static hashing technique of the previous section. Most databases grow larger over time. If we are to use static hashing for such a database, we have three classes of options:

1. Choose a hash function based on the current file size. This option will result in performance degradation as the database grows.
2. Choose a hash function based on the anticipated size of the file at some point in the future. Although performance degradation is avoided, a significant amount of space may be wasted initially.
3. Periodically reorganize the hash structure in response to file growth. Such a reorganization involves choosing a new hash function, recomputing the hash function on every record in the file, and generating new bucket assignments. This reorganization is a massive, time-consuming operation. Furthermore, it is necessary to forbid access to the file during reorganization.

Several **dynamic hashing** techniques allow the hash function to be modified dynamically to accommodate the growth or shrinkage of the database. In this section we describe one form of dynamic hashing, called **extendable hashing**. The bibliographical notes provide references to other forms of dynamic hashing.

24.5.2.1 Data Structure

Extendable hashing copes with changes in database size by splitting and coalescing buckets as the database grows and shrinks. As a result, space efficiency is retained. Moreover, since the reorganization is performed on only one bucket at a time, the resulting performance overhead is acceptably low.

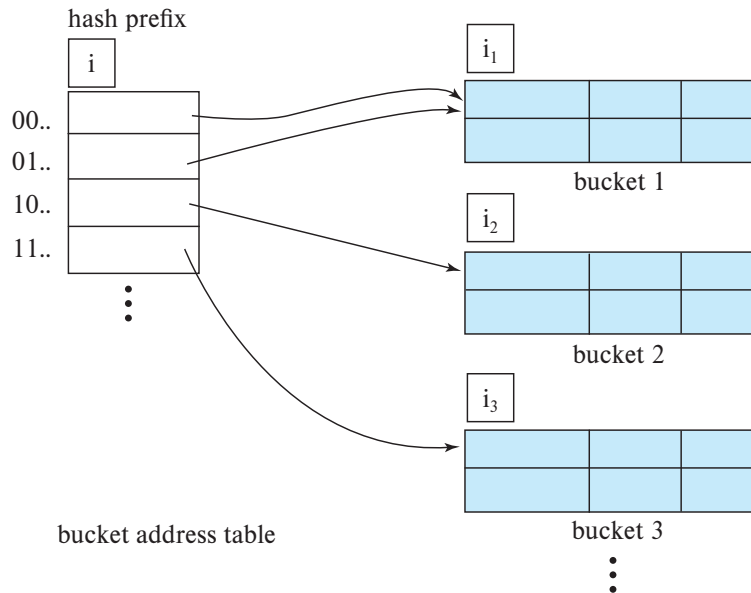


Figure 24.8 General extendable hash structure.

With extendable hashing, we choose a hash function h with the desirable properties of uniformity and randomness. However, this hash function generates values over a relatively large range—namely, b -bit binary integers. A typical value for b is 32.

We do not create a bucket for each hash value. Indeed, 2^{32} is over 4 billion, and that many buckets is unreasonable for all but the largest databases. Instead, we create buckets on demand, as records are inserted into the file. We do not use the entire b bits of the hash value initially. At any point, we use i bits, where $0 \leq i \leq b$. These i bits are used as an offset into an additional table of bucket addresses. The value of i grows and shrinks with the size of the database.

Figure 24.8 shows a general extendable hash structure. The i appearing above the bucket address table in the figure indicates that i bits of the hash value $h(K)$ are required to determine the correct bucket for K . This number will change as the file grows. Although i bits are required to find the correct entry in the bucket address table, several consecutive table entries may point to the same bucket. All such entries will have a common hash prefix, but the length of this prefix may be less than i . Therefore, we associate with each bucket an integer giving the length of the common hash prefix. In Figure 24.8 the integer associated with bucket j is shown as i_j . The number of bucket-address-table entries that point to bucket j is

$$2^{(i-i_j)}$$

24.5.2.2 Queries and Updates

We now see how to perform lookup, insertion, and deletion on an extendable hash structure.

To locate the bucket containing search-key value K_I , the system takes the first i high-order bits of $h(K_I)$, looks at the corresponding table entry for this bit string, and follows the bucket pointer in the table entry.

To insert a record with search-key value K_I , the system follows the same procedure for lookup as before, ending up in some bucket—say, j . If there is room in the bucket, the system inserts the record in the bucket. If, on the other hand, the bucket is full, it must split the bucket and redistribute the current records, plus the new one. To split the bucket, the system must first determine from the hash value whether it needs to increase the number of bits that it uses.

- If $i = i_j$, only one entry in the bucket address table points to bucket j . Therefore, the system needs to increase the size of the bucket address table so that it can include pointers to the two buckets that result from splitting bucket j . It does so by considering an additional bit of the hash value. It increments the value of i by 1, thus doubling the size of the bucket address table. It replaces each entry with two entries, both of which contain the same pointer as the original entry. Now two entries in the bucket address table point to bucket j . The system allocates a new bucket (bucket z) and sets the second entry to point to the new bucket. It sets i_j and i_z to i . Next, it rehashes each record in bucket j and, depending on the first i bits (remember the system has added 1 to i), either keeps it in bucket j or allocates it to the newly created bucket.

The system now reattempts the insertion of the new record. Usually, the attempt will succeed. However, if all the records in bucket j , as well as the new record, have the same hash-value prefix, it will be necessary to split a bucket again, since all the records in bucket j and the new record are assigned to the same bucket. If the hash function has been chosen carefully, it is unlikely that a single insertion will require that a bucket be split more than once, unless there are a large number of records with the same search key. If all the records in bucket j have the same search-key value, no amount of splitting will help. In such cases, overflow buckets are used to store the records, as in static hashing.

- If $i > i_j$, then more than one entry in the bucket address table points to bucket j . Thus, the system can split bucket j without increasing the size of the bucket address table. Observe that all the entries that point to bucket j correspond to hash prefixes that have the same value on the leftmost i_j bits. The system allocates a new bucket (bucket z), and sets i_j and i_z to the value that results from adding 1 to the original i_j value. Next, the system needs to adjust the entries in the bucket address table that previously pointed to bucket j . (Note that with the new value for i_j , not all the entries correspond to hash prefixes that have the same value on the leftmost i_j bits.) The system leaves the first half of the entries as they were (pointing to bucket

<i>dept_name</i>	$h(\text{dept_name})$
Biology	0010 1101 1111 1011 0010 1100 0011 0000
Comp. Sci.	1111 0001 0010 0100 1001 0011 0110 1101
Elec. Eng.	0100 0011 1010 1100 1100 0110 1101 1111
Finance	1010 0011 1010 0000 1100 0110 1001 1111
History	1100 0111 1110 1101 1011 1111 0011 1010
Music	0011 0101 1010 0110 1100 1001 1110 1011
Physics	1001 1000 0011 1111 1001 1100 0000 0001

Figure 24.9 Hash function for *dept_name*.

j), and sets all the remaining entries to point to the newly created bucket (bucket z). Next, as in the previous case, the system rehashes each record in bucket j , and allocates it either to bucket j or to the newly created bucket z .

The system then reattempts the insert. In the unlikely case that it again fails, it applies one of the two cases, $i = i_j$ or $i > i_j$, as appropriate.

Note that, in both cases, the system needs to recompute the hash function on only the records in bucket j .

To delete a record with search-key value K_j , the system follows the same procedure for lookup as before, ending up in some bucket—say, j . It removes both the search key from the bucket and the record from the file. The bucket, too, is removed if it becomes empty. Note that, at this point, several buckets can be coalesced, and the size of the bucket address table can be cut in half. The procedure for deciding on which buckets can be coalesced and how to coalesce buckets is left to you to do as an exercise. The conditions under which the bucket address table can be reduced in size are also left to you as an exercise. Unlike coalescing of buckets, changing the size of the bucket address table is a rather expensive operation if the table is large. Therefore it may be worthwhile to reduce the bucket-address-table size only if the number of buckets reduces greatly.

To illustrate the operation of insertion, we use the *instructor* file and assume that the search key is *dept_name* with the 32-bit hash values as appear in Figure 24.9. Assume that, initially, the file is empty, as in Figure 24.10. We insert the records one by one. To

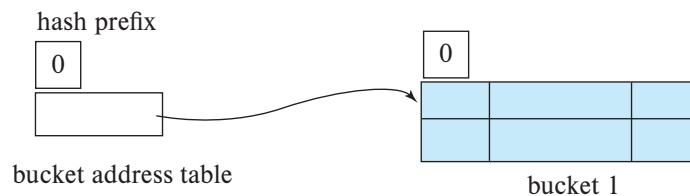


Figure 24.10 Initial extendable hash structure.

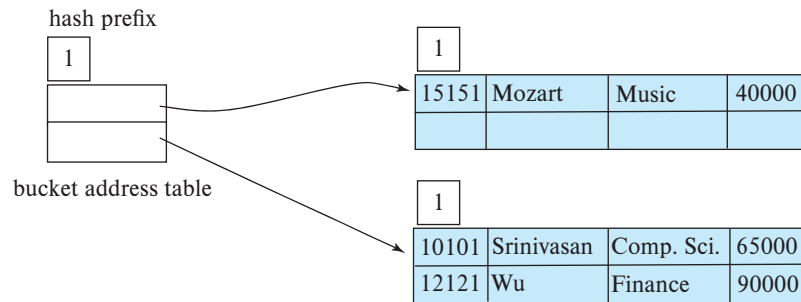


Figure 24.11 Hash structure after three insertions.

illustrate all the features of extendable hashing in a small structure, we shall make the unrealistic assumption that a bucket can hold only two records.

We insert the record (10101, Srinivasan, Comp. Sci., 65000). The bucket address table contains a pointer to the one bucket, and the system inserts the record. Next, we insert the record (12121, Wu, Finance, 90000). The system also places this record in the one bucket of our structure.

When we attempt to insert the next record (15151, Mozart, Music, 40000), we find that the bucket is full. Since $i = i_0$, we need to increase the number of bits that we use from the hash value. We now use 1 bit, allowing us $2^1 = 2$ buckets. This increase in the number of bits necessitates doubling the size of the bucket address table to two entries. The system splits the bucket, placing in the new bucket those records whose search key has a hash value beginning with 1, and leaving in the original bucket the other records. Figure 24.11 shows the state of our structure after the split.

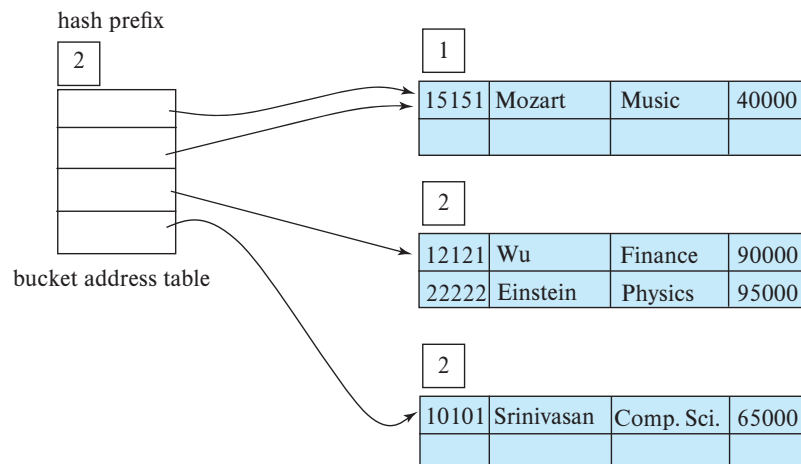


Figure 24.12 Hash structure after four insertions.

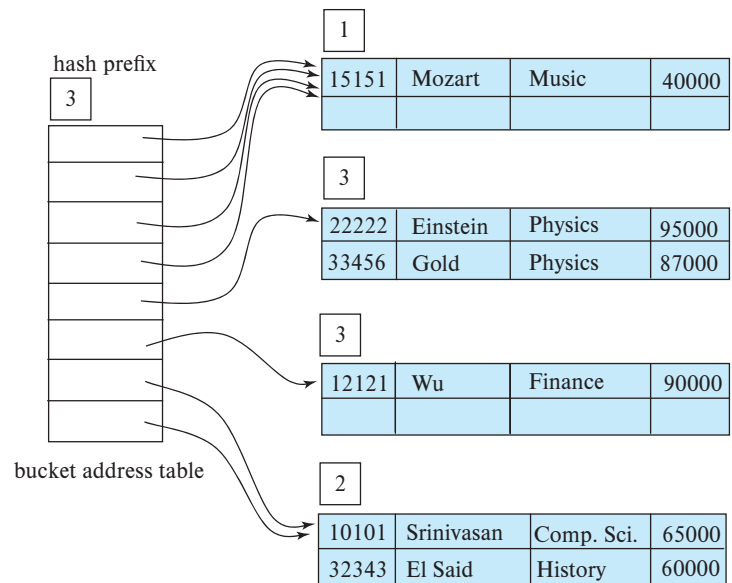


Figure 24.13 Hash structure after six insertions.

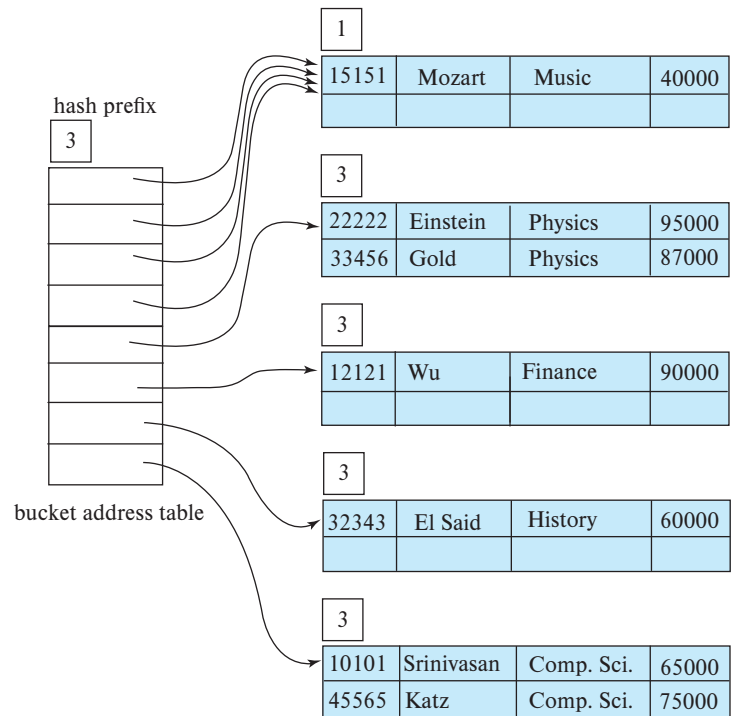


Figure 24.14 Hash structure after seven insertions.

Next, we insert (22222, Einstein, Physics, 95000). Since the first bit of $h(\text{Physics})$ is 1, we must insert this record into the bucket pointed to by the “1” entry in the bucket address table. Once again, we find the bucket full and $i = i_1$. We increase the number of bits that we use from the hash to 2. This increase in the number of bits necessitates doubling the size of the bucket address table to four entries, as in Figure 24.12. Since the bucket of Figure 24.11 for hash prefix 0 was not split, the two entries of the bucket address table of 00 and 01 both point to this bucket.

For each record in the bucket of Figure 24.11 for hash prefix 1 (the bucket being split), the system examines the first two bits of the hash value to determine which bucket of the new structure should hold it.

Next, we insert (32343, El Said, History, 60000), which goes in the same bucket as Comp. Sci. The following insertion of (33456, Gold, Physics, 87000) results in a bucket overflow, leading to an increase in the number of bits and a doubling of the size of the bucket address table (see Figure 24.13).

The insertion of (45565, Katz, Comp. Sci., 75000) leads to another bucket overflow; this overflow, however, can be handled without increasing the number of bits, since the bucket in question has two pointers pointing to it (see Figure 24.14).

Next, we insert the records of “Califieri”, “Singh”, and “Crick” without any bucket overflow. The insertion of the third Comp. Sci. record (83821, Brandt, Comp. Sci.,

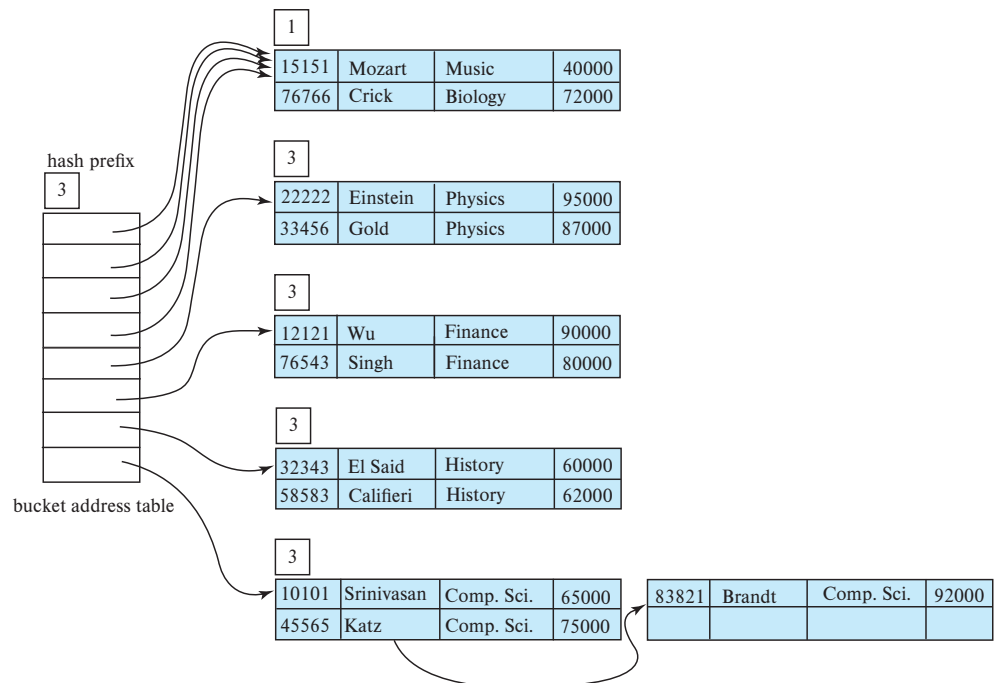


Figure 24.15 Hash structure after 11 insertions.

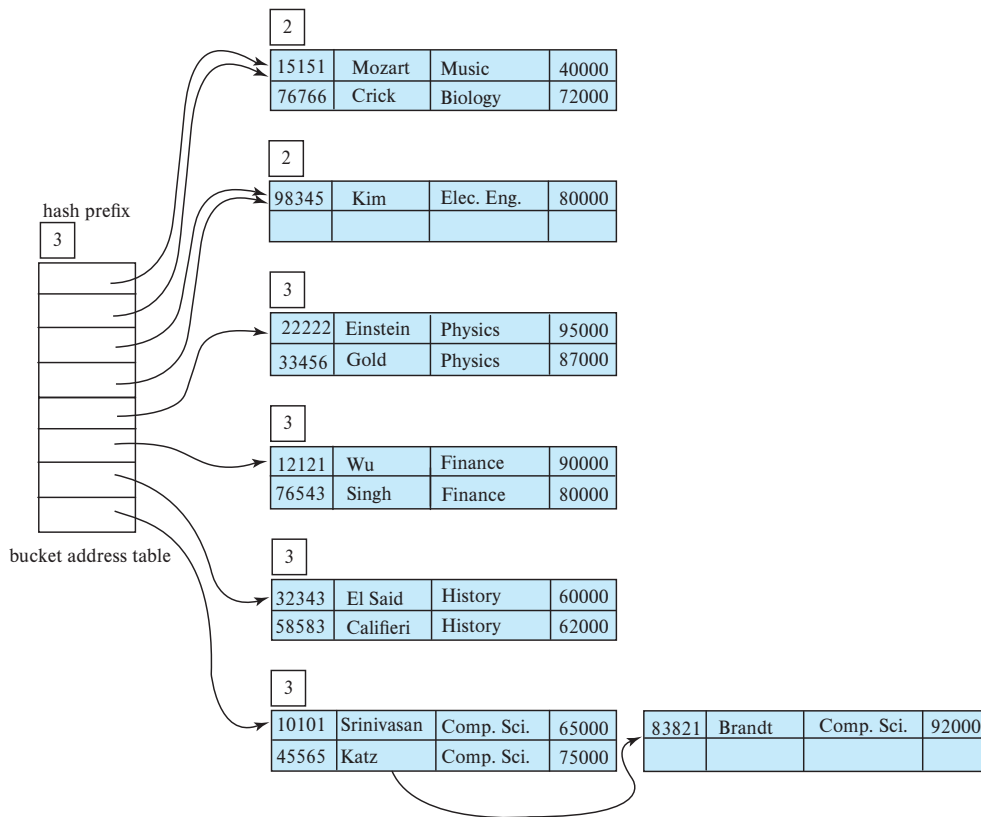


Figure 24.16 Extendable hash structure for the *instructor* file.

92000), however, leads to another overflow. This overflow cannot be handled by increasing the number of bits, since there are three records with exactly the same hash value. Hence the system uses an overflow bucket, as in Figure 24.15. We continue in this manner until we have inserted all the *instructor* records of Figure 14.1. The resulting structure appears in Figure 24.16.

24.5.2.3 Static Hashing versus Dynamic Hashing

We now examine the advantages and disadvantages of extendable hashing, compared with static hashing. The main advantage of extendable hashing is that performance does not degrade as the file grows. Furthermore, there is minimal space overhead. Although the bucket address table incurs additional overhead, it contains one pointer for each hash value for the current prefix length. This table is thus small. The main space saving of extendable hashing over other forms of hashing is that no buckets need to be reserved for future growth; rather, buckets can be allocated dynamically.

A disadvantage of extendable hashing is that lookup involves an additional level of indirection, since the system must access the bucket address table before accessing the

bucket itself. This extra reference has only a minor effect on performance. Although the hash structures that we discussed in Section 24.5.1 do not have this extra level of indirection, they lose their minor performance advantage as they become full. A further disadvantage of extendable hashing is the cost of periodic doubling of the bucket address table.

The bibliographical notes also provide references to another form of dynamic hashing called **linear hashing**, which avoids the extra level of indirection associated with extendable hashing, at the possible cost of more overflow buckets.

24.5.3 Comparison of Ordered Indexing and Hashing

We have seen several ordered-indexing schemes and several hashing schemes. We can organize files of records as ordered files by using index-sequential organization or B⁺-tree organizations. Alternatively, we can organize the files by using hashing. Finally, we can organize them as heap files, where the records are not ordered in any particular way.

Each scheme has advantages in certain situations. A database-system implementor could provide many schemes, leaving the final decision of which schemes to use to the database designer. However, such an approach requires the implementor to write more code, adding both to the cost of the system and to the space that the system occupies.

Most database systems support B⁺-trees for indexing disk-based data, and many databases also support B⁺-tree file organization. However, most databases do not support hash file organizations or hash indices for disk-based data. One of the important reasons is the fact that many applications benefit from support for range queries. A second reason is the fact that B⁺-tree indices handle relation size increases gracefully, via a series of node splits, each of which is of low cost, in contrast to the relatively high cost of doubling of the bucket address table, which extendable hashing requires. Another reason for preferring B⁺-trees is the fact that B⁺-trees give good worst-case bounds for deletion operations with duplicate keys, unlike hash indices.

However, hash indices are used for in-memory indexing, if range queries are not common. In particular, they are widely used for creating temporary in-memory indices while processing join operations using the hash-join technique, as we see in Section 15.5.5.

24.6 Summary

- The key idea of the log structured merge tree is to replace random I/O operations during tree inserts, updates, and deletes with a smaller number of sequential I/O operations.
- Bitmap indices are specialized indices designed for easy querying on multiple keys. Bitmaps work best for attributes that take only a small number of distinct values.

- A **bitmap** is an array of bits. In its simplest form, a **bitmap index** on the attribute A of relation r consists of one bitmap for each value that A can take. Each bitmap has as many bits as the number of records in the relation.
- Bitmap indices are useful for selections mainly when there are selections on multiple keys.
- An important use of bitmaps is to count the number of tuples satisfying a given selection. Such queries are important for data analysis.
- Indices are required for efficient access to spatial data and must efficiently support queries such as range and nearest neighbor queries.
- A **quadtree** is an alternative representation for two-dimensional data where the space is divided by a quadtree. Each node of a quadtree is associated with a rectangular region of space.
- An **R-tree** is a storage structure that is useful for indexing of objects such as points, line segments, rectangles, and other polygons. An R-tree is a balanced tree structure with the indexed objects stored in leaf nodes, much like a B^+ -tree. However, instead of a range of values, a rectangular **bounding box** is associated with each tree node.
- **Static hashing** uses hash functions in which the set of bucket addresses is fixed. Such hash functions cannot easily accommodate databases that grow significantly larger over time.
- **Dynamic hashing techniques** allow the hash function to be modified. One example is *extendable hashing*, which copes with changes in database size by splitting and coalescing buckets as the database grows and shrinks.

Review Terms

- | | |
|--|---------------------|
| • Log-structured merge tree (LSM tree) | • Region quadtrees |
| • Rolling merge | • R-tree |
| • Deletion entry | • Bounding box |
| • Stepped-merge index | • Quadratic split |
| • Write amplification | • Bucket overflow |
| • Bloom filter | • Skew |
| • Bitmap | • Closed addressing |
| • Bitmap index | • Closed hashing |
| • Existence bitmap | • Open addressing |
| • Quadtree | • Open hashing |

- Dynamic hashing
- Extendable hashing
- Linear hashing

Practice Exercises

- 24.1** Both LSM trees and buffer trees (described in Section 14.8.2) offer benefits to write-intensive workloads, compared to normal B⁺-trees, and buffer trees offer potentially better lookup performance. Yet LSM trees are more frequently used in Big Data settings. What is the most important reason for this preference?
- 24.2** Consider the optimized technique for counting the number of bits that are set in a bitmap. What are the tradeoffs in choosing a smaller versus a larger array size, keeping cache size in mind?
- 24.3** Suppose you want to store line segments in an R-tree. If a line segment is not parallel to the axes, the bounding box for it can be large, containing a large empty area.
- Describe the effect on performance of having large bounding boxes on queries that ask for line segments intersecting a given region.
 - Briefly describe a technique to improve performance for such queries and give an example of its benefit. Hint: You can divide segments into smaller pieces.
- 24.4** Give a search algorithm on an R-tree for efficiently finding the nearest neighbor to a given query point.
- 24.5** Give a recursive procedure to efficiently compute the spatial join of two relations with R-tree indices. (Hint: Use bounding boxes to check if leaf entries under a pair of internal nodes may intersect.)
- 24.6** Suppose that we are using extendable hashing on a file that contains records with the following search-key values:

2, 3, 5, 7, 11, 17, 19, 23, 29, 31

Show the extendable hash structure for this file if the hash function is $h(x) = x \bmod 8$ and buckets can hold three records.

- 24.7** Show how the extendable hash structure of Exercise 24.6 changes as the result of each of the following steps:
- Delete 11.
 - Delete 31.
 - Insert 1.

d. Insert 15.

- 24.8** Give pseudocode for deletion of entries from AVI an extendable hash structure, including details of when and how to coalesce buckets. Do not bother about reducing the size of the bucket address table.
- 24.9** Suggest an efficient way to test if the bucket address table in extendable hashing can be reduced in size by storing an extra count with the bucket address table. Give details of how the count should be maintained when buckets are split, coalesced, or deleted. (*Note:* Reducing the size of the bucket address table is an expensive operation, and subsequent inserts may cause the table to grow again. Therefore, it is best not to reduce the size as soon as it is possible to do so, but instead do it only if the number of index entries becomes small compared to the bucket-address-table size.)

Exercises

- 24.10** The stepped merge variant of the LSM tree allows multiple trees per level. What are the tradeoffs in having more trees per level?
- 24.11** Suppose you want to use the idea of a quadtree for data in three dimensions. How would the resultant data structure (called an *octtree*) divide up space?
- 24.12** Explain the distinction between closed and open hashing. Discuss the relative merits of each technique in database applications.
- 24.13** What are the causes of bucket overflow in a hash file organization? What can be done to reduce the occurrence of bucket overflows?
- 24.14** Why is a hash structure not the best choice for a search key on which range queries are likely?
- 24.15** Our description of static hashing assumes that a large contiguous stretch of disk blocks can be allocated to a static hash table. Suppose you can allocate only C contiguous blocks. Suggest how to implement the hash table, if it can be much larger than C blocks. Access to a block should still be efficient.

Further Reading

The log-structured merge (LSM) tree is presented in [O’Neil et al. (1996)], while the stepped merge tree is presented in [Jagadish et al. (1997)]. [Vitter (2001)] provides an extensive survey of external-memory data structures and algorithms.

Bitmap indices are described in [O’Neil and Quass (1997)]. They were first introduced in the IBM Model 204 file manager on the AS 400 platform. They provide very

large speedups on certain types of queries and are today implemented in most database systems.

[Samet (2006)] provides a textbook coverage of spatial data structures. [Bentley (1975)] describes the k-d tree, and [Robinson (1981)] describes the k-d-B tree. The R-tree was originally presented in [Guttman (1984)].

Discussions of the basic data structures in hashing can be found in [Cormen et al. (2009)]. Extendable hashing was introduced by [Fagin et al. (1979)]. Linear hashing was introduced by [Litwin (1978)] and [Litwin (1980)].

Bibliography

- [Bentley (1975)] J. L. Bentley, “Multidimensional Binary Search Trees Used for Associative Searching”, *Communications of the ACM*, Volume 18, Number 9 (1975), pages 509–517.
- [Cormen et al. (2009)] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd edition, MIT Press (2009).
- [Fagin et al. (1979)] R. Fagin, J. Nievergelt, N. Pippenger, and H. R. Strong, “Extendible Hashing – A Fast Access Method for Dynamic Files”, *ACM Transactions on Database Systems*, Volume 4, Number 3 (1979), pages 315–344.
- [Guttman (1984)] A. Guttman, “R-Trees: A Dynamic Index Structure for Spatial Searching”, In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1984), pages 47–57.
- [Jagadish et al. (1997)] H. V. Jagadish, P. P. S. Narayan, S. Seshadri, S. Sudarshan, and R. Kanneganti, “Incremental Organization for Data Recording and Warehousing”, In *Proceedings of the 23rd International Conference on Very Large Data Bases, VLDB '97* (1997), pages 16–25.
- [Litwin (1978)] W. Litwin, “Virtual Hashing: A Dynamically Changing Hashing”, In *Proc. of the International Conf. on Very Large Databases* (1978), pages 517–523.
- [Litwin (1980)] W. Litwin, “Linear Hashing: A New Tool for File and Table Addressing”, In *Proc. of the International Conf. on Very Large Databases* (1980), pages 212–223.
- [O’Neil and Quass (1997)] P. O’Neil and D. Quass, “Improved Query Performance with Variant Indexes”, In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1997), pages 38–49.
- [O’Neil et al. (1996)] P. O’Neil, E. Cheng, D. Gawlick, and E. O’Neil, “The Log-structured Merge-tree (LSM-tree)”, *Acta Inf.*, Volume 33, Number 4 (1996), pages 351–385.
- [Robinson (1981)] J. Robinson, “The k-d-B Tree: A Search Structure for Large Multidimensional Indexes”, In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1981), pages 10–18.
- [Samet (2006)] H. Samet, *Foundations of Multidimensional and Metric Data Structures*, Morgan Kaufmann (2006).

[Vitter (2001)] J. S. Vitter, “External Memory Algorithms and Data Structures: Dealing with Massive Data”, *ACM Computing Surveys*, Volume 33, (2001), pages 209–271.

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.

CHAPTER 25



Advanced Application Development

There are a number of tasks in application development. We saw in Chapter 6 to Chapter 9 how to design and build an application. One of the aspects of application design is the performance one expects out of the application. In fact, it is common to find that once an application has been built, it runs slower than the designers wanted or handles fewer transactions per second than they required. An application that takes an excessive amount of time to perform requested actions can cause user dissatisfaction at best and be completely unusable at worst.

Applications can be made to run significantly faster by performance tuning, which consists of finding and eliminating bottlenecks and adding appropriate hardware such as memory or disks. There are many things an application developer can do to tune the application, and there are things that a database-system administrator can do to speed up processing for an application.

Benchmarks are standardized sets of tasks that help to characterize the performance of database systems. They are useful to get a rough idea of the hardware and software requirements of an application, even before the application is built.

Applications must be tested as they are being developed. Testing requires generation of database states and test inputs, and verifying that the outputs match the expected outputs. We discuss issues in application testing. Legacy systems are application systems that are outdated and usually based on older-generation technology. However, they are often at the core of organizations and run mission-critical applications. We outline issues in interfacing with and issues in migrating away from legacy systems, replacing them with more modern systems.

Standards are very important for application development, especially in the age of the internet, since applications need to communicate with each other to perform useful tasks. A variety of standards have been proposed that affect database-application development, which we outline in this chapter. Organizations often store information about users in directory systems. Applications often use such directory systems to authenticate users and to get basic information about users, such as user categories (e.g.,

student, instructor, and so on). We briefly describe the architecture of directory systems.

25.1 Performance Tuning

Tuning the performance of a system involves adjusting various parameters and design choices to improve its performance for a specific application. Various aspects of a database-system design—ranging from high-level aspects such as the schema and transaction design to database parameters such as buffer sizes, down to hardware issues such as number of disks—affect the performance of an application. Each of these aspects can be adjusted so that performance is improved.

25.1.1 Motivation for Tuning

Applications sometimes exhibit poor performance, with queries taking a long time to complete, leading to users being unable to carry out tasks that they need to do. We describe a few real-world examples that we have seen, including their causes and how tuning fixed the problems.

In one of the applications, we found that users were experiencing long delays and time-outs in the web applications. On monitoring the database, we found that the CPU usage was very high, with negligible disk and network usage. Further analysis of queries running on the database showed that a simple lookup query on a large relation was using a full relation scan, which was quite expensive. Adding an index to the attribute used in the lookup drastically reduced the execution time of the query and a key performance problem vanished immediately.

In a second application, we found that a query had very poor performance. Examining the query, we found that the programmer had written an unnecessarily complicated query, with several nested subqueries, and the optimizer produced a bad plan for the query, as we realized after observing the query plan. To fix the problem, we rewrote the query using joins instead of nested subqueries, that is, we decorrelated the query; this change greatly reduced the execution time.

In a third application, we found that the application fetched a large number of rows from a query, and issued another database query for each row that it fetched. This resulted in a large number of separate queries being sent to the database, resulting in poor performance. It is possible to replace such a large number of queries with a single query that fetches all required data, as we see later in this section. Such a change improved the performance of the application by an order of magnitude.

In a fourth application, we found that while the application performed fine under light load during testing, it completely stopped working when subjected to heavy load when it was used by actual users. In this case, we found that in some of the interfaces, programmers had forgotten to close JDBC connections. Databases typically support only a limited number of JDBC connections, and once that limit was reached, the application was unable to connect to the database, and thus it stopped working.

Ensuring that connections were closed fixed this problem. While this was technically a bug fix, not a tuning action, we thought it is a good idea to highlight this problem since we have found many applications have this problem. Connection pooling, which keeps database connections open for use by subsequent transactions, is a related application tuning optimization, since it avoids the cost of repeated opening and closing of database connections.

It is also worth pointing out that in several cases above the performance problems did not show up during testing, either because the test database was much smaller than the actual database size or because the testing was done with a much lighter load (number of concurrent users) than the load on the live system. It is important that performance testing be done on realistic database sizes, with realistic load, so problems show up during testing, rather than on a live system.

25.1.2 Location of Bottlenecks

The performance of most systems (at least before they are tuned) is usually limited primarily by the performance of one or a few components, called **bottlenecks**. For instance, a program may spend 80 percent of its time in a small loop deep in the code, and the remaining 20 percent of the time on the rest of the code; the small loop then is a bottleneck. Improving the performance of a component that is not a bottleneck does little to improve the overall speed of the system; in the example, improving the speed of the rest of the code cannot lead to more than a 20 percent improvement overall, whereas improving the speed of the bottleneck loop could result in an improvement of nearly 80 percent overall, in the best case.

Hence, when tuning a system, we must first try to discover what the bottlenecks are and then eliminate them by improving the performance of system components causing the bottlenecks. When one bottleneck is removed, it may turn out that another component becomes the bottleneck. In a well-balanced system, no single component is the bottleneck. If the system contains bottlenecks, components that are not part of the bottleneck are underutilized, and could perhaps have been replaced by cheaper components with lower performance.

For simple programs, the time spent in each region of the code determines the overall execution time. However, database systems are much more complex, and query execution involves not only CPU time, but also disk I/O and network communication. A first step in diagnosing problems is to use monitoring tools provided by operating systems to find the usage level of the CPU, disks, and network links.

It is also important to monitor the database itself, to find out what is happening in the database system. For example, most databases provide ways to find out which queries (or query templates, where the same query is executed repeatedly with different constants) are taking up the maximum resources, such as CPU, disk I/O, or network capacity. In addition to hardware resource bottlenecks, poor performance in a database system may potentially be due to contention on locks, where transactions wait in lock

Note 25.1 DATABASE PERFORMANCE MONITORING TOOLS

Most database systems provide view relations that can be queried to monitor database system performance. For example, PostgreSQL provides view relations `pg_stat_statements` and `pgpgrowlocks` to monitor resource usage of SQL statements and lock contention respectively. MySQL supports a command `show processinfo` that can be used to monitor what transactions are currently executing and their resource usage. Microsoft SQL Server provides stored procedures `sp_monitor`, `sp_who`, and `sp_lock` to monitor system resource usage. The Oracle Database SQL Tuning Guide, available online, provides details of similar views in Oracle.

queues for a long time. Again, most databases provide mechanisms to monitor lock contention.

Monitoring tools can help detect where the bottleneck lies (such as CPU, I/O, or locks), and to locate the queries that are causing the maximum performance problems. In this chapter, we discuss a number of techniques that can be used to fix performance problems, such as adding required indices or materialized views, rewriting queries, rewriting applications, or adding hardware to improve performance.

To understand the performance of database systems better, it is very useful to model database systems as **queueing systems**. A transaction requests various services from the database system, starting from entry into a server process, disk reads during execution, CPU cycles, and locks for concurrency control. Each of these services has a queue associated with it, and small transactions may spend most of their time waiting in queues—especially in disk I/O queues—instead of executing code. Figure 25.1 illustrates some of the queues in a database system. Note that each lockable item has a separate queue in the concurrency control manager. The database system may have a single queue at the disk manager or may have separate queues for different disks in case the disks are directly controlled by the database. The transaction queue is used by the database system to control the admission of new queries when the number of requests exceeds the number of concurrent query execution tasks that the database allows.

As a result of the numerous queues in the database, bottlenecks in a database system typically show up in the form of long queues for a particular service, or, equivalently, in high utilizations for a particular service. If requests are spaced exactly uniformly, and the time to service a request is less than or equal to the time before the next request arrives, then each request will find the resource idle and can therefore start execution immediately without waiting. Unfortunately, the arrival of requests in a database system is never so uniform and is often random.

If a resource, such as a disk, has a low utilization, then when a request is made, the resource is likely to be idle, in which case the waiting time for the request will be 0. Assuming uniformly randomly distributed arrivals, the length of the queue (and

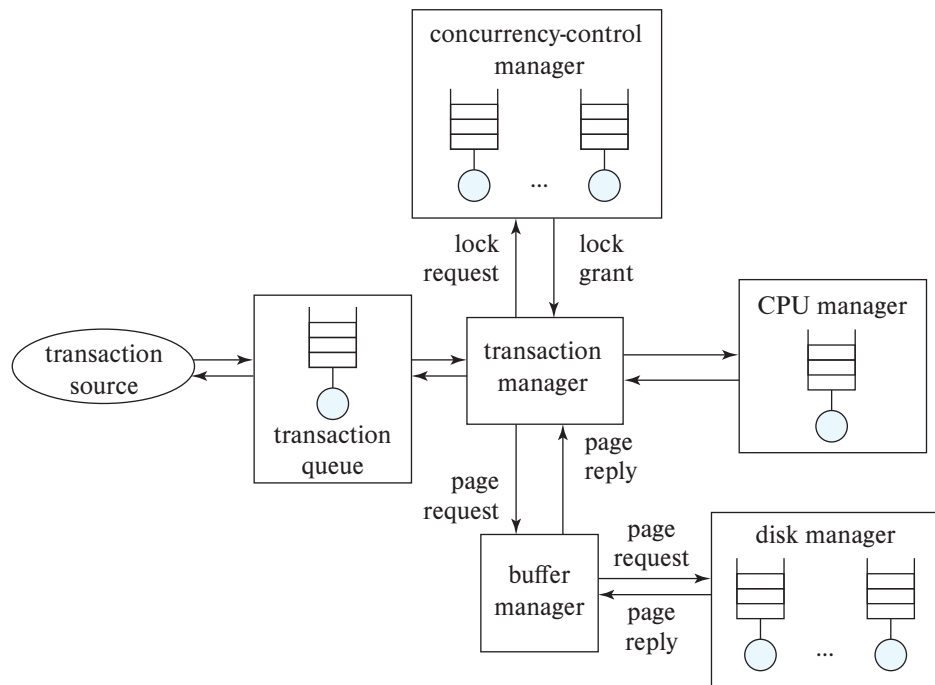


Figure 25.1 Queues in a database system.

correspondingly the waiting time) goes up exponentially with utilization; as utilization approaches 100 percent, the queue length increases sharply, resulting in excessively long waiting times. The utilization of a resource should be kept low enough that queue length is short. As a rule of the thumb, utilizations of around 70 percent are considered to be good, and utilizations above 90 percent are considered excessive, since they will result in significant delays. To learn more about the theory of queueing systems, generally referred to as [queueing theory](#), you can consult the references cited in the bibliographical notes.

25.1.3 Tuning Levels

Tuning is typically done in the context of applications, and can be done at the database system layer, or outside the database system.

Tuning at layers above the database is application dependent, and is not our focus, but we mention a few such techniques. Profiling application code to find code blocks that have a heavy CPU consumption, and rewriting them to reduce CPU load is an option for CPU intensive applications. Application servers often have numerous parameters that can be tuned to improve performance, or to ensure that the application does not run out of memory. Multiple application servers that work in parallel are often

used to handle higher workloads. A *load balancer* is used to route requests to one of the application servers; to ensure session continuity, requests from a particular source are always routed to the same application server. Connection pooling (described in Section 9.7.1) is another widely technique to reduce the overhead of database connection creation. Web application interfaces may be tuned to improve responsiveness, for example by replacing legacy web interfaces by ones based on JavaScript and Ajax (described in Section 9.5.1.3).

Returning to database tuning, database administrators and application developers can tune a database system at three levels.

The highest level of database tuning, which is under the control of application developers, includes the schema and queries. The developer can tune the design of the schema, the indices that are created, and the transactions that are executed to improve performance. Tuning at this level is comparatively system independent.

The second level consists of the database-system parameters, such as buffer size and checkpointing intervals. The exact set of database-system parameters that can be tuned depends on the specific database system. Most database-system manuals provide information on what database-system parameters can be adjusted, and how you should choose values for the parameters. Well-designed database systems perform as much tuning as possible automatically, freeing the user or database administrator from the burden. For instance, in many database systems the buffer size is fixed but tunable. If the system automatically adjusts the buffer size by observing indicators such as page-fault rates, then the database administrator will not have to worry about tuning the buffer size.

The lowest level is at the hardware level. Options for tuning systems at this level include replacing hard disks with solid-state drives (which use flash storage), adding more disks or using a RAID system if disk I/O is a bottleneck, adding more memory if the disk buffer size is a bottleneck, or moving to a system with more processors if CPU usage is a bottleneck.

The three levels of tuning interact with one another; we must consider them together when tuning a system. For example, tuning at a higher level may result in the hardware bottleneck changing from the disk system to the CPU, or vice versa. Tuning of queries and the physical schema is usually the first step to improving performance. Tuning of database system parameters, in case the database system does automate this task, can also be done in parallel. If performance is still poor, tuning of logical schema and tuning of hardware are the next logical steps.

25.1.4 Tuning of Physical Schema

Tuning of the physical schema, such as indices and materialized views, is the least disruptive mode of tuning, since it does not affect application code in any way. We now study different aspects of tuning of the physical schema.

25.1.4.1 Tuning of Indices

We can tune the indices in a database system to improve performance. If queries are the bottleneck, we can often speed them up by creating appropriate indices on relations. If updates are the bottleneck, there may be too many indices, which have to be updated when the relations are updated. Removing indices may speed up certain updates.

The choice of the type of index also is important. Some database systems support different kinds of indices, such as hash indices, B⁺-tree indices, and write-optimized indices such as LSM trees (Section 24.2). If range queries are common, B⁺-tree indices are preferable to hash indices. If the system has a very high write load, but a relatively low read load, write-optimized LSM tree indices may be preferable to B⁺-tree indices.

Whether to make an index a clustered index is another tunable parameter. Only one index on a relation can be made clustered, by storing the relation sorted on the index attributes. Generally, the index that benefits the greatest number of queries and updates should be made clustered.

To help identify what indices to create, and which index (if any) on each relation should be clustered, most commercial database systems provide *tuning wizards*; these are described in more detail in Section 25.1.4.4. These tools use the past history of queries and updates (called the *workload*) to estimate the effects of various indices on the execution time of the queries and updates in the workload. Recommendations on what indices to create are based on these estimates.

25.1.4.2 Using Materialized Views

Maintaining materialized views can greatly speed up certain types of queries, in particular aggregate queries. Recall the example from Section 16.5 where the total salary for each department (obtained by summing the salary of each instructor in the department) is required frequently. As we saw in that section, creating a materialized view storing the total salary for each department can greatly speed up such queries.

Materialized views should be used with care, however, since there is not only space overhead for storing them but, more important, there is also time overhead for maintaining materialized views. In the case of **immediate view maintenance**, if the updates of a transaction affect the materialized view, the materialized view must be updated as part of the same transaction. The transaction may therefore run slower. In the case of **deferred view maintenance**, the materialized view is updated later; until it is updated, the materialized view may be inconsistent with the database relations. For instance, the materialized view may be brought up to date when a query uses the view, or periodically. Using deferred maintenance reduces the burden on update transactions.

The database administrator is responsible for the selection of materialized views and for view-maintenance policies. The database administrator can make the selection manually by examining the types of queries in the workload and finding out which queries need to run faster and which updates/queries may be executed more slowly. From the examination, the database administrator may choose an appropriate set of

materialized views. For instance, the administrator may find that a certain aggregate is used frequently, and choose to materialize it, or may find that a particular join is computed frequently, and choose to materialize it.

However, manual choice is tedious for even moderately large sets of query types, and making a good choice may be difficult, since it requires understanding the costs of different alternatives; only the query optimizer can estimate the costs with reasonable accuracy without actually executing the query. Thus, a good set of views may be found only by trial and error—that is, by materializing one or more views, running the workload, and measuring the time taken to run the queries in the workload. The administrator repeats the process until a set of views is found that gives acceptable performance.

A better alternative is to provide support for selecting materialized views within the database system itself, integrated with the query optimizer. This approach is described in more detail in Section 25.1.4.4.

25.1.4.3 Horizontal Partitioning of Relation Schema

Horizontal partitioning of relations is widely used for parallel and distributed storage and query processing. However, it can also be used in a centralized system to improve queries and updates by breaking up the tuples of a relation into partitions.

For example, suppose that a database stores a large relation that has a *date* attribute, and most operations work on data inserted within the past few months. Suppose now that the relation is partitioned on the *date* attribute, with one partition for each (*year*, *month*) combination. Then, queries that contain a selection on *date*, such as *date*='2018-06-01', need access only partitions that could possibly contain such tuples, skipping all other partitions.

More importantly, indices could be created independently on each partition. Suppose an index is created on an attribute *ID*, with a separate index on each partition. A query that specifies selection on *ID*, along with a date or a date range, need look up the index on only those partitions that match the specified date or date range. Since each partition is smaller than the whole relation, the indices too are smaller, speeding up index lookup. Index insertion is also much faster, since the index size is much smaller than an index on the entire relation. And most importantly, even as the total data size grows, the partition size never grows beyond some limit, ensuring that the performance of such queries does not degrade with time.

There is a cost to such partitioning: queries that do not contain a selection on the partitioning attribute need to individually access each of the partitions, potentially slowing down such queries significantly. If such queries are rare, the benefits of partitioning outweigh the costs, making them an attractive technique for optimization.

Even if the database does not support partitioning internally, it is possible to replace a relation r by multiple physical relations $r_1, r_2, \dots, r_n$, and the original relation r is defined by the view $r = r_1 \cup r_2 \cup \dots \cup r_n$. Suppose that the database optimizer knows

the predicate defining each r_i (in our example, the date range corresponding to each r_i). Then the optimizer can replace a query on r that includes a selection on the partitioning attribute (*date*, in our example), with a query on the only relevant r_i s. Indices would have to be created separately on each of the r_i s.

25.1.4.4 Automated Tuning of Physical Design

Most commercial database systems today provide tools to help the database administrator with index and materialized view selection and other tasks related to physical database design such as how to partition data in a parallel database system.

These tools examine the **workload** (the history of queries and updates) and suggest indices and views to be materialized. The database administrator may specify the importance of speeding up different queries, which the tool takes into account when selecting views to materialize. Often tuning must be done before the application is fully developed, and the actual database contents may be small on the development database but are expected to be much larger on a production database. Thus, some tuning tools also allow the database administrator to specify information about the expected size of the database and related statistics.

Microsoft's Database Tuning Assistant, for example, allows the user to ask "what if" questions, whereby the user can pick a view, and the optimizer then estimates the effect of materializing the view on the total cost of the workload and on the individual costs of different types of queries and updates in the workload.

The automatic selection of indices and materialized views is usually implemented by enumerating different alternatives and using the query optimizer to estimate the costs and benefits of selecting each alternative by using the workload. Since the number of design alternatives and the potential workload may be extremely large, the selection techniques must be designed carefully.

The first step is to generate a workload. This is usually done by recording all the queries and updates that are executed during some time period. Next, the selection tools perform **workload compression**, that is, create a representation of the workload using a small number of updates and queries. For example, updates of the same form can be represented by a single update with a weight corresponding to how many times the update occurred. Queries of the same form can be similarly replaced by a representative with appropriate weight. After this, queries that are very infrequent and do not have a high cost may be discarded from consideration. The most expensive queries may be chosen to be addressed first. Such workload compression is essential for large workloads.

With the help of the optimizer, the tool would come up with a set of indices and materialized views that could help the queries and updates in the compressed workload. Different combinations of these indices and materialized views can be tried out to find the best combination. However, an exhaustive approach would be totally impractical, since the number of potential indices and materialized views is already large, and each

Note 25.2 TUNING TOOLS

Tuning tools, such as the Database Engine Tuning Advisor provided by SQL Server and the SQL Tuning Advisor of Oracle, provide recommendations such as what indices or materialized views to add, or how to partition a relation, to improve performance. These recommendations can then be accepted and implemented by a database administrator.

Auto Tuning in Microsoft Azure SQL can automatically create and drop indices to improve query performance. A risk with automatically changing the physical schema is that some queries may perform poorly. For example, an optimizer may choose a plan using a newly created index, assuming, based on wrong estimates of cost, that the new plan is cheaper than the plan used before the index was created. In reality, the query may run slower using the new plan, which may affect users. The “force last good plan” feature can monitor query performance after any change such as addition of an index, and if performance is worse, it can force the database to use the old plan before the change (as long as it is still valid).

Oracle also provides auto tuning support, for example recommending if an index should be added, or monitoring the use of a query to decide if it should be optimized for fetching only a few rows or for fetching all rows (the best plan may be very different if only the first few rows are fetched or if all rows are fetched).

subset of these is a potential design alternative, leading to an exponential number of alternatives. Heuristics are used to reduce the space of alternatives, that is, to reduce the number of combinations considered.

Greedy heuristics for index and materialized view selection operate as follows: They estimate the benefits of materializing different indices or views (using the optimizer’s cost estimation functionality as a subroutine). They then choose the index or view that gives either the maximum benefit or the maximum benefit per unit space (i.e., benefit divided by the space required to store the index or view). The cost of maintaining the index or view must be taken into account when computing the benefit. Once the heuristic has selected an index or view, the benefits of other indices or views may have changed, so the heuristic recomputes these and chooses the next best index or view for materialization. The process continues until either the available disk space for storing indices or materialized views is exhausted or the cost of maintaining the remaining candidates is more than the benefit to queries that could use the indices or views.

Real-world index and materialized-view selection tools usually incorporate some elements of greedy selection but use other techniques to get better results. They also support other aspects of physical database design, such as deciding how to partition a relation in a parallel database, or what physical storage mechanism to use for a relation.

25.1.5 Tuning of Queries

The performance of an application can often be significantly improved by rewriting queries or by changing how the application issues queries to the database.

25.1.5.1 Tuning of Query Plans

In the past, optimizers on many database systems were not particularly good, so how a query was written would have a big influence on how it was executed, and therefore on the performance. Today's advanced optimizers can transform even badly written queries and execute them efficiently, so the need for tuning individual queries is less important than it used to be. However, sometimes query optimizers choose bad plans for one of several reasons, which we describe next.

Before checking if something needs to be tuned in the plan for a query, it is useful to find out what plan is being used for the query. Most databases support an `explain` command, which allows you to see what plan is being used for a query. The `explain` command also shows the statistics that the optimizer used or computed for different parts of the query plan, and estimates of the costs of each part of a query plan. Variants of the `explain` command also execute the query and get actual tuple counts and execution time for different parts of the query plan.

Incorrect statistics are often the reason for the choice of a bad plan. For example, if the optimizer thinks that the relations involved in a join have very few tuples, it may choose nested loops join, which would be very inefficient if the relations actually have a large number of tuples.

Ideally, database statistics should be updated whenever relations are updated. However, doing so adds unacceptable overhead to update queries. Instead, databases either periodically update statistics or leave it to the system administrator to issue a command to update statistics. Some databases, such as PostgreSQL and MySQL support a command called `analyze`,¹ which can be used to recompute statistics. For example, `analyze instructor` would recompute statistics for the `instructor` relation, while `analyze` with no arguments would recompute statistics for all relations in PostgreSQL. It is highly recommended to run this command after loading data into the database, or after making a significant number of inserts or deletes on a relation.

Some databases such as Oracle and Microsoft SQL Server keep track of inserts and deletes to relations, and they update statistics whenever the relation size changes by a significant fraction, making execution of the `analyze` command unnecessary.

Another reason for poor performance of queries is the lack of required indices. As we saw earlier, the choice of indices can be done as part of the tuning of the physical schema, but examining a query helps us understand what indices may be useful to speed up that query.

Indices are particularly important for queries that fetch only a few rows from a large relation, based on a predicate. For example, a query that finds students in a department

¹The command is called `analyze table` in the case of MySQL.

may benefit from an index on the *student* relation on the attribute *dept_name*. Indices on join attributes are often very useful. For example, if the above query also included a join of *student* with *takes* on the attribute *takes.ID*, an index on *takes.ID* could be useful.

Note that databases typically create indices on primary-key attributes, which can be used for selections as well as joins. For example, in our university schema, the primary-key index on *takes* has *ID* as its first attribute and may thus be useful for the above join.

Complex queries containing *nested subqueries* are not optimized very well by many optimizers. We saw techniques for nested subquery decorrelation in Section 16.4.4. If a subquery is not decorrelated, it gets executed repeatedly, potentially resulting in a great deal of random I/O. In contrast, decorrelation allows efficient set-oriented operations such as joins to be used, minimizing random I/O. Most database query optimizers incorporate some forms of decorrelation, but some can handle only very simple nested subqueries. The execution plan chosen by the optimizer can be found as described in Chapter 16. If the optimizer has not succeeded in decorrelating a nested subquery, the query can be decorrelated by rewriting it manually.

25.1.5.2 Improving Set Orientation

When SQL queries are executed from an application program, it is often the case that a query is executed frequently, but with different values for a parameter. Each call has an overhead of communication with the server, in addition to processing overheads at the server.

For example, consider a program that steps through each department, invoking an embedded SQL query to find the total salary of all instructors in the department:

```
select sum(salary)
from instructor
where dept_name= ?
```

If the *instructor* relation does not have a clustered index on *dept_name*, each such query will result in a scan of the relation. Even if there is such an index, a random I/O operation will be required for each *dept_name* value.

Instead, we can use a single SQL query to find total salary expenses of each department:

```
select dept_name, sum(salary)
from instructor
group by dept_name;
```

This query can be evaluated with a single scan of the *instructor* relation, avoiding random I/O for each department. The results can be fetched to the client side using a single round of communication, and the client program can then step through the results to find the aggregate for each department. Combining multiple SQL queries into a single

```
PreparedStatement pstmt = conn.prepareStatement(
    "insert into instructor values(?,?,?,?)");
pstmt.setString(1, "88877");
pstmt.setString(2, "Perry");
pstmt.setInt(3, "Finance");
pstmt.setInt(4, 125000);
pstmt.addBatch( );
pstmt.setString(1, "88878");
pstmt.setString(2, "Thierry");
pstmt.setInt(3, "Physics");
pstmt.setInt(4, 100000);
pstmt.addBatch( );
pstmt.executeBatch( );
```

Figure 25.2 Batch update in JDBC.

SQL query as above can reduce execution costs greatly in many cases—for example, if the *instructor* relation is very large and has a large number of departments.

The JDBC API also provides a feature called **batch update** that allows a number of inserts to be performed using a single communication with the database. Figure 25.2 illustrates the use of this feature. The code shown in the figure requires only one round of communication with the database, when the `executeBatch()` method is executed, in contrast to similar code without the batch update feature that we saw in Figure 5.2. In the absence of batch update, as many rounds of communication with the database are required as there are instructors to be inserted. The batch update feature also enables the database to process a batch of inserts at once, which can potentially be done much more efficiently than a series of single record inserts.

Another technique used widely in client-server systems to reduce the cost of communication and SQL compilation is to use stored procedures, where queries are stored at the server in the form of procedures, which may be precompiled. Clients can invoke these stored procedures rather than communicate a series of queries.

25.1.5.3 Tuning of Bulk Loads and Updates

When loading a large volume of data into a database (called a **bulk load** operation), performance is usually very poor if the inserts are carried out as separate SQL insert statements. One reason is the overhead of parsing each SQL query; a more important reason is that performing integrity constraint checks and index updates separately for each inserted tuple results in a large number of random I/O operations. If the inserts were done as a large batch, integrity-constraint checking and index update can be done

in a much more set-oriented fashion, reducing overheads greatly; performance improvements of an order of magnitude or more are not uncommon.

To support bulk load operations, most database systems provide a **bulk import** utility and a corresponding **bulk export** utility. The bulk-import utility reads data from a file and performs integrity constraint checking as well as index maintenance in a very efficient manner. Common input and output file formats supported by such bulk import/export utilities include text files with characters such as commas or tabs separating attribute values, with each record in a line of its own (such file formats are referred to as *comma-separated values* or *tab-separated values* formats). Database-specific binary formats as well as XML formats are also supported by bulk import/export utilities. The names of the bulk import/export utilities differ by database. In PostgreSQL, the utilities are called **pg_dump** and **pg_restore** (PostgreSQL also provides an SQL command **copy**, which provides similar functionality). The bulk import/export utility in Oracle is called **SQL*Loader**, the utility in DB2 is called **load**, and the utility in SQL Server is called **bcp** (SQL Server also provides an SQL command called **bulk insert**).

We now consider the case of tuning of bulk updates. Suppose we have a relation *funds_received*(*dept_name*, *amount*) that stores funds received (say, by electronic funds transfer) for each of a set of departments. Suppose now that we want to add the amounts to the balances of the corresponding department budgets. In order to use the SQL update statement to carry out this task, we have to perform a look up on the *funds_received* relation for each tuple in the *department* relation. We can use subqueries in the update clause to carry out this task, as follows: We assume for simplicity that the relation *funds_received* contains at most one tuple for each department.

```
update department set budget = budget +
    (select amount
     from funds_received
     where funds_received.dept_name = department.dept_name)
where exists(
    select *
    from funds_received
    where funds_received.dept_name = department.dept_name);
```

Note that the condition in the **where** clause of the update ensures that only accounts with corresponding tuples in *funds_received* are updated, while the subquery within the **set** clause computes the amount to be added to each such department.

There are many applications that require updates such as that illustrated above. Typically, there is a table, which we shall call the **master table**, and updates to the master table are received as a batch. Now the master table has to be correspondingly updated. SQL:2003 introduced a special construct, called the **merge** construct, to simplify the task of performing such merging of information. For example, the preceding update can be expressed using **merge** as follows:

```

merge into department as A
using      (select *
           from funds_received) as F
on (A.dept_name = F.dept_name)
when matched then
    update set budget = budget + F.amount;

```

When a record from the subquery in the **using** clause matches a record in the *department* relation, the **when matched** clause is executed, which can execute an update on the relation; in this case, the matching record in the *department* relation is updated as shown.

The **merge** statement can also have a **when not matched then** clause, which permits insertion of new records into the relation. In the preceding example, when there is no matching department for a *funds_received* tuple, the insertion action could create a new department record (with a null *building*) using the following clause:

```

when not matched then
    insert values (F.dept_name, null, F.budget)

```

Although not very meaningful in this example,² the **when not matched then** clause can be quite useful in other cases. For example, suppose the local relation is a copy of a master relation, and we receive updated as well as newly inserted records from the master relation. The **merge** statement can update matched records (these would be updated old records) and insert records that are not matched (these would be new records).

Not all SQL implementations support the **merge** statement currently; see the respective system manuals for further details.

25.1.6 Tuning of the Logical Schema

Performance of queries can sometimes be improved by tuning of the logical schema. For example, within the constraints of the chosen normal form, it is possible to partition relations vertically. Consider the *course* relation, with the schema:

course (*course_id*, *title*, *dept_name*, *credits*)

for which *course_id* is a key. Within the constraints of the normal forms (BCNF and 3NF), we can partition the *course* relation into two relations:

course_credit (*course_id*, *credits*)
course_title_dept (*course_id*, *title*, *dept_name*)

²A better action here would have been to insert these records into an error relation, but that cannot be done with the **merge** statement.

The two representations are logically equivalent, since *course_id* is a key, but they have different performance characteristics.

If most accesses to course information look at only the *course_id* and *credits*, then they can be run against the *course_credit* relation, and access is likely to be somewhat faster, since the *title* and *dept_name* attributes are not fetched. For the same reason, more tuples of *course_credit* will fit in the buffer than corresponding tuples of *course*, again leading to faster performance. This effect would be particularly marked if the *title* and *dept_name* attributes were large. Hence, a schema consisting of *course_credit* and *course_title_dept* would be preferable to a schema consisting of the *course* relation in this case.

On the other hand, if most accesses to course information require both *dept_name* and *credits*, using the *course* relation would be preferable, since the cost of the join of *course_credit* and *course_title_dept* would be avoided. Also, the storage overhead would be lower, since there would be only one relation, and the attribute *course_id* would not be replicated.

The **column store** approach to storing data are based on vertical partitioning but takes it to the limit by storing each attribute (column) of the relation in a separate file, as we saw in Section 13.6. Note that in a column store it is not necessary to repeat the primary-key attribute since the i^{th} row can be reconstructed by taking the i^{th} entry for each desired column. Column stores have been shown to perform well for several data-warehouse applications by reducing I/O, improving cache performance, enabling greater gains from data compression, and allowing effective use of CPU vector-processing capabilities.

Another trick to improve performance is to store a *denormalized relation*, such as a join of *instructor* and *department*, where the information about *dept_name*, *building*, and *budget* is repeated for every instructor. More effort has to be expended to make sure the relation is consistent whenever an update is carried out. However, a query that fetches the names of the instructors and the associated buildings will be speeded up, since the join of *instructor* and *department* will have been precomputed. If such a query is executed frequently, and has to be performed as efficiently as possible, the denormalized relation could be beneficial.

Materialized views can provide the benefits that denormalized relations provide, at the cost of some extra storage. A major advantage to materialized views over denormalized relations is that maintaining consistency of redundant data becomes the job of the database system, not the programmer. Thus, materialized views are preferable, whenever they are supported by the database system.

Another approach to speed up the computation of the join without materializing it is to cluster records that would match in the join on the same disk page. We saw such clustered file organizations in Section 13.3.3.

25.1.7 Tuning of Concurrent Transactions

Concurrent execution of different types of transactions can sometimes lead to poor performance because of contention on locks. We first consider the case of read-write

contention, which is more common, and then consider the case of write-write contention.

As an example of **read-write contention**, consider the following situation on a banking database. During the day, numerous small update transactions are executed almost continuously. Suppose that a large query that computes statistics on branches is run at the same time. If the query performs a scan on a relation, it may block out all updates on the relation while it runs, and that can have a disastrous effect on the performance of the system.

Several database systems—Oracle, PostgreSQL, and Microsoft SQL Server, for example— support snapshot isolation, whereby queries are executed on a snapshot of the data, and updates can go on concurrently. (Snapshot isolation is described in detail in Section 18.8.) Snapshot isolation should be used, if available, for large queries, to avoid lock contention in the above situation. In SQL Server, executing the statement

set transaction isolation level snapshot

at the beginning of a transaction results in snapshot isolation being used for that transaction. In Oracle and PostgreSQL, using the keyword **serializable** in place of the keyword **snapshot** in the above command has the same effect, since these systems actually use snapshot isolation (serializable snapshot isolation, in the case of PostgreSQL version 9.1 onwards) when the isolation level is set to serializable.

If snapshot isolation is not available, an alternative option is to execute large queries at times when updates are few or nonexistent. However, for databases supporting web sites, there may be no such quiet period for updates.

Another alternative is to use weaker levels of consistency, such as the **read committed** isolation level, whereby evaluation of the query has a minimal impact on concurrent updates, but the query results are not guaranteed to be consistent. The application semantics determine whether approximate (inconsistent) answers are acceptable.

We now consider the case of **write-write contention**. Data items that are updated very frequently can result in poor performance with locking, with many transactions waiting for locks on those data items. Such data items are called **update hot spots**. Update hot spots can cause problems even with snapshot isolation, causing frequent transaction aborts due to write-validation failures. A commonly occurring situation that results in an update hot spot is as follows: transactions need to assign unique identifiers to data items being inserted into the database, and to do so they read and increment a sequence counter stored in a tuple in the database. If inserts are frequent, and the sequence counter is locked in a two-phase manner, the tuple containing the sequence counter becomes a hot spot.

One way to improve concurrency is to release the lock on the sequence counter immediately after it is read and incremented; however, after doing so, even if the transaction aborts, the update to the sequence counter should not be rolled back. To understand why, suppose T_1 increments the sequence counter, and then T_2 increments the sequence counter before T_1 commits; if T_1 then aborts, rolling back its update, either

by restoring the counter to the original value or by decrementing the counter, will result in the sequence value used by T_2 getting reused by a subsequent transaction.

Most databases provide a special construct for creating **sequence counters** that implement early, non-two-phase lock release, coupled with special-case treatment of undo logging so that updates to the counter are not rolled back if the transaction aborts. The SQL standard allows a sequence counter to be created using the command:

```
create sequence counter1;
```

In the above command, *counter1* is the name of the sequence; multiple sequences can be created with different names. The syntax to get a value from the sequence is not standardized; in Oracle, *counter1.nextval* would return the next value of the sequence, after incrementing it, while the function call *nextval ('counter1')* would have the same effect in PostgreSQL, and DB2 uses the syntax **nextval for** *counter1*.

The SQL standard provides an alternative to using an explicit sequence counter, which is useful when the goal is to give unique identifiers to tuples inserted into a relation. To do so, the keyword **identity** can be added to the declaration of an integer attribute of a relation (usually this attribute would also be declared as the primary key). If the value for that attribute is left unspecified in an insert statement for that relation, a unique new value is created automatically for each newly inserted tuple. A non-two-phase locked sequence counter is used internally to implement the **identity** declaration, with the counter incremented each time a tuple is inserted. Several databases, including DB2 and SQL Server support the **identity** declaration, although the syntax varies. PostgreSQL supports a data type called **serial**, which provides the same effect; the PostgreSQL type **serial** is implemented by transparently creating a non-two-phase locked sequence.

It is worth noting that since the acquisition of a sequence number by a transaction cannot be rolled back if the transaction aborts (for reasons discussed earlier), transaction aborts may result in *gaps in the sequence numbers* in tuples inserted in the database. For example, there may be tuples with identifier value 1001 and 1003, but no tuple with value 1002, if the transaction that acquired the sequence number 1002 did not commit. Such gaps are not acceptable in some applications; for example, some financial applications require that there be no gaps in bill or receipt numbers. Database provided sequences and automatically incremented attributes should not be used for such applications, since they can result in gaps. A sequence counter stored in normal tuples, which is locked in a two-phase manner, would not be susceptible to such gaps since a transaction abort would restore the sequence counter value, and the next transaction would get the same sequence number, avoiding a gap.

Long update transactions can cause performance problems with system logs and can increase the time taken to recover from system crashes. If a transaction performs many updates, the system log may become full even before the transaction completes, in which case the transaction will have to be rolled back. If an update transaction runs for a long time (even with few updates), it may block deletion of old parts of the log,

if the logging system is not well designed. Again, this blocking could lead to the log getting filled up.

To avoid such problems, many database systems impose strict limits on the number of updates that a single transaction can carry out. Even if the system does not impose such limits, it is often helpful to break up a large update transaction into a set of smaller update transactions where possible. For example, a transaction that gives a raise to every employee in a large corporation could be split up into a series of small transactions, each of which updates a small range of employee-ids. Such transactions are called **minibatch transactions**. However, minibatch transactions must be used with care. First, if there are concurrent updates on the set of employees, the result of the set of smaller transactions may not be equivalent to that of the single large transaction. Second, if there is a failure, the salaries of some of the employees would have been increased by committed transactions, but salaries of other employees would not. To avoid this problem, as soon as the system recovers from failure, we must execute the transactions remaining in the batch.

Long transactions, whether read-only or update, can also result in the lock table becoming full. If a single query scans a large relation, the query optimizer would ensure that a relation lock is obtained instead of acquiring a large number of tuple locks. However, if a transaction executes a large number of small queries or updates, it may acquire a large number of locks, resulting in the lock table becoming full.

To avoid this problem, some databases provide for automatic **lock escalation**; with this technique, if a transaction has acquired a large number of tuple locks, tuple locks are upgraded to page locks, or even full relation locks. Recall that with multiple-granularity locking (Section 18.3), once a coarser-level lock is obtained, there is no need to record finer-level locks, so tuple lock entries can be removed from the lock table, freeing up space. On databases that do not support lock escalation, it is possible for the transaction to explicitly acquire a relation lock, thereby avoiding the acquisition of tuple locks.

25.1.8 Tuning of Hardware

Hardware bottlenecks could include memory, I/O, CPU and network capacity. We focus on memory and I/O tuning in this section. The availability of processors with a large number of CPU cores, and support for multiple CPUs on a single machine allows system designers to choose the CPU model and number of CPUs to meet the CPU requirements of the application at an acceptable cost. How to tune or choose between CPU and network interconnect options is a topic outside the domain of database tuning.

Even in a well-designed transaction processing system, each transaction usually has to do at least a few I/O operations, if the data required by the transaction are on disk. An important factor in tuning a transaction processing system is to make sure that the disk subsystem can handle the rate at which I/O operations are required. For instance, consider a hard disk that supports an access time of about 10 milliseconds, and average transfer rate of 25 to 100 megabytes per second (a fairly typical disk today). Such a disk

would support a little under 100 random-access I/O operations of 4 kilobytes each per second. If each transaction requires just two I/O operations, a single disk would support at most 50 transactions per second.

An obvious way to improve performance is to replace a hard disk with a solid-state drive (SSD), since a single SSD can support tens of thousands of random I/O operations per second. A drawback of using SSDs is that they cost a lot more than hard disks for a given storage capacity. Another way to support more transactions per second is to increase the number of disks. If the system needs to support n transactions per second, each performing two I/O operations, data must be striped (or otherwise partitioned) across at least $n/50$ hard disks (ignoring skew), or $n/5000$ SSDs, if the SSD supports 10,000 random I/O operations per second.

Notice here that the limiting factor is not the capacity of the disk, but the speed at which random data can be accessed (limited in a hard disk by the speed at which the disk arm can move). The number of I/O operations per transaction can be reduced by storing more data in memory. If all data are in memory, there will be no disk I/O except for writes. Keeping frequently used data in memory reduces the number of disk I/Os and is worth the extra cost of memory. Keeping very infrequently used data in memory would be a waste, since memory is much more expensive than disk.

The question is, for a given amount of money available for spending on disks or memory, what is the best way to spend the money to achieve the maximum number of transactions per second? A reduction of one I/O per second saves:

$$(price\ per\ disk\ drive)/(access\ per\ second\ per\ disk)$$

Thus, if a particular page is accessed once in m seconds, the saving due to keeping it in memory is $\frac{1}{m}$ times the above value. Storing a page in memory costs:

$$(price\ per\ megabyte\ of\ memory)/(pages\ per\ megabyte\ of\ memory)$$

Thus, the break-even point is:

$$\frac{1}{m} * \frac{price\ per\ disk\ drive}{access\ per\ second\ per\ disk} = \frac{price\ per\ megabyte\ of\ memory}{pages\ per\ megabyte\ of\ memory}$$

We can rearrange the equation and substitute current values for each of the above parameters to get a value for m ; if a page is accessed more frequently than once in m seconds, it is worth buying enough memory to store it.

As of 2018, hard-disk technology and memory and disk prices (which we assume to be about \$50 for a 1-terabyte disk and \$80 for 16-gigabytes of memory) give a value of m around 4 hours for 4-kilobytes pages that are randomly accessed; that is, if a page on hard disk is accessed at least once in 4 hours, it makes sense to purchase enough memory to cache it in memory. Note that if we use larger pages, the time decreases; for example, a page size of 16-kilobytes will lead to a value of m of 1 hour instead of 4 hours.

With disk and memory cost and speeds as of the 1980/1990s, the corresponding value was 5 minutes with 4-kilobytes pages. Thus, a widely used rule of thumb, called the **five minute rule**, which said that data should be cached in memory if it is accessed more frequently than once in 5 minutes.

With SSD technology and prices as of 2018 (which we assume to be around \$500 for a 800 gigabytes SSD, which supports 67,000 random reads and 20,000 random writes per second), if we make the same comparison between keeping a page in memory versus fetching it from SSD, the time comes to around 7 minutes with 4-kilobyte pages. That is, if a page on SSD is accessed more frequently than once in 7 minutes, it is worth purchasing enough memory to cache it in memory.

For data that are sequentially accessed, significantly more pages can be read per second. Assuming 1 megabyte of data are read at a time, the breakeven point for hard disk currently is about 2.5 minutes. Thus, sequentially accessed data on hard disk should be cached in memory if they are used at least once in 2.5 minutes. For SSDs, the breakeven point is much smaller, at 1.6 seconds. In other words, there is little benefit in caching sequentially accessed data in memory unless it is very frequently accessed.

The above rules of thumb take only the number of I/O operations per second into account and do not consider factors such as response time. Some applications need to keep even infrequently used data in memory to support response times that are less than or comparable to disk-access time.

Since SSD storage is more expensive than disk, one way to get faster random I/O for frequently used data, while paying less for storing less frequently used data, is to use the **flash-as-buffer** approach. In this approach, flash storage is used as a persistent buffer, with each block having a permanent location on disk, but stored in flash instead of being written to disk as long as it is frequently used. When flash storage is full, a block that is not frequently used is evicted and flushed back to disk if it was updated after being read from disk. Disk subsystems that provide hard disks along with SSDs that act as buffers are commercially available. A rule of thumb for deciding how much SSD storage to purchase is that a 4-kilobyte page should be kept on SSD, instead of hard disk, if it is accessed more frequently than once in a day (the computation is similar to the case of caching in main memory versus fetching from disk/SSD). Note that in such a setup, the database system cannot control what data reside in which part of the storage.

If the storage system allows direct access to SSDs as well as hard disks, the database administrator can control the mapping of relations or indices to disks and allocate frequently used relations/indices to flash storage. The tablespace feature, supported by most database systems, can be used to control the mapping by creating a tablespace on flash storage and assigning desired relations and indices to that tablespace. Controlling the mapping at a finer level of granularity than a relation, however, requires changes to the database-system code.

Another aspect of tuning is whether to use RAID 1 or RAID 5. The answer depends on how frequently the data are updated, since RAID 5 is much slower than RAID 1 on

random writes: RAID 5 requires 2 reads and 2 writes to execute a single random write request. If an application performs r random reads and w random writes per second to support a particular throughput rate, a RAID 5 implementation would require $r + 4w$ I/O operations per second, whereas a RAID 1 implementation would require $r + 2w$ I/O operations per second. We can then calculate the number of disks required to support the required I/O operations per second by dividing the result of the calculation by 100 I/O operations per second (for current-generation disks). For many applications, r and w are large enough that the $(r + w)/100$ disks can easily hold two copies of all the data. For such applications, if RAID 1 is used, the required number of disks is actually less than the required number of disks if RAID 5 is used! Thus, RAID 5 is useful only when the data storage requirements are very large, but the update rates, and particularly random update rates, are small.

25.1.9 Performance Simulation

To test the performance of a database system even before it is installed, we can create a performance-simulation model of the database system. Each service shown in Figure 25.1, such as the CPU, each disk, the buffer, and the concurrency control, is modeled in the simulation. Instead of modeling details of a service, the simulation model may capture only some aspects of each service, such as the **service time**—that is, the time taken to finish processing a request once processing has begun. Thus, the simulation can model a disk access from just the average disk-access time.

Since requests for a service generally have to wait their turn, each service has an associated queue in the simulation model. A transaction consists of a series of requests. The requests are queued up as they arrive and are serviced according to the policy for that service, such as first come, first served. The models for services such as CPU and the disks conceptually operate in parallel, to account for the fact that these subsystems operate in parallel in a real system.

Once the simulation model for transaction processing is built, the system administrator can run a number of experiments on it. The administrator can use experiments with simulated transactions arriving at different rates to find how the system would behave under various load conditions. The administrator could run other experiments that vary the service times for each service to find out how sensitive the performance is to each of them. System parameters, too, can be varied, so that performance tuning can be done on the simulation model.

25.2 Performance Benchmarks

As database servers become more standardized, the differentiating factor among the products of different vendors is those products' performance. **Performance benchmarks** are suites of tasks that are used to quantify the performance of software systems.

25.2.1 Suites of Tasks

Since most software systems, such as databases, are complex, there is a good deal of variation in their implementation by different vendors. As a result, there is a significant amount of variation in their performance on different tasks. One system may be the most efficient on a particular task; another may be the most efficient on a different task. Hence, a single task is usually insufficient to quantify the performance of the system. Instead, the performance of a system is measured by suites of standardized tasks, called *performance benchmarks*.

Combining the performance numbers from multiple tasks must be done with care. Suppose that we have two tasks, T_1 and T_2 , and that we measure the throughput of a system as the number of transactions of each type that run in a given amount of time—say, 1 second. Suppose that system A runs T_1 at 99 transactions per second and T_2 at 1 transaction per second. Similarly, let system B run both T_1 and T_2 at 50 transactions per second. Suppose also that a workload has an equal mixture of the two types of transactions.

If we took the average of the two pairs of numbers (i.e., 99 and 1, versus 50 and 50), it might appear that the two systems have equal performance. However, it is *wrong* to take the averages in this fashion—if we ran 50 transactions of each type, system A would take about 50.5 seconds to finish, whereas system B would finish in just 2 seconds!

The example shows that a simple measure of performance is misleading if there is more than one type of transaction. The right way to average out the numbers is to take the **time to completion** for the workload, rather than the average *throughput* for each transaction type. We can then compute system performance accurately in transactions per second for a specified workload. Thus, system A takes 50.5/100, which is 0.505 seconds per transaction, whereas system B takes 0.02 seconds per transaction, on average. In terms of throughput, system A runs at an average of 1.98 transactions per second, whereas system B runs at 50 transactions per second. Assuming that transactions of all the types are equally likely, the correct way to average out the throughputs on different transaction types is to take the **harmonic mean** of the throughputs. The harmonic mean of n throughputs $t_1, t_2, \dots, t_n$ is defined as:

$$\frac{n}{\frac{1}{t_1} + \frac{1}{t_2} + \dots + \frac{1}{t_n}}$$

For our example, the harmonic mean for the throughputs in system A is 1.98. For system B, it is 50. Thus, system B is approximately 25 times faster than system A on a workload consisting of an equal mixture of the two example types of transactions.

25.2.2 Database-Application Classes

Online transaction processing (OLTP) and **decision support**, including **online analytical processing (OLAP)**, are two broad classes of applications handled by database systems. These two classes of tasks have different requirements. High concurrency and clever

techniques to speed up commit processing are required for supporting a high rate of update transactions. On the other hand, good query-evaluation algorithms and query optimization are required for decision support. The architecture of some database systems has been tuned to transaction processing; that of others, such as the Teradata series of parallel database systems, has been tuned to decision support. Other vendors try to strike a balance between the two tasks.

Applications usually have a mixture of transaction-processing and decision-support requirements. Hence, which database system is best for an application depends on what mix of the two requirements the application has.

Suppose that we have throughput numbers for the two classes of applications separately, and the application at hand has a mix of transactions in the two classes. We must be careful even about taking the harmonic mean of the throughput numbers because of **interference** between the transactions. For example, a long-running decision-support transaction may acquire a number of locks, which may prevent all progress of update transactions. The harmonic mean of throughputs should be used only if the transactions do not interfere with one another.

25.2.3 The TPC Benchmarks

The **Transaction Processing Performance Council (TPC)** has defined a series of benchmark standards for database systems.

The TPC benchmarks are defined in great detail. They define the set of relations and the sizes of the tuples. They define the number of tuples in the relations not as a fixed number, but rather as a multiple of the number of claimed transactions per second, to reflect that a larger rate of transaction execution is likely to be correlated with a larger number of accounts. The performance metric is throughput, expressed as **transactions per second (TPS)**. When its performance is measured, the system must provide a response time within certain bounds, so that a high throughput cannot be obtained at the cost of very long response times. Further, for business applications, cost is of great importance. Hence, the TPC benchmark also measures performance in terms of **price per TPS**. A large system may have a high number of transactions per second, but it may be expensive (i.e., have a high price per TPS). Moreover, a company cannot claim TPC benchmark numbers for its systems *without* an external audit that ensures that the system faithfully follows the definition of the benchmark, including full support for the ACID properties of transactions.

The first in the series was the **TPC-A benchmark**, which was defined in 1989. This benchmark simulates a typical bank application by a single type of transaction that models cash withdrawal and deposit at a bank teller. The transaction updates several relations—such as the bank balance, the teller’s balance, and the customer’s balance—and adds a record to an audit-trail relation. The benchmark also incorporates communication with terminals, to model the end-to-end performance of the system realistically. The **TPC-B benchmark** was designed to test the core performance of the database system, along with the operating system on which the system runs. It removes the parts

of the TPC-A benchmark that deal with users, communication, and terminals, to focus on the backend database server. Neither TPC-A nor TPC-B is in use today.

The **TPC-C benchmark** was designed to model a more complex system than the TPC-A benchmark. The TPC-C benchmark concentrates on the main activities in an order-entry environment, such as entering and delivering orders, recording payments, checking status of orders, and monitoring levels of stock. The TPC-C benchmark is still widely used for benchmarking online transaction processing (OLTP) systems.

The more recent **TPC-E benchmark** is also aimed at OLTP systems but is based on a model of a brokerage firm, with customers who interact with the firm and generate transactions. The firm in turn interacts with financial markets to execute transactions.

The **TPC-D benchmark** was designed to test the performance of database systems on decision-support queries. Decision-support systems are becoming increasingly important today. The TPC-A, TPC-B, and TPC-C benchmarks measure performance on transaction-processing workloads and should not be used as a measure of performance on decision-support queries. The D in TPC-D stands for **decision support**. The TPC-D benchmark schema models a sales/distribution application, with parts, suppliers, customers, and orders, along with some auxiliary information. The sizes of the relations are defined as a ratio, and database size is the total size of all the relations, expressed in gigabytes. TPC-D at scale factor 1 represents the TPC-D benchmark on a 1-gigabyte database, while scale factor 10 represents a 10-gigabyte database. The benchmark workload consists of a set of 17 SQL queries modeling common tasks executed on decision-support systems. Some of the queries make use of complex SQL features, such as aggregation and nested queries.

The benchmark's users soon realized that the various TPC-D queries could be significantly speeded up by using materialized views and other redundant information. There are applications, such as periodic reporting tasks, where the queries are known in advance, and materialized views can be selected carefully to speed up the queries. It is necessary, however, to account for the overhead of maintaining materialized views.

The **TPC-H benchmark** (where "H" represents ad hoc) is a refinement of the TPC-D benchmark. The schema is the same, but there are 22 queries, of which 16 are from TPC-D. In addition, there are two updates, a set of inserts, and a set of deletes. TPC-H prohibits materialized views and other redundant information and permits indices only on primary and foreign keys. This benchmark models ad hoc querying where the queries are not known beforehand, so it is not possible to create appropriate materialized views ahead of time. A variant, TPC-R (where R stands for "reporting"), which is no longer in use, allowed the use of materialized views and other redundant information.

The **TPC-DS benchmark** is a follow-up to the TPC-H benchmark and models the decision-support functions of a retail product supplier, including information about customers, orders, and products, and with multiple sales channels such as retail stores and online sales. It has two subparts of the schema, corresponding to ad hoc querying and reporting, similar to TPC-H and TPC-R. There is a query workload, as well as a data maintenance workload.

TPC-H and TPC-DS measure performance in this way: The **power test** runs the queries and updates one at a time sequentially, and 3600 seconds divided by the geometric mean of the execution times of the queries (in seconds) gives a measure of queries per hour. The **throughput test** runs multiple streams in parallel, with each stream executing all 22 queries. There is also a parallel update stream. Here the total time for the entire run is used to compute the number of queries per hour.

The **composite query per hour metric**, which is the overall metric, is then obtained as the square root of the product of the power and throughput metrics. A **composite price/performance metric** is defined by dividing the system price by the composite metric.

There are several other TPC benchmarks, such as a data integration benchmark (TPC-DI), benchmarks for big data systems based on Hadoop/Spark (TPCx-HS), and for back-end processing of internet-of-things data (TPCx-IoT).

25.3 Other Issues in Application Development

In this section, we discuss two issues in application development: testing of applications and migration of applications.

25.3.1 Testing Applications

Testing of programs involves designing a **test suite**, that is, a collection of test cases. Testing is not a one-time process, since programs evolve continuously, and bugs may appear as an unintended consequence of a change in the program; such a bug is referred to as program **regression**. Thus, after every change to a program, the program must be tested again. It is usually infeasible to have a human perform tests after every change to a program. Instead, expected test outputs are stored with each test case in a test suite. **Regression testing** involves running the program on each test case in a test suite and checking that the program generates the expected test output.

In the context of database applications, a test case consists of two parts: a database state and an input to a specific interface of the application.

SQL queries can have subtle bugs that can be difficult to catch. For example, a query may execute a join when it should have performed an outer join (i.e., $r \bowtie s$, when it should have actually performed $r \Join s$). The difference between these two queries would be found only if the test database had an r tuple with no matching s tuple. Thus, it is important to create test databases that can catch commonly occurring errors. Such errors are referred to as **mutations**, since they are usually small changes to a query (or program). A test case that produces different outputs on an intended query and a mutant of the query is said to **kill the mutant**. A test suite should have test cases that kill (most) commonly occurring mutants.

If a test case performs an update on the database, to check that it executed properly one must verify that the contents of the database match the expected contents. Thus,

the expected output consists not only of data displayed on the user's screen, but also (updates to) the database state.

Since the database state can be rather large, multiple test cases would share a common database state. Testing is complicated by the fact that if a test case performs an update on the database, the results of other test cases run subsequently on the same database may not match the expected results. The other test cases would then be erroneously reported as having failed. To avoid this problem, whenever a test case performs an update, the database state must be restored to its original state after running the test.

Testing can also be used to ensure that an application meets performance requirements. To carry out such **performance testing**, the test database must be of the same size as the real database would be. In some cases, there is already existing data on which performance testing can be carried out. In other cases, a test database of the required size must be generated; there are several tools available for generating such test databases. These tools ensure that the generated data satisfy constraints such as primary- and foreign-key constraints. They may additionally generate data that look meaningful, for example, by populating a name attribute using meaningful names instead of random strings. Some tools also allow data distributions to be specified; for example, a university database may require a distribution with most students in the range of 18 to 25 years and most faculty in the range of 25 to 65 years.

Even if there is an existing database, organizations usually do not want to reveal sensitive data to an external organization that may be carrying out the performance tests. In such a situation, a copy of the real database may be made, and the values in the copy may be modified in such a way that any sensitive data, such as credit-card numbers, social security numbers, or dates of birth, are **obfuscated**. Obfuscation is done in most cases by replacing a real value with a randomly generated value (taking care to also update all references to that value, in case the value is a primary key). On the other hand, if the application execution depends on the value, such as the date of birth in an application that performs different actions based on the date of birth, obfuscation may make small random changes in the value instead of replacing it completely.

25.3.2 Application Migration

Legacy systems are older-generation application systems that are still in use despite being obsolete. They continue in use due to the cost and risk in replacing them. For example, many organizations developed applications in-house, but they may decide to replace them with a commercial product. In some cases, a legacy system may use old technology that is incompatible with current-generation standards and systems. Some legacy systems in operation today are several decades old and are based on technologies such as databases that use the network or hierarchical data models, or use Cobol and file systems without a database. Such systems may still contain valuable data and may support critical applications.

Replacing legacy applications with new applications is often costly in terms of both time and money, since they are often very large, consisting of millions of lines of code

developed by teams of programmers, often over several decades. They contain large amounts of data that must be ported to the new application, which may use a completely different schema. Switchover from an old to a new application involves retraining large numbers of staff. Switchover must usually be done without any disruption, with data entered in the old system available through the new system as well.

Many organizations attempt to avoid replacing legacy systems and instead try to interoperate them with newer systems. One approach used to interoperate between relational databases and legacy databases is to build a layer, called a **wrapper**, on top of the legacy systems that can make the legacy system appear to be a relational database. The wrapper may provide support for ODBC or other interconnection standards such as OLE-DB, which can be used to query and update the legacy system. The wrapper is responsible for converting relational queries and updates into queries and updates on the legacy system.

When an organization decides to replace a legacy system with a new system, it may follow a process called **reverse engineering**, which consists of going over the code of the legacy system to come up with schema designs in the required data model (such as an E-R model or an object-oriented data model). Reverse engineering also examines the code to find out what procedures and processes were implemented, in order to get a high-level model of the system. Reverse engineering is needed because legacy systems usually do not have high-level documentation of their schema and overall system design. When coming up with a new system, developers review the design so that it can be improved rather than just reimplemented as is. Extensive coding is required to support all the functionality (such as user interface and reporting systems) that was provided by the legacy system. The overall process is called **re-engineering**.

When a new system has been built and tested, the system must be populated with data from the legacy system, and all further activities must be carried out on the new system. However, abruptly transitioning to a new system, which is called the **big-bang approach**, carries several risks. First, users may not be familiar with the interfaces of the new system. Second, there may be bugs or performance problems in the new system that were not discovered when it was tested. Such problems may lead to great losses for companies, since their ability to carry out critical transactions such as sales and purchases may be severely affected. In some extreme cases the new system has even been abandoned, and the legacy system reused, after an attempted switchover failed.

An alternative approach, called the **chicken-little approach**, incrementally replaces the functionality of the legacy system. For example, the new user interfaces may be used with the old system in the back end, or vice versa. Another option is to use the new system only for some functionality that can be decoupled from the legacy system. In either case, the legacy and new systems coexist for some time. There is therefore a need for developing and using wrappers on the legacy system to provide required functionality to interoperate with the new system. This approach therefore has a higher development cost.

25.4 Standardization

Standards define the interface of a software system. For example, standards define the syntax and semantics of a programming language, or the functions in an application-program interface, or even a data model (such as the object-oriented database standards). Today, database systems are complex, and they are often made up of multiple independently created parts that need to interact. For example, client programs may be created independently of backend systems, but the two must be able to interact with each other. A company that has multiple heterogeneous database systems may need to exchange data between the databases. Given such a scenario, standards play an important role.

Formal standards are those developed by a standards organization or by industry groups through a public process. Dominant products sometimes become **de facto standards**, in that they become generally accepted as standards without any formal process of recognition. Some formal standards, like many aspects of the SQL-92 and SQL:1999 standards, are **anticipatory standards** that lead the marketplace; they define features that vendors then implement in products. In other cases, the standards, or parts of the standards, are **reactionary standards**, in that they attempt to standardize features that some vendors have already implemented, and that may even have become de facto standards. SQL-89 was in many ways reactionary, since it standardized features, such as integrity checking, that were already present in the IBM SAA SQL standard and in other databases.

Formal standards committees are typically composed of representatives of the vendors and of members from user groups and standards organizations such as the International Organization for Standardization (ISO) or the American National Standards Institute (ANSI), or professional bodies, such as the Institute of Electrical and Electronics Engineers (IEEE). Formal standards committees meet periodically, and members present proposals for features to be added to or modified in the standard. After a (usually extended) period of discussion, modifications to the proposal, and public review, members vote on whether to accept or reject a feature. Some time after a standard has been defined and implemented, its shortcomings become clear and new requirements become apparent. The process of updating the standard then begins, and a new version of the standard is usually released after a few years. This cycle usually repeats every few years, until eventually (perhaps many years later) the standard becomes technologically irrelevant or loses its user base.

This section gives a very high-level overview of different standards, concentrating on the goals of the standard. Detailed descriptions of the standards mentioned in this section appear in the bibliographic notes for this chapter, available online.

25.4.1 SQL Standards

Since SQL is the most widely used query language, much work has been done on standardizing it. ANSI and ISO, with the various database vendors, have played a leading

role in this work. The SQL-86 standard was the initial version. The IBM Systems Application Architecture (SAA) standard for SQL was released in 1987. As people identified the need for more features, updated versions of the formal SQL standard were developed, called SQL-89 and SQL-92.

The SQL:1999 version of the SQL standard added a variety of features to SQL. We have seen many of these features in earlier chapters.

Subsequent versions of the SQL standard include the following:

- SQL:2003, which is a minor extension of the SQL:1999 standard. Some features such as the SQL:1999 OLAP features (Section 11.3.3) were specified as an amendment to the earlier version of the SQL:1999 standard, instead of waiting for the release of SQL:2003.
- SQL:2006, which added several features related to XML.
- SQL:2008, which introduced only minor extensions to the SQL language such as extensions to the `merge` clause.
- SQL:2011, which added a number of temporal extensions to SQL, including the ability to associate time periods with tuples, optionally using existing columns as start and end times, and primary key definitions based on the time periods. The extensions support deletes and updates with associated periods; such deletes and updates may result in modification of the time period of existing tuples, along with deletes or inserts of new tuples. A number of operators related to time periods, such as `overlaps` and `contains`, were also introduced in SQL:2011.

In addition, the standard provided a number of other features, such as further extensions to the `merge` construct, extensions to the window constructs that were introduced in earlier versions of SQL, and extensions to limit the number of results fetched by a query, using a `fetch` clause.

- SQL:2016, which added a number of features related to JSON support, and support for the aggregate operation `listagg`, which concatenates attributes from a group of tuples into a string.

It is worth mentioning that most of the new features are supported by only a few database systems, and conversely most database systems support a number of features that are not part of the standard.

25.4.2 Database Connectivity Standards

The **ODBC** standard is a widely used standard for communication between client applications and database systems and defines APIs in several languages. The JDBC standard for communication between Java applications and databases was modeled on ODBC and provides similar functionality.

ODBC is based on the SQL **Call Level Interface (CLI)** standards developed by the **X/Open** industry consortium and the SQL Access Group, but it has several extensions.

The ODBC API defines a CLI, an SQL syntax definition, and rules about permissible sequences of CLI calls. The standard also defines conformance levels for the CLI and the SQL syntax. For example, the core level of the CLI has commands to connect to a database, to prepare and execute SQL statements, to get back results or status values, and to manage transactions. The next level of conformance (level 1) requires support for catalog information retrieval and some other features over and above the core-level CLI; level 2 requires further features, such as the ability to send and retrieve arrays of parameter values and to retrieve more detailed catalog information.

ODBC allows a client to connect simultaneously to multiple data sources and to switch among them, but transactions on each are independent; ODBC does not support two-phase commit.

A distributed system provides a more general environment than a client-server system. The X/Open consortium has also developed the [X/Open XA standards](#) for interoperation of databases. These standards define transaction-management primitives (such as transaction begin, commit, abort, and prepare-to-commit) that compliant databases should provide; a transaction manager can invoke these primitives to implement distributed transactions by two-phase commit. The XA standards are independent of the data model and of the specific interfaces between clients and databases to exchange data. Thus, we can use the XA protocols to implement a distributed transaction system in which a single transaction can access relational as well as object-oriented databases, yet the transaction manager ensures global consistency via two-phase commit.

There are many data sources that are not relational databases, and in fact may not be databases at all. Examples are flat files and email stores. Microsoft's [OLE-DB](#) is a C++ API with goals similar to ODBC, but for nondatabase data sources that may provide only limited querying and update facilities. Just like ODBC, OLE-DB provides constructs for connecting to a data source, starting a session, executing commands, and getting back results in the form of a rowset, which is a set of result rows.

The [ActiveX Data Objects \(ADO\)](#) and [ADO.NET](#) APIs, created by Microsoft, provide an interface to access data from not only relational databases, but also some other types of data sources, such as OLE-DB data sources.

25.4.3 Object Database Standards

Standards in the area of object-oriented databases (OODB) have so far been driven primarily by OODB vendors. The [Object Database Management Group \(ODMG\)](#) was a group formed by OODB vendors to standardize the data model and language interfaces to OODBs. ODMG is no longer active. JDO is a standard for adding persistence to Java.

There were several other attempts to standardize object databases and related object-based technologies such as services. However, most were not widely adopted, and they are rarely used anymore.

Object-relational mapping technologies, which store data in relational databases at the back end but provide programmers with an object-based API to access and manip-

ulate data, have proven quite popular. Systems that support object-relational mapping include Hibernate, which supports Java, and the data layer of the popular Django Web framework, which is based on the Python programming language. However, there are no widely accepted formal standards in this area.

25.5 Distributed Directory Systems

Consider an organization that wishes to make data about its employees available to a variety of people in the organization; examples of the kinds of data include name, designation, employee-id, address, email address, phone number, fax number, and so on. Such data are often shared via directories, which allow users to browse and search for desired information.

In general, a directory is a listing of information about some class of objects such as persons. Directories can be used to find information about a specific object, or in the reverse direction to find objects that meet a certain requirement.

A major application of directories today is to authenticate users: applications can collect authentication information such as passwords from users and authenticate them using the directory. Details about the user category (e.g., is the user a student or an instructor), as well as authorizations that a user has been given, may also be shared through a directory. Multiple applications in an organization can then authenticate users using a common directory service and user category and authorization information from the directory to provide users only with data that they are authorized to see.

Directories can be used for storing other types of information, much like file system directories. For instance, web browsers can store personal bookmarks and other browser settings in a directory system. A user can thus access the same settings from multiple locations, such as at home and at work, without having to share a file system.

25.5.1 Directory Access Protocols

Directory information can be made available through web interfaces, as many organizations, and phone companies in particular, do. Such interfaces are good for humans. However, programs too need to access directory information.

Several **directory access protocols** have been developed to provide a standardized way of accessing data in a directory. The most widely used among them today is the **Lightweight Directory Access Protocol (LDAP)**.

All the types of data in our examples can be stored without much trouble in a database system and accessed through protocols such as JDBC or ODBC. The question then is, why come up with a specialized protocol for accessing directory information? There are at least two answers to the question.

- First, directory access protocols are simplified protocols that cater to a limited type of access to data. They evolved in parallel with the database access protocols.

- Second, and more important, directory systems provide a simple mechanism to name objects in a hierarchical fashion, similar to file system directory names, which can be used in a distributed directory system to specify what information is stored in each of the directory servers. For example, a particular directory server may store information for Bell Laboratories employees in Murray Hill, while another may store information for Bell Laboratories employees in Bangalore, giving both sites autonomy in controlling their local data. The directory access protocol can be used to obtain data from both directories across a network. More important, the directory system can be set up to automatically forward queries made at one site to the other site, without user intervention.

For these reasons, several organizations have directory systems to make organizational information available online through a directory access protocol. Information in an organizational directory can be used for a variety of purposes, such as to find addresses, phone numbers, or email addresses of people, to find which departments people are in, and to track department hierarchies.

As may be expected, several directory implementations find it beneficial to use relational databases to store data instead of creating special-purpose storage systems.

25.5.2 LDAP: Lightweight Directory Access Protocol

In general a directory system is implemented as one or more servers, which service multiple clients. Clients use the API defined by the directory system to communicate with the directory servers. Directory access protocols also define a data model and access control. The **X.500 directory access protocol**, defined by the International Organization for Standardization (ISO), is a standard for accessing directory information. However, the protocol is rather complex and is not widely used. The **Lightweight Directory Access Protocol (LDAP)** provides many of the X.500 features, but with less complexity, and is widely used. In addition to several open-source implementations, the Microsoft Active Directory system, which is based on LDAP, is used in a large number of organizations.

In the rest of this section, we shall outline the data model and access protocol details of LDAP.

25.5.2.1 LDAP Data Model

In LDAP, directories store **entries**, which are similar to objects. Each entry must have a **distinguished name (DN)**, which uniquely identifies the entry. A DN is in turn made up of a sequence of **relative distinguished names (RDNs)**. For example, an entry may have the following distinguished name:

cn=Silberschatz, ou=Computer Science, o=Yale University, c=USA

As you can see, the distinguished name in this example is a combination of a name and (organizational) address, starting with a person's name, then giving the organizational unit (ou), the organization (o), and country (c). The order of the components of a distinguished name reflects the normal postal address order, rather than the reverse

order used in specifying path names for files. The set of RDNs for a DN is defined by the schema of the directory system.

Entries can also have attributes. LDAP provides binary, string, and time types, and additionally the types `tel` for telephone numbers, and `PostalAddress` for addresses (lines separated by a “\$” character). Unlike those in the relational model, attributes are multivalued by default, so it is possible to store multiple telephone numbers or addresses for an entry.

LDAP allows the definition of **object classes** with attribute names and types. Inheritance can be used in defining object classes. Moreover, entries can be specified to be of one or more object classes. It is not necessary that there be a single most-specific object class to which an entry belongs.

Entries are organized into a **directory information tree (DIT)**, according to their distinguished names. Entries at the leaf level of the tree usually represent specific objects. Entries that are internal nodes represent objects such as organizational units, organizations, or countries. The children of a node have a DN containing all the RDNs of the parent, and one or more additional RDNs. For instance, an internal node may have a DN `c=USA`, and all entries below it have the value `USA` for the RDN `c`.

The entire distinguished name need not be stored in an entry. The system can generate the distinguished name of an entry by traversing up the DIT from the entry, collecting the RDN=value components to create the full distinguished name.

Entries may have more than one distinguished name—for example, an entry for a person in more than one organization. To deal with such cases, the leaf level of a DIT can be an **alias** that points to an entry in another branch of the tree.

25.5.2.2 Data Manipulation

Unlike SQL, LDAP does not define either a data-definition language or a data-manipulation language. However, LDAP defines a network protocol for carrying out data definition and manipulation. Users of LDAP can either use an application-programming interface or use tools provided by various vendors to perform data definition and manipulation. LDAP also defines a file format called **LDAP Data Interchange Format (LDIF)** that can be used for storing and exchanging information.

The querying mechanism in LDAP is very simple, consisting of just selections and projections, without any join. A query must specify the following:

- A base—that is, a node within a DIT—by giving its distinguished name (the path from the root to the node).
- A search condition, which can be a Boolean combination of conditions on individual attributes. Equality, matching by wild-card characters, and approximate equality (the exact definition of approximate equality is system dependent) are supported.
- A scope, which can be just the base, the base and its children, or the entire subtree beneath the base.

- Attributes to return.
- Limits on number of results and resource consumption.

The query can also specify whether to automatically dereference aliases; if alias dereferences are turned off, alias entries can be returned as answers.

We omit further details of query support in LDAP but note that LDAP implementations support an API for querying/updating LDAP data and may additionally support web services for querying LDAP data.

25.5.2.3 Distributed Directory Trees

Information about an organization may be split into multiple DITs, each of which stores information about some entries. The **suffix** of a DIT is a sequence of RDN=value pairs that identify what information the DIT stores; the pairs are concatenated to the rest of the distinguished name generated by traversing from the entry to the root. For instance, the suffix of a DIT may be o=Nokia, c=USA, while another may have the suffix o=Nokia, c=India. The DITs may be organizationally and geographically separated.

A node in a DIT may contain a **referral** to another node in another DIT; for instance, the organizational unit Bell Labs under o=Nokia, c=USA may have its own DIT, in which case the DIT for o=Nokia, c=USA would have a node ou=Bell Labs representing a referral to the DIT for Bell Labs.

Referrals are the key component that help organize a distributed collection of directories into an integrated system. When a server gets a query on a DIT, it may return a referral to the client, which then issues a query on the referenced DIT. Access to the referenced DIT is transparent, proceeding without the user's knowledge. Alternatively, the server itself may issue the query to the referred DIT and return the results along with locally computed results.

The hierarchical naming mechanism used by LDAP helps break up control of information across parts of an organization. The referral facility then helps integrate all the directories in an organization into a single virtual directory.

Although it is not an LDAP requirement, organizations often choose to break up information either by geography (for instance, an organization may maintain a directory for each site where the organization has a large presence) or by organizational structure (for instance, each organizational unit, such as department, maintains its own directory). Many LDAP implementations support master-slave and multimaster replication of DITs.

25.6 Summary

- Tuning of the database-system parameters, as well as the higher-level database design—such as the schema, indices, and transactions—is important for good performance. Tuning is best done by identifying bottlenecks and eliminating them.

- Database tuning can be done at the level of schema and queries, at the level of database system parameters, and at the level of hardware. Database systems usually have a variety of tunable parameters, such as buffer sizes.
- The right choice of indices and materialized views, and the use of horizontal partitioning can provide significant performance benefits. Tools for automated tuning based on workload history can help significantly in such tuning. The set of indices and materialized views can be appropriately chosen to minimize overall cost. Vertical partitioning, and columnar storage can lead to significant benefits in online analytical processing systems.
- Transactions can be tuned to minimize lock contention; snapshot isolation and sequence numbering facilities supporting early lock release are useful tools for reducing read-write and write-write contention.
- Hardware tuning includes choice of memory size, the use of SSDs versus magnetic hard disks, and increasingly, the number of CPU cores.
- Performance benchmarks play an important role in comparisons of database systems, especially as systems become more standards compliant. The TPC benchmark suites are widely used, and the different TPC benchmarks are useful for comparing the performance of databases under different workloads.
- Applications need to be tested extensively as they are developed and before they are deployed. Testing is used to catch errors as well as to ensure that performance goals are met.
- Legacy systems are systems based on older-generation technologies such as nonrelational databases or even directly on file systems. Interfacing legacy systems with new-generation systems is often important when they run mission-critical systems. Migrating from legacy systems to new-generation systems must be done carefully to avoid disruptions, which can be very expensive.
- Standards are important because of the complexity of database systems and their need for interoperation. Formal standards exist for SQL. De facto standards, such as ODBC and JDBC, and standards adopted by industry groups have played an important role in the growth of client-server database systems.
- Distributed directory systems have played an important role in many applications, and can be viewed as distributed databases. LDAP is widely used for authentication and for tracking employee information in organizations.

Review Terms

- Performance tuning
- Bottlenecks
- Queueing systems
- Tuning of physical schema

- Tuning of indices
- Materialized views
- Immediate view maintenance
- Deferred view maintenance
- Tuning of physical design
- Workload
- Tuning of queries
- Set orientation
- Batch update (JDBC)
- Bulk load
- Bulk update
- Merge statement
- Tuning of logical schema
- Tunable parameters
- Tuning of concurrent transactions
- Sequences
- Minibatch transactions
- Tuning of hardware
- Five minute rule
- Performance simulation
- Performance benchmarks
- Service time
- Throughput
- Database-application classes
- OLTP
- Decision support
- The TPC benchmarks
 - TPC-C
 - TPC-D
 - TPC-E
 - TPC-H
 - TPC-DS
- Regression testing
- Killing mutants
- Application migration
- Legacy systems
- Reverse engineering
- Re-engineering
- Standardization
 - Formal standards
 - De facto standards
 - Anticipatory standards
 - Reactionary standards
- Database connectivity standards
- X/Open XA standards
- Object database standards
- XML-based standards
- LDAP
- Directory information tree
- Distributed directory trees

Practice Exercises

- 25.1** Find out all performance information your favorite database system provides. Look for at least the following: what queries are currently executing or executed recently, what resources each of them consumed (CPU and I/O), what fraction of page requests resulted in buffer misses (for each query, if available), and what locks have a high degree of contention. Also get information about CPU, I/O and network utilization, including the number of open network connections using your operating system utilities.

- 25.2** Many applications need to generate sequence numbers for each transaction.
- If a sequence counter is locked in two-phase manner, it can become a concurrency bottleneck. Explain why this may be the case.
 - Many database systems support built-in sequence counters that are not locked in two-phase manner; when a transaction requests a sequence number, the counter is locked, incremented and unlocked.
 - Explain how such counters can improve concurrency.
 - Explain why there may be gaps in the sequence numbers belonging to the final set of committed transactions.
- 25.3** Suppose you are given a relation $r(a, b, c)$.
- Give an example of a situation under which the performance of equality selection queries on attribute a can be greatly affected by how r is clustered.
 - Suppose you also had range selection queries on attribute b . Can you cluster r in such a way that the equality selection queries on $r.a$ and the range selection queries on $r.b$ can both be answered efficiently? Explain your answer.
 - If clustering as above is not possible, suggest how both types of queries can be executed efficiently by choosing appropriate indices.
- 25.4** When a large number of records are inserted into a relation in a short period of time, it is often recommended that all indices be dropped, and recreated after the inserts have been completed.
- What is the motivation for this recommendation?
 - Dropping and recreation of indices can be avoided by bulk-updating of the indices. Suggest how this could be done efficiently for B⁺-tree indices.
 - If the indices were write-optimized indices such as LSM trees, would this advice be meaningful?
- 25.5** Suppose that a database application does not appear to have a single bottleneck; that is, CPU and disk utilization are both high, and all database queues are roughly balanced. Does that mean the application cannot be tuned further? Explain your answer.
- 25.6** Suppose a system runs three types of transactions. Transactions of type A run at the rate of 50 per second, transactions of type B run at 100 per second, and transactions of type C run at 200 per second. Suppose the mix of transactions has 25 percent of type A, 25 percent of type B, and 50 percent of type C.

- a. What is the average transaction throughput of the system, assuming there is no interference between the transactions?
- b. What factors may result in interference between the transactions of different types, leading to the calculated throughput being incorrect?

25.7 Suppose an application programmer was supposed to write a query

```
select *
from r natural left outer join s;
```

on relations $r(A, B)$ and $s(B, C)$, but instead wrote the query

```
select *
from r natural join s;
```

- a. Give sample data for r and s on which both queries would give the same result.
 - b. Give sample data for r and s where the two queries would give different results, thereby exposing the error in the query,
- 25.8** List some benefits and drawbacks of an anticipatory standard compared to a reactionary standard.
- 25.9** Describe how LDAP can be used to provide multiple hierarchical views of data, without replicating the base-level data.

Exercises

- 25.10** Database tuning:
- a. What are the three broad levels at which a database system can be tuned to improve performance?
 - b. Give two examples of how tuning can be done for each of the levels.
- 25.11** When carrying out performance tuning, should you try to tune your hardware (by adding disks or memory) first, or should you try to tune your transactions (by adding indices or materialized views) first. Explain your answer.
- 25.12** Suppose that your application has transactions that each access and update a single tuple in a very large relation stored in a B⁺-tree file organization. Assume that all internal nodes of the B⁺-tree are in memory, but only a very small fraction of the leaf pages can fit in memory. Explain how to calculate the minimum number of disks required to support a workload of 1000 transactions

per second. Also calculate the required number of disks, using values for disk parameters given in Section 12.3.

- 25.13** What is the motivation for splitting a long transaction into a series of small ones? What problems could arise as a result, and how can these problems be averted?
- 25.14** Suppose the price of memory falls by half, and the speed of disk access (number of accesses per second) doubles, while all other factors remain the same. What would be the effect of this change on the 5-minute and 1-minute rule?
- 25.15** List at least four features of the TPC benchmarks that help make them realistic and dependable measures.
- 25.16** Why was the TPC-D benchmark replaced by the TPC-H and TPC-R benchmarks?
- 25.17** Explain what application characteristics would help you decide which of TPC-C, TPC-H, or TPC-R best models the application.
- 25.18** Given that the LDAP functionality can be implemented on top of a database system, what is the need for the LDAP standard?

Further Reading

[Harchol-Balte (2013)] provides textbook coverage of queuing theory from a computer science perspective.

Information about tuning support in IBM DB2, Oracle and Microsoft SQL Server may be found in their respective manuals online, as well as in numerous books. [Shasha and Bonnet (2002)] provides detailed coverage of database tuning principles. [O’Neil and O’Neil (2000)] provides a very good textbook coverage of performance measurement and tuning. The 5-minute and 1-minute rules are described in [Gray and Graefe (1997)], [Graefe (2008)], and [Appuswamy et al. (2017)].

An early proposal for a database-system benchmark (the Wisconsin benchmark) was made by [Bitton et al. (1983)]. The TPC-A, TPC-B, and TPC-C benchmarks are described in [Gray (1991)]. An online version of all the TPC benchmark descriptions, as well as benchmark results, is available on the World Wide Web at the URL www.tpc.org; the site also contains up-to-date information about new benchmark proposals.

The XData system (www.cse.iitb.ac.in/infolab/xdata) provides tools for generating test data to catch errors in SQL queries, as well as for grading student SQL queries.

A number of standards documents, including several parts of the SQL standard, can be found on the ISO/IEC website (standards.iso.org/ittf/PubliclyAvailableStandards/index.html). Information about ODBC, OLE-DB, ADO, and ADO.NET can be found on the web site

www.microsoft.com/data. A wealth of information on XML-based standards and tools is available online on the web site www.w3c.org.

Bibliography

- [Appuswamy et al. (2017)] R. Appuswamy, R. Borovica, G. Graefe, and A. Ailamaki, “The Five minute Rule Thirty Years Later and its Impact on the Storage Hierarchy”, In *Proceedings of the 7th International Workshop on Accelerating Analytics and Data Management Systems Using Modern Processor and Storage Architectures* (2017).
- [Bitton et al. (1983)] D. Bitton, D. J. DeWitt, and C. Turbyfill, “Benchmarking Database Systems: A Systematic Approach”, In *Proc. of the International Conf. on Very Large Databases* (1983), pages 8–19.
- [Graefe (2008)] G. Graefe, “The Five-Minute Rule 20 Years Later: and How Flash Memory Changes the Rules”, *ACM Queue*, Volume 6, Number 4 (2008), pages 40–52.
- [Gray (1991)] J. Gray, *The Benchmark Handbook for Database and Transaction Processing Systems*, 2nd edition, Morgan Kaufmann (1991).
- [Gray and Graefe (1997)] J. Gray and G. Graefe, “The Five-Minute Rule Ten Years Later, and Other Computer Storage Rules of Thumb”, *SIGMOD Record*, Volume 26, Number 4 (1997), pages 63–68.
- [Harchol-Balte (2013)] M. Harchol-Balte, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*, Cambridge University Press (2013).
- [O’Neil and O’Neil (2000)] P. O’Neil and E. O’Neil, *Database: Principles, Programming, Performance*, 2nd edition, Morgan Kaufmann (2000).
- [Shasha and Bonnet (2002)] D. Shasha and P. Bonnet, *Database Tuning: Principles, Experiments, and Troubleshooting Techniques*, Morgan Kaufmann (2002).

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.

CHAPTER 26



Blockchain Databases

At the most basic level, a blockchain provides an alternative data format for storing a database, and its paradigm for transaction processing enables a high level of decentralization.

A major application of blockchain technology is in the creation of **digital ledgers**. A ledger in the financial world is a book of financial accounts, that keeps track of transactions. For example, each time you deposit or withdraw money from your account, an entry is added to a ledger maintained by the bank. Since the ledger is maintained by the bank, a customer of the bank implicitly trusts the bank to not cheat by adding unauthorized transactions to the ledger, such as an unauthorized withdrawal, or modifying the ledger by deleting transactions such as a deposit.

Blockchain-based distributed ledgers maintain a ledger cooperatively among several parties, in such a way that each transaction is digitally signed as proof of authenticity, and further, the ledger is maintained in such a way that once entries are added, they cannot be deleted or modified by one party, without detection by others.

Blockchains form a key foundation of Bitcoin and other cryptocurrencies. Although much of the technology underlying blockchains was initially developed in the 1980s and 1990s, blockchain technology gained widespread popular attention in the 2010s as a result of boom (and subsequent bust) in Bitcoin and other cryptocurrencies.

However, beyond the many cryptocurrency schemes, blockchains can provide a secure data-storage and data-processing foundation for business applications, without requiring complete trust in any one party. For example, consider a large corporation and its suppliers, all of whom maintain data about where products and components are located at any time as part of the manufacturing process. Even if the organizations are presumed trustworthy, there may a situation where one of them has a strong incentive to cheat and rewrite the record. A blockchain can help protect from such fraudulent updates. Ownership documents, such as real-estate deeds, are another example of the potential for blockchain use. Criminals may commit real-estate fraud by creating fake ownership deeds, which could allow them to sell a property that they do not own, or could allow the same property to be sold multiple times by an actual owner. Blockchains can help verify the authenticity of digitally represented ownership documents; blockchains can also ensure that once an owner has sold a property, the

owner cannot sell it again to another person without getting detected. The security provided by the blockchain data structure makes it possible to allow the public to view these real-estate records without putting them at risk. We describe other applications for blockchains later in the chapter.

In this chapter, we shall look at blockchain from a database perspective. We shall identify the ways in which blockchain databases differ from the traditional databases we have studied elsewhere in this book and show how these distinguishing features are implemented. We shall consider alternatives to Bitcoin-style algorithms and implementation that are more suited to an enterprise database environment. With this database-oriented focus, we shall not consider the financial implications of cryptocurrencies, nor the issues of managing one's holding of such currencies via a cryptocurrency wallet or exchange.

26.1 Overview

Before we study blockchains in detail, we first give an overview of cryptocurrencies, which have driven the development and usage of blockchains. We note, however, that blockchains have many uses beyond cryptocurrencies.

Traditional currencies, also known as “fiat currencies” are typically issued by a central bank of a country, and guaranteed by the government of that country. Currency notes are at one level just a piece of paper; the only reason they are of value is that the government that issues the currency guarantees the value of the currency, and users trust the government. Today, although financial holdings continue to be denominated in terms of a currency, most of the financial holdings are not physically present as currency notes; they are merely entries in the ledger of a bank or other financial institution. Users of the currency are forced to trust the organization that maintains the ledger.

A **cryptocurrency** is a currency created purely online, and recorded in a way that does not require any one organization (or country) to be totally trusted. This term arises from the fact that any such scheme has to be based on encryption technologies. Since any digital information can be copied easily, unlike currency notes, any cryptocurrency scheme must be able to prevent “double spending” of money. To solve this problem, cryptocurrencies use ledgers to record transactions. Further, the ledgers are stored in a secure, distributed infrastructure, with no requirement to trust any one party. These two key concepts, decentralization and trustlessness, are fundamental to cryptocurrencies. Cryptocurrencies typically aim, like regular currency, and unlike credit card or debit card transactions, to provide transaction anonymity, to preserve the privacy of users of the currency. However, since cryptocurrency blockchains are public data analytics may be used to compromise or limit anonymity.

Bitcoin, which was the first successful cryptocurrency, emerged with the publication of a paper by Satoshi Nakamoto¹ in 2008 and the subsequent publication of the open-source Bitcoin code in 2009. The ideas in the original bitcoin paper solved

¹Satoshi Nakamoto is a pseudonym for a person or group that anonymously created Bitcoin.

a number of problems, and thereby allowed cryptocurrencies, which had earlier been considered impractical, to become a reality.

However, the underlying concepts and algorithms in many cases go back decades in their development. The brilliance of Nakamoto's work was a combination of innovation and well-architected use of prior research. The successes of Bitcoin prove the value of this contribution, but the target—an anonymous, trustless, fully distributed concurrency—drove many technical decisions in directions that work less well in a database setting. The Further Reading section at the end of the chapter cites key historical papers in the development of these ideas.

At its most basic level, a blockchain is a linked list of blocks of data that can be thought of as constituting a log of updates to data. What makes blockchain technology interesting is that blockchains can be managed in a distributed manner in such a way that they are highly tamper resistant, and cannot be easily modified or manipulated by any one participant, except by appending digitally signed records to the blockchain.

In a business setting, trustless distributed control is valuable, but absolute anonymity runs counter to both principles of accounting and regulatory requirements. This leads to two main scenarios for the use of blockchains. Bitcoin's blockchain is referred to as a **public blockchain**, since it allows any site to join and participate in the tasks of maintaining the blockchain. In contrast, most enterprise blockchains are more restricted and referred to as **permissioned blockchains**. In a permissioned blockchain, participation is not open to the public. Access is granted by a permissioning authority, which may be an enterprise, a consortium of enterprises, or a government agency.

Bitcoin introduced a number of ideas that made public blockchains practical, but these have a significant cost in terms of CPU power (and thereby, electrical power) needed to run the blockchain, as well as latencies in processing transactions. By relaxing Bitcoin's strong assumptions about trustlessness and anonymity, it is possible to overcome many of the inefficiencies and high latencies of the Bitcoin model and design blockchains that further the goals of enterprise data management.

In this chapter, we begin by looking at the classic blockchain structure as used in Bitcoin and use that to introduce the key distinguishing properties of a blockchain. Achieving many of these properties relies upon one-way cryptographic hash functions. These hash functions are quite different from those used in Chapter 24 as a means of indexing databases. Cryptographic hash functions need to have some specific mathematical properties such as the following: given a data value x and a hash function h , it must be relatively easy to compute $h(x)$ but virtually impossible to find x given $h(x)$.

When a blockchain is stored distributed across multiple systems, an important issue is to ensure that the participating systems agree on what are the contents of the blockchain, and what gets added to it at each step. When participants trust each other, but may be vulnerable to failure, consensus techniques that we studied earlier in Section 23.8 can be used to ensure that all participants agree on the contents of a log (and a blockchain is, at its core, a log). However, reaching agreement on what data get added to a blockchain is much more challenging when participants in the blockchain do not trust each other and have no centralized control. The basic consensus algorithms are

not applicable in such a setting. For example, an attacker could create a large number of systems, each of which joins the blockchain as a participant; the attacker could thereby control a majority of the participating systems. Any decision based on a majority can then be controlled by the attacker, who can force decisions that can tamper with the contents of the blockchain. The tamper resistance property of the blockchain would then be compromised.

We begin by describing the energy-intensive approach of Bitcoin, but we then consider a variety of alternative, more efficient approaches used in other cryptocurrencies. Finally, we consider approaches based on Byzantine-consensus algorithms, which are consensus algorithms that are resistant to some fraction of the participating nodes not just failing, but also lying and attempting to disrupt consensus. Byzantine consensus is well-suited to an enterprise blockchain environment, and can be used if the blockchain is *permissioned*, that is, some organization controls who can have permission to access the blockchain. Byzantine consensus is an old problem and solutions have been around for many years. However, the special constraints of blockchain databases have led to some newer approaches to this problem. References to more details on Byzantine consensus techniques may be found in the Further Reading section at the end of the chapter.

Blockchain databases store more than just currency-based debit-credit transactions. Like any database, they may store a variety of types of data about the enterprise. A traditional blockchain data organization makes it difficult to retrieve such data efficiently, but pairing a blockchain with a traditional database or building the blockchain on top of a database can enable faster query processing. We shall explore a variety of means of speeding up not only queries but also update transactions, both within the blockchain itself and by performing certain operations “off chain” and adding them in bulk to the blockchain at a later time.

After covering blockchain algorithms, we shall explore (in Section 26.8) some of the most promising applications of blockchain databases.

26.2 Blockchain Properties

At its most basic level, a blockchain is a linked list of blocks of data. A distinguishing feature of the blockchain data structure is that the pointers in the linked list include not only the identifier of the next older block, but also a hash of that older block. This structure is shown in Figure 26.1. The initial block, or **genesis block**, is shown as block 0 in the figure. It is set up by the creator of the blockchain. Each time a block is added to the chain, it includes the pair of values (pointer-to-previous-block, hash-of-previous-block). As a result, any change made to block is easily detected by comparing a hash of that block to the hash value contained in the next block in the chain. The hash value in the next block could be changed, but then the block after that would also have to be changed, and so on.

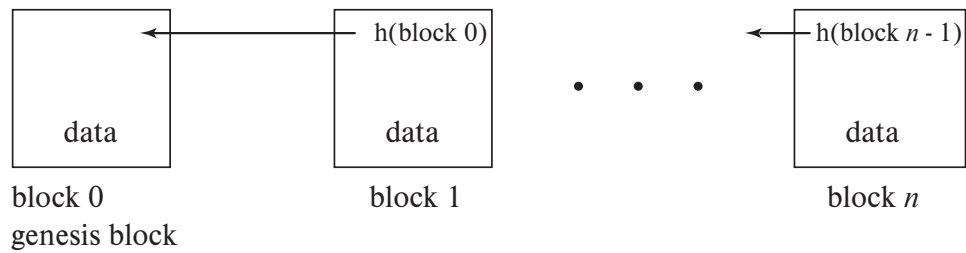


Figure 26.1 Blockchain data structure.

This hash-validated pointer format in a blockchain makes tampering with a blockchain hard. To make tampering virtually impossible, it is necessary to ensure that any tampering with the blockchain is easily detected and that the correct version of the blockchain is easily determined. To achieve this, the hash function must have certain mathematical properties that we shall discuss shortly. Further, the chain itself must be replicated and distributed among many independent nodes so that no single node or small group of nodes can tamper with the blockchain. Since the blockchain is replicated across multiple nodes, a distributed consensus algorithm needs to be used to maintain agreement regarding the correct current state of the blockchain. In this way, even if some nodes try to tamper with the blockchain contents, as long as a majority are honest, making decisions based on a majority vote can ensure the integrity of the blockchain.

The above approach works if the set of nodes that participates in the blockchain is controlled in some fashion that makes it difficult for an adversary to control a majority of the nodes. However, such control goes against the goal of not have any central control, and is viewed as unacceptable in public blockchains such as Bitcoin, which are based on public blockchains in which the number of participating nodes may change continuously. Any computer may download the blockchain and attempt to add blocks (the code for implementing blockchains is available in open source). As a result, a majority-based approach can be overwhelmed by an adversary who sets up a large number of low-cost computers as nodes. Such an attack is called a **Sybil attack**.

The way in which consensus is achieved among independent nodes varies among blockchains. The variations address trade-offs between performance (latency and throughput) and robustness to adversarial attacks on the consensus mechanism, including Sybil attacks. When we addressed distributed consensus in Chapter 23, we assumed that a single organization controlled the entire distributed system, and so the consensus algorithm had to tolerate only possible failures of nodes or the network that were fail-stop, where participants do not behave in an adversarial manner.

In a typical blockchain application, the chain is shared among multiple independent organizations. In the extreme case, for example Bitcoin, anyone can set up a node and participate, possibly for nefarious purposes. This implies that the types of failure that may occur are not just cases where a device or system stops working, but also cases

where a system remains operational but behaves in an adversarial manner. In most enterprise settings, the blockchain is **permissioned**, providing some control over the set of participants, but still without direct controls to prevent malicious behavior.

A node participating in a blockchain fully needs to participate in the consensus mechanism and maintain its own replica of the blockchain. Such a node is called a **full node**. In some applications, there is a need for low-cost nodes that submit updates to the blockchain, but do not have the storage or computational power to participate in the consensus process. Such a node is called a **light node**.

We discuss blockchain consensus algorithms in detail in Section 26.4. Blockchain consensus algorithms can be placed into one of several broad categories:

- **Proof of work:** Proof of work, which is described in detail in Section 26.4.1, provides a solution to Sybil attacks by making it very expensive for an attacker to control a majority of the nodes. Specifically, the nodes agree that the next block on the blockchain will be added by the first node to solve a certain hard mathematical problem. This is referred to as **mining** a block. Proof-of-work algorithms are robust to adversarial behavior as long as the adversary does not control more than half the computing power in the entire network. To ensure this requirement, the problems are made intentionally hard, and require a lot of computational effort. Thus, robustness comes at the price of a huge amount of otherwise useless computation along with the price of electricity needed to carry out the computation.
- **Proof of stake:** Proof of stake, which is described in Section 26.4.2, provides another solution to Sybil attacks. Here, the nodes agree to select the next node to add a block to the blockchain based on an amount of the blockchain's currency owned or held in reserve by a node.
- **Byzantine consensus:** Byzantine consensus does not solve the problem of Sybil attacks, but can be used in non-public blockchains, where entry of nodes to the system can be controlled. While some nodes may behave maliciously, it is assumed that a substantial majority are honest. In Byzantine consensus, described in Section 26.4.3, the next node to add a block to the blockchain is decided by an algorithm from the class of algorithms referred to as Byzantine-consensus algorithms. Like the basic consensus algorithms we described earlier in Section 23.8, these algorithms achieve agreement by message passing, but unlike those algorithms, these algorithms can tolerate some number of nodes being malicious by either disrupting consensus or trying to cause an incorrect consensus to be reached. This approach requires significantly more messages to be exchanged than in the case of the basic consensus algorithms of Section 26.4.3, but this is a worthwhile trade-off for the ability for a system to work correctly in the presence of a certain number of malicious nodes.
- **Other approaches:** There are several other less widely used consensus mechanisms, some of which are variants of the preceding mechanisms. These include proof of

activity, proof of burn, proof of capacity, and proof of elapsed time. See the Further Reading section at the end of the chapter for details.

Another way to damage a blockchain besides attempting to alter blocks is to add a new block to a block other than the most recent one. This is called a **fork**. Forking may occur due to malicious activity, but there are two sources of nonmalicious forks:

1. Two distinct nodes may add a new block after the most recent block, but they do it so close together in time that both are added successfully, thus creating a forked chain. These accidental forks are resolved by a protocol rule that nodes always attempt to add blocks to the end of the longest chain. This probabilistically limits these accidental forks to a short length. The blocks on the shorter forks are said to be *orphaned*, and the contents of those blocks will get inserted on the real chain later if those contents are not already there.
2. A majority of blockchain users may agree to fork the blockchain in order to change some aspect of the blockchain protocol or data structure. This is a rare event and one that, when it has occurred in major blockchains, has caused major controversy. Such a fork is said to be a **soft fork** if prior blocks are not invalidated by the fork. That is, the old version of the blockchain software will recognize blocks from the new version as valid. This permits a gradual transition from the old version of the blockchain software to the new version. In a **hard fork**, the old version of the blockchain software will deem blocks from the new version to be invalid. After a hard fork, if the old version of the blockchain software remains in use, it will lead to a separate blockchain with different contents.

Because of the possibility of orphaned blocks, it may be necessary to wait for several additional blocks to be added before it is safe to assume a block will not be orphaned.

Note 26.1 on page 1258 presents a few examples of notable blockchain forks.

So far, we have not said much about the actual data in the blocks. The contents of blocks vary by application domain. In a cryptocurrency application, the most common data found in blocks are basic currency-transfer transactions. Since any node can add a block, there needs to be a way to ensure that transactions entered are in fact genuine. This is achieved via a technique called a **digital signature** that allows a user to “sign” a transaction and allows every node to verify that signature. This prevents fake transactions from being added to the chain and prevents participants in the transaction from subsequently denying their involvement in the transaction. This latter property is referred to as **irrefutability**.

Transactions are broadcast to all nodes participating in the blockchain; when a node adds a block to the chain, the block contains all transactions received by the node that have not already been added to the chain.

The users who submit transactions may be known to the blockchain administrator in a permissioned blockchain, but in a public blockchain like Bitcoin, there is no direct

Note 26.1 Blockchain Fork Examples

There have been several notable forks of major blockchains. We list a few here.

- **Hard fork: Bitcoin/Bitcoin Cash:** Bitcoin's built-in block-size limit was an acknowledged problem in the Bitcoin community but agreeing on a solution proved controversial. A hard fork in August 2017 created a new cryptocurrency, Bitcoin Cash, with a larger block-size limit. Holders of Bitcoin at the time of the fork received an equal amount of Bitcoin Cash, and thus could spend both.
- **Soft fork: Bitcoin SegWit:** SegWit (short for segregated witness) moves certain transaction-signature data (referred to as *witness* data) outside the block. This allows more transactions per block while retaining the existing block size limit. The relocated witness data are needed only for transaction validation. SegWit was introduced in August 2017 via a soft fork. This was a soft fork because the old blocks were recognized as valid and nodes not yet upgraded were able to retain a high degree of compatibility.
- **Hard fork: Ethereum/Ethereum Classic:** This fork arose from the failure of a crowd-funded venture-capital operation running as a smart contract in the Ethereum blockchain. Its code contained a design flaw that enabled a hack in 2016 that stole ether valued in the tens of millions of U.S. dollars. A controversial hard fork refunded the stolen funds, but opponents of the fork, believing in the inviolability of blockchain immutability, retained the original blockchain and created Ethereum Classic.

connection between a user ID and any real-world entity. This anonymity property is a key feature of Bitcoin, but its value is diminished because of the possibility to tie a user ID to some off-chain activity, thereby de-anonymizing the user. De-anonymization can occur if the user enters into a transaction with a user whose user ID has already been de-anonymized. De-anonymization can occur also via data mining on the blockchain data and correlating on-chain activity by a specific user ID with the “real-world” activity of a specific individual.

Finally, a feature of blockchains is the ability to store executable code, referred to as a **smart contract**. A smart contract can implement complex transactions, take action at some point in the future based on specified conditions, and, more generally, encode a complex agreement among a set of users. Blockchains differ not only in the language used for smart contracts but also in the power of the language used. Many are Turing complete, but some (notably, Bitcoin) have more limited power. We discuss smart contracts, including how and when their code is executed, in Section 26.6.

We summarize this discussion by listing a set of properties of blockchains:²

- **Decentralization:** In a public blockchain, control of the blockchain is by majority consensus with no central controlling authority. In a permissioned blockchain, the degree of central control is limited, typically only to access authorization and identity management. All other actions happen in a decentralized manner.
- **Tamper Resistance:** Without gaining control over a majority of the blockchain network, it is infeasible to change the contents of blocks.
- **Irrefutability:** Activity by a user on a blockchain is signed cryptographically by the user. These signatures can be validated easily by anyone and thus prove that the user indeed is responsible for the transaction.
- **Anonymity:** Users of a blockchain have user IDs that are not tied directly to any personally identifying information, though anonymity may be compromised indirectly. Permissioned blockchains may offer only limited anonymity or none at all.

26.3 Achieving Blockchain Properties via Cryptographic Hash Functions

In this section, we focus on the use of cryptographic hash functions to ensure some of the properties of blockchains. We begin with a discussion of special types of hash function for which it is infeasible to compute the inverse function or find hash collisions. We show how these concepts extend to public-key encryption, which we first saw in Section 9.9. We then show how cryptographic hash functions can be used to ensure the anonymity, irrefutability, and tamper-resistance properties. We show how hash functions are used in mining algorithms later in Section 26.4.1.

26.3.1 Properties of Cryptographic Hash Functions

In Section 14.5, hash functions were used as a means of accessing data. Here, we use hash functions for a very different set of purposes, and as a result, we shall need hash functions with additional properties beyond those discussed earlier.

A hash function h takes input from some (large) domain of values and generates as its output a fixed-length bit string. Typically, the cardinality of the domain is much larger than the cardinality of the range. Furthermore, the hash function must have a *uniform distribution*, that is, each range value must be equally probable given random input. A hash function h is **collision resistant** if it is infeasible to find two distinct values x and y such that $h(x) = h(y)$. By **infeasible**, we mean that there is strong mathematical

²These properties pertain to blockchains, but not to most cryptocurrency exchanges. Most exchanges hold not only customers' data but also their keys, which means that a hack against the exchange's database can result in theft of users' private keys.

evidence, if not an actual proof, that there is no way to find two distinct values x and y such that $h(x) = h(y)$ that is any better than random guessing.

The current standard choice of a cryptographic hash function is called SHA-256, a function that generates output 256 bits in length. This means that given a value x , the chance that a randomly chosen y will hash to the same value to which x hashes is $1/2^{256}$. This means that even using the fastest computers, the probability of a successful guess is effectively zero.³

The collision-resistance property contributes to the tamper resistance of a blockchain in a very important way. Suppose an adversary wishes to modify a block B . Since the next-newer block after B contains not only a pointer to B but also the hash of B , any modification to B must be such that the hash of B remains unchanged after the modification in order to avoid having to modify also that next-newer block. Finding such a modification is infeasible if the hash function has the collision-resistance property, and, therefore, any attempt to tamper with a block requires changing all newer blocks in the chain.

A second important property that we require of a cryptographic hash function is **irreversibility**, which means that given only $h(x)$, it is infeasible to find x . The term *irreversible* comes from the property that, given x , it is easy to compute $h(x)$, but given only $h(x)$, it is infeasible to find $h^{-1}(h(x))$. The next section shows how this concept is applied to blockchains.⁴

26.3.2 Public-Key Encryption, Digital Signatures, and Irrefutability

Section 9.9 described two categories of encryption methods: *private-key* encryption, where users share a secret key, and *public-key* encryption, where each user has two keys, a public key and a private key. The main problem with private-key encryption is that users must find a way at the outset to share the secret private key. Public-key encryption allows users who have never met to communicate securely. This property of public-key encryption is essential to blockchain applications that serve arbitrarily large communities of users worldwide.

Each user U_i has a public key E_i and a private key D_i . A message encrypted using E_i can be decrypted only with the key D_i , and, symmetrically, a message encrypted using D_i can be decrypted only with the key E_i . If user u_1 wishes to send a secure message x to U_2 , U_1 encrypts x using the public key E_2 of user U_2 . Only U_2 has the key D_2 to decrypt the result. For this to work, the specific function used must have the irreversibility property so that given a public key E_i it is infeasible to compute the inverse function,

³ 2^{256} is larger than 10^{77} . If a computer could make one guess per cycle it would take more than 10^{67} seconds to have a 50 percent chance of guessing correctly. That translates to more than 10^{59} years. To put that in context, astronomers predict that the sun will have grown in size to envelop Earth within 10^{10} years.

⁴This property has long been used for storing passwords. Rather than storing user passwords in clear text, leaving them susceptible to being stolen, hashes are kept instead. Then, when a user logs in and enters a password, the hash of that password is computed and compared to the stored value. Were an attacker to steal the hashes, that attacker would still lack the actual passwords, and, if the hash function in use has the irreversibility property, then it is infeasible for the hacker to reverse-engineer the user passwords.

that is, to find D_i . This creates a mechanism for users who have never met to share secret messages.

Suppose now that instead of seeking to send a secret message, user U_1 wishes to “sign” a document x . User U_1 can encrypt x using the private key D_1 . Since this key is private, no one besides U_1 could have computed that value, but anyone can verify the signature by decrypting using the public key of U_1 , that is, E_1 . This provides a public proof that user U_1 has signed document x .

In blockchain applications, the concept of a digital signature is used to validate transactions. Observe that the linkage of blocks in the blockchain, using a pointer and the hash of block to which the pointer points, means that a user can sign an entire chain simply by signing the hash of the newest block in the chain. See the Further Reading section at the end of the chapter for references to the mathematics of public-key encryption.

26.3.3 Simple Blockchain Transactions

In our discussion of database transactions in Chapter 17, we described a transaction as a sequence of steps that read and/or write data values from the database. That concept of a transaction is based on a data model where there is a single store of data values that are accessed by transactions. A blockchain, in its simplest form, is more closely an analog of a database log in that it records the actual transactions and not just final data values. That analogy breaks down, however, in most blockchains, because transactions are either fully independent or depend explicitly on each other. The model we describe here corresponds to simple Bitcoin transactions.

As an example, consider two users, A and B , and assume A wishes to pay B 10 units of some currency. If this were a traditional banking application with a fiat currency such as the U.S. dollar, the transaction implementing this transfer would read A ’s account balance, decrement it by 10, and write that value to the database, and then read B ’s balance, add 10, and write that value to the database. In a blockchain-based system, this transaction is specified in a different manner.

Rather than referencing data items, a Bitcoin-style blockchain transaction references users and other transactions. Users are referenced by their user ID. User A would locate a transaction or set of transactions from past history $T_1, T_2, \dots, T_n$ that paid A a total of at least 10 units of the currency. A would then create a transaction T that takes the output (i.e., the amount paid to A) by those transactions as input, and as its output pays 10 units of the currency to B and the remainder back to A as the “change.” The original transactions $T_1, T_2, \dots, T_n$ are then treated as having been spent.

Thus, each transaction indicates how much money has been paid to whom; the currency balance of a user A is defined by a set of unspent transactions that have paid money to A . Assuming A is honest, those transactions’ outputs (i.e., the output of $T_1, T_2, \dots, T_n$) would not have been spent already by A in a previous transaction. If A were indeed dishonest and T attempted to spend the output of some T_1 a second time, T would be a **double-spend** transaction. Double-spend transactions and other in-

valid transactions are detected in the mining process that we discuss in Section 26.4, by keeping track of all unspent transactions and verifying that each transaction T_i that is input to T is unspent when T is executed. After T is executed, each such T_i is treated as spent.

Ethereum uses a different and more powerful model, where the blockchain maintains state (including current balance) for each account in the system. Transactions update the state, and can transfer funds from one account to another. The model used in Ethereum is discussed in Section 26.5.

A Bitcoin-style transaction T specifies:

- The input transactions $T_1, T_2, \dots, T_n$.
- The set of users being paid and the amount to be paid to each, which in our example is 10 units to B and the remainder to A .⁵
- A 's signature of the transaction, to prove that A in fact authorized this transaction.
- A more complex transaction might include executable code as part of its specification, but we shall defer that to Section 26.6.
- Data to be stored in the blockchain; the data must be under some size, which is blockchain dependent.

The transaction model described here is quite distinct from that of a traditional database system in a variety of ways, including:

- Existing data items are not modified. Instead, transactions add new information. As a result, not only the current state but also the history leading to the current state are fully visible.
- Conflicts in transaction ordering are prevented. If conflicts occur, the transaction causing a conflict is detected and deemed invalid as part of the process of adding a block to the chain, described in Section 26.4.
- Although the blockchain is a distributed system, a transaction is created locally. It becomes part of the permanent, shared blockchain only through the mining process. This is, in effect, a form of deferred transaction commit.
- Dependencies of one transaction upon another are stated explicitly in a transaction since a transaction lists those transactions whose outputs it uses as input. If we view this in terms of the precedence graph introduced in Chapter 17, our example would include precedence-graph edges $T_1 \rightarrow T, T_2 \rightarrow T, \dots, T_n \rightarrow T$.
- There is no explicit concurrency control. Much of the need for concurrency control is eliminated by the maintenance of a complete history and the direct sequencing

⁵In a real system, there may also be a payout to the miner of the transaction, that is, the node that adds the block to the blockchain, as we discuss in Section 26.4.

of transactions. Thus, there is no contention for the current value of any database data item.

This Bitcoin-based example is not the only way blockchain systems manage transaction ordering. We shall see another example when we consider smart contracts in Section 26.6.

The fact that data may be stored in the blockchain makes the blockchain more than just a tamper-resistant transaction log. It allows for the representation of any sort of information that might be stored in a traditional database. In Section 26.5.2, we shall see how this capability, particularly in blockchains with a concept of blockchain state, makes the blockchain a true database.

26.4 Consensus

Because the blockchain is replicated at all participating nodes, each time a new block is added, all nodes must eventually agree first on which node may propose a new block and then agree on the actual block itself.

In a traditional distributed database system, the consensus process is simplified by the fact that all participants are part of one controlling organization. Therefore, the distributed system can implement global concurrency control and enforce two-phase commit to decide on transaction commit or abort. In a blockchain, there may be no controlling organization, as is the case for a public blockchain like Bitcoin. In the case of a permissioned blockchain, there may be a desire to have a high degree of decentralized control in all matters except the actual permissioning of participants, which is managed by the organization controlling the permissioned blockchain.

When transactions are created, they are broadcast to the blockchain network. Nodes may collect a set of transactions to place in a new block to be added to the chain. The consensus mechanisms used in blockchains fall roughly into two categories:

1. Those where the nodes reach agreement on one node to add the next block. These typically use Byzantine consensus (Section 26.4.3).
2. Those where the blockchain is allowed temporarily to **fork** by allowing multiple nodes to create a block following the last block in the chain. In this approach, nodes attempt to add blocks to the longest linear subchain. Those blocks not on that longest chain are **orphaned** and not considered part of the blockchain. To avoid a massive number of forks being created, this approach limits the rate at which blocks may be added so that the expected length of orphaned branches is short. These typically use proof-of-work (Section 26.4.1) or proof-of-stake (Section 26.4.2).

A node that adds a block to the chain must first check that block of transactions. This entails checking that:

- Each transaction is well-formed.
- The transaction is not double-spending by using as input (i.e., spending) currency units that have been used already by a prior transaction. To do so, each node must track the set of all unspent currency units (transactions), and look up this set for each transaction T to ensure that all the currency units that are inputs to T are unspent.
- The transaction is correctly signed by the submitting user.

When a node is selected to add a block to the chain, that block is propagated to all nodes, and each checks the block for validity before adding it to its local copy of the chain.

We next need to consider the question of why any node would want to use its resources for mining, that is to carry out the work needed to append blocks to the chain. Mining is a service to the blockchain network as a whole, and so miners are paid (in the currency of the blockchain) for their efforts. There are two sources of payment to miners:

1. A fee paid by the system in new coins in the currency of the blockchain.
2. A fee included by the submitter of the transaction. In this case the output of the transaction includes an additional output representing a payment to the miner of the block containing the transaction. Users are incented to include fees since such fees incent miners to include their transactions preferentially in new blocks.

The exact means of paying miners varies among blockchains.

In this section, we look at various ways to achieve consensus. We begin by assuming a public blockchain and describe consensus based on two approaches: *proof-of-work* and *proof-of-stake*. We then consider permissioned blockchains that in many cases choose to use a consensus mechanism based on *Byzantine consensus*.

26.4.1 Proof of Work

Proof-of-work consensus is designed for public blockchains in which the number of participating nodes is changing continuously. Any computer may download the blockchain and attempt to add blocks. As a result, a majority-based approach can be overwhelmed by an adversary who sets up a large number of low-cost computers as nodes. As mentioned earlier, such an attack is called a *Sybil attack*. Instead, proof-of-work requires a node to solve a computationally hard, but not infeasible, mathematical problem. An attacker cannot overwhelm a blockchain network simply by adding inexpensive nodes. Rather, the attacker would need to have access to computation capacity that forms a majority of the network's total computation capacity, a task that is much more difficult and costly than launching a Sybil attack.

The computationally hard problem is based on the concept of cryptographic hashing. A node that wishes to mine a block B as the next block needs to find a value, called a **nonce**, that, when concatenated to B and the hash of the previous block, hashes to a value less than a preset target value specified for the blockchain. The nonce is typically a 32-bit value. If the target is set very low, say to 4, and assuming the usual 256-bit hash, a miner would have only a $1/2^{254}$ chance of succeeding for a single choice for the nonce. If the target were set very high, say to 2^{255} , the miner would have a 50 percent chance of success. Blockchain implementations are designed to vary the target so as to control the rate of mining of blocks across the whole system. This variability allows the system to adjust as computation power increases whether due to hardware advances or due to additional nodes joining the network. The target times vary for different blockchains. Bitcoin targets having some node in the system successfully mine a block every 10 minutes. Ethereum targeted a mining time of 10 to 15 seconds with its proof-of-work mechanism. As of late 2018, Ethereum is moving to a proof-of-stake mechanism and is expected to target a slightly faster rate. While faster may appear to be better, note that if mining occurs at a faster rate than the time it takes to propagate a new block throughout the network, the probability of forks and orphaned blocks increases.

Now that we have seen how proof-of-work mining works, let us recall the properties we stated about cryptographic hash functions. If there were an efficient algorithm for finding a nonce that results in a hash less than the target, miners might find nonces too quickly. Therefore, the hash function must ensure that there is no better way to find a nonce than simply trying each possible nonce value in turn. This leads us to require one additional property for cryptographic hash functions, the **puzzle-friendliness** property. This property requires that given a value k , for any n -bit value y it is infeasible to find a value x such that $h(x||k) = y$ in time significantly less than 2^n , where $||$ denotes concatenation of bit strings.

Proof-of-work mining is controversial. On the positive side, for a large network, it would be highly costly for an adversary to obtain enough computational power to dominate mining. However, on the negative side, the amount of energy used in mining is huge. Estimates as this chapter is being written suggest that Bitcoin mining worldwide consumes about 1 percent of the power consumed by the United States, or more than the entire consumption of several nations, for example Ireland. The large amount of computation needed has created incentives to design special-purpose computing chips for mining and incentives to locate large mining installations near sources of cheap power sources.

These concerns are causing a movement to alternatives, such as proof-of-stake, which we discuss next. These concerns have led also to interest in alternative forms of proof-of-work that, for example, require having a large amount of main memory in order quickly to find a nonce. Memory-intensive schemes retain the cost barrier of proof-of-work while reducing the energy waste. They are a subject of current research. Furthermore, we shall see that for enterprise permissioned-blockchain applications, much less costly means of consensus are possible.

In practice, a group of users may unite to form a *mining pool*, which is a consortium that works together to mine blocks and then shares the proceeds among its members.

26.4.2 Proof of Stake

The concept of proof-of-stake is to allow nodes holding a large stake in the currency of the blockchain to be chosen preferentially to add blocks. This rule cannot be applied absolutely, since then a single largest stakeholder would control the chain. Instead, the probability of mining success, using proof-of-work, is made higher for nodes in proportion to their stake. By adjusting both the stake requirements and the mining difficulty, it remains possible to control the rate at which blocks are mined.

There are a wide variety of proof-of-stake schemes. They may include measurement not only of overall stake, but also the total time a stake has been held. They may require that the stake or some fraction of it be held inactive for some period of time in the future.

Properly tuning a proof-of-stake mechanism is difficult. Not only are there more parameters to consider than in proof-of-work, but also one must guard against a situation where there is too little cost penalty for an adversary to add blocks to a fork other than the longest one.

26.4.3 Byzantine Consensus

An important alternative to work- or stake-based consensus is message-based consensus. Message-based consensus is widely used in distributed database systems. As we noted earlier, the basic consensus protocols do not work for blockchain consensus because it cannot be assumed that there are no malicious nodes.

Message-based systems aim to achieve consensus via a majority vote. Such systems are vulnerable to a Sybil attack. In an enterprise permissioned blockchain, in which users have to be granted permission to participate, Sybil attacks are not possible since the permissioning authority can easily deny permission when a malicious user attempts to add an excessive number of nodes. However, even in this setting, one cannot assume every user is totally honest.

For example, consider a supply-chain blockchain in which all suppliers enter data on the chain pertaining to each item being supplied either to another supplier or the ultimate manufacturer of an end-user product. Some supplier might choose to falsify data for its own advantage, but, when a fraud investigation begins, that supplier may then seek to fork the blockchain to cover-up its fraud. Thus, even absent the possibility of Sybil attacks, there remains the possibility of adversarial behavior. It is difficult to anticipate every possible form of adversarial behavior.

For this reason, we model this situation using the concept of **Byzantine failure** in which it is assumed that a “failed” node can behave in an arbitrary manner, and the network of non-failed nodes must be robust to all such misbehavior, including misbehavior that takes exactly the needed set of steps to sabotage the network. The assumption of

Byzantine failure is quite different from the assumption made by consensus protocols, where the only type of failure considered is the absence of function, that is, the only way a node or network link fails to stop working and thus do nothing. This is referred to as a *fail-stop* model and precludes any malicious behavior.

In Section 23.8, we discussed distributed consensus protocols, notably Paxos and Raft. These protocols depend on the fail-stop assumption, but allow agreement using majority rule (in contrast, 2PC requires unanimity of agreement). For Byzantine consensus, we must seek a form of majority rule that overcomes not only the failure of a minority of nodes, but also the possible malicious behavior of that minority. For example, a malicious node n_1 may tell node n_2 that it desires to commit a transaction, but tell n_3 that it desires to abort the transaction. As one might expect, achieving consensus in the face of such malicious nodes requires a higher cost in the number of messages sent to achieve agreement, but in a blockchain, that higher cost is acceptable since it can be much lower than the cost of proof-of-work or proof-of-stake mining.

The development of Byzantine consensus algorithms began in the early 1980s; see the Further Reading section at the end of the chapter for references. There has been much theoretical work relating the number of rounds of messaging, the total number of messages sent, and the fraction of the nodes that can be malicious without causing the protocol to fail. Early work made assumptions about network behavior, such as the time it takes to deliver a message or that the network behaves in a highly synchronous manner. Modern Byzantine consensus algorithms are based on real-world assumptions and incorporate cryptographic signatures to guard against forged messages. The degree of synchronization is reduced, but truly asynchronous fault-tolerant consensus is provably impossible. One widely used approach, called *Practical Byzantine Fault Tolerance*, tolerates malicious failure of up to $\lfloor \frac{n-1}{3} \rfloor$ nodes and is viewed as providing an acceptable level of performance. Other protocols are referenced in the Further Reading section at the end of the chapter.

26.5 Data Management in a Blockchain

Until now we have not been concerned about the efficiency of looking up information in a blockchain. While individual users can track their unspent transactions, that is not sufficient to validate a block. Each node needs to be able to check each transaction in a block to see if it was already spent. In principle, that could be done by searching the entire blockchain, but that is far too costly since it could involve searching backwards to the very first block in the chain. In this section, we shall consider data structures to make such lookups efficient.

Furthermore, not every blockchain uses a transaction model in which transaction inputs are restricted to be the direct output of other transactions. Some, notably Ethereum, allow for the maintenance of a state for each user (account, in Ethereum parlance) that holds the account balance (in the Ethereum currency, Ether) and some

associated storage. This transaction and data model comes closer to that of a database system. Simply storing this information in a database, however, would not preserve the blockchain properties we listed in Section 26.2. In this section, we consider this richer model and how it can be represented physically either via specialized data structures or with the help of database system concepts.

26.5.1 Efficient Lookup in a Blockchain

As we noted earlier, in order to validate a Bitcoin-style transaction, a node needs to check three items:

1. The transaction is syntactically well formed (proper data format, sum of inputs equals sum of outputs, and so on). This is relatively straightforward.
2. The transaction is signed by the user submitting it. This is a matter of ensuring that the signature, which should have been produced by the user submitting the transaction using her or his private key, can be decrypted with that user's public key to obtain the transaction itself. This is not a highly costly step.
3. The transaction's inputs have not been spent already. This entails looking up each individual input transaction in the blockchain. These transactions could be anywhere in the blockchain since they can be arbitrarily old. Without a good means of performing this lookup, this step would be prohibitively costly.

To test for an input transaction having been used already, it is necessary to be able to check that transaction did not appear earlier as input to another transaction. Thus, it suffices for each node to maintain an index on all unspent transactions. Entries in this index point to the location of the corresponding transaction in the blockchain, allowing the details of the input transaction to be validated.

Bitcoin, like many other blockchains, facilitates lookup and validation by storing transactions within a block in a Merkle tree, which we discussed in Section 23.6.6. In that section, we noted that a Merkle tree enables the efficient verification of a collection (transactions, in the case of a blockchain) that may have been corrupted by a malicious user. In a blockchain, there are optimizations to the Merkle tree possible, such as truncating the tree to remove subtrees consisting solely of spent transactions. This reduces significantly the space requirements for nodes to store the full blockchain. Space is a major consideration since major blockchains grow faster than one gigabyte per month, a rate likely to increase as blockchain applications grow.

The Merkle-tree structure is particularly useful for light nodes (i.e., nodes that do not store the entire blockchain) since they need to retain only the root hash of the tree for verification. A full node can then provide any needed data to the light node by providing those data plus the hashes needed for the light node to verify that the provided data are consistent with its stored hash value (see Section 23.6.6).

26.5.2 Maintaining Blockchain State

The simple blockchain transaction model of Section 26.3.3 showed how a basic Bitcoin transaction works. There are more complex transactions possible in Bitcoin, but they follow the same pattern of a set of input transactions and a set of payments to users.

In this section, we look at the model used by certain other blockchains, notably Ethereum, that maintain a *state* that holds the balance in each account. Transactions move currency units (*ether* in Ethereum) among accounts. Since transactions are serialized into blocks by miners, there is no need for concurrency-control protocols like those of Chapter 18. Each block contains a sequence of transactions but also contains the state as it existed after execution of transactions in the block. It would be wasteful to replicate the entire state in each block since the modest number of transactions in one block are likely to change a relatively small fraction of the overall state. This creates a need for a data structure allowing better use of storage.

Recall that transactions within a block are stored in a Merkle tree. State is stored similarly. This would appear to offer the possibility of saving space by allowing pointers (plus the associated hash) back to earlier blocks for those parts of the state that are unchanged. The only challenge here is that it must be possible not only to change tree nodes, but also to insert and delete them. A variant of the Merkle-tree data structure, called a **Merkle-Patricia tree**, is used for this purpose in some blockchains, including Ethereum. This data structure allows for efficient key-based search in the tree. Instead of actually deleting and inserting tree nodes, a new tree root is created and the tree itself structured so as to reference (and thus reuse) subtrees of prior trees. Those prior trees are immutable, so rather than making new parent pointer (which we can't do), a leaf-to-root path is generated by reversing a root-to-leaf path that is easily obtained in the Merkle-Patricia tree structure. Details of this data structure can be found in the references in the Further Reading section at the end of the chapter.

Corda, Hyperledger Fabric, and BigchainDB are examples of blockchains that use a database to store state and allow querying of that state. Fabric and BigchainDB use NoSQL databases. Corda uses an embedded-SQL database. In contrast, Ethereum state is stored in a key-value store.

26.6 Smart Contracts

So far, we have focused on simple funds-transfer transactions. Actual blockchain transactions can be more complex because they may include executable code. Blockchains differ not only in the supported language(s) for such code, but also, and more importantly, in the power of those languages. Some blockchains offer Turing-complete languages, that is, languages that can express all possible computations. Others offer more limited languages.

26.6.1 Languages and Transactions

Bitcoin uses a language of limited power that is suitable for defining many standard types of conditional funds-transfer transactions. Key to this capability is its *multisig*

instruction, which requires m of n specified users to approve the transfer. This enables escrow transactions in which a trusted third party resolves any dispute between the two parties to the actual transfer. It also enables grouping several transactions between two users into one larger transaction without having to submit each component transaction separately to the blockchain. Because adding transactions to the blockchain has a time delay and a cost in transaction fees, this feature is quite important. This concept has been extended in off-chain processing systems, which we discuss in Section 26.7.

Ethereum as well as most blockchains targeting enterprise applications include a language that is Turing complete. Many use familiar programming languages or variants based heavily on such languages. This would seem like an obvious advantage over less-powerful languages, but it comes at some risk. Whereas it is impossible to write an infinite loop in Bitcoin's language, it is possible to do so in any Turing-complete language. A malicious user could submit a transaction that encodes an infinite loop, thereby consuming an arbitrarily large amount of resources for any node attempting to include that transaction in a newly mined block. Testing code for termination, the halting problem, is a provably unsolvable problem in the general case. Even if the malicious user avoids an infinite loop, that user could submit code that runs for an exceptionally long time, again consuming miner resources. The solution to this problem is that users submitting a transaction agree to pay the miner for code execution, with an upper bound placed on the payment. This limits the amount of total execution to some bounded amount of time.

The decentralized nature of a blockchain leads to an incentive system for users to convince miners to include their transaction and thus execute their code. Ethereum's solution is based on the concept of *gas*, so named as to provide an analogy to automobile fuel. Each instruction consumes a fixed amount of gas. Gas consumption in a transaction is governed by three parameters:

1. **Gas price:** the amount of ether the user is offering to pay the miner for one unit of gas.
2. **Transaction gas limit:** the upper bound on transaction gas consumption. Transactions that exceed their gas limit are aborted. The miner keeps the payment, but the transaction actions are never committed to the blockchain.
3. **Block gas limit:** a limit in the blockchain system itself on the sum over all transactions in a block of their transaction gas limits.

A user who sets a gas price too low may face a long wait to find a miner willing to include the transaction. Setting the gas price too high results in the user overpaying.

Another hard choice is that of the gas limit. It is hard to set the limit to the precise amount of gas that the contract will use. A user who sets the limit too low risks transaction failure, while a user who, fearing "running out of gas," sets the transaction gas limit excessively high may find that miners are unwilling to include the transaction because it consumes too large a fraction of the block gas limit. The result of this is an

interesting problem for transaction designers in optimizing for both cost and speed of mining.

In a Bitcoin-style transaction, transaction ordering is explicit. In a state-based blockchain like Ethereum, there is no explicit concept of input transactions. However, there may be important reasons why a smart contract may wish to enforce a transaction order. For transactions coming from the same account, Ethereum forces those transactions to be mined in the order in which the account created them by means of an *account nonce* associated with the transaction. An account nonce is merely a sequence number associated with each transaction from an account, and the set of transactions from an account must have consecutive sequence numbers. Two transactions from an account cannot have the same sequence number, and a transaction is accepted only after the transaction with the previous sequence number has been accepted, thus preventing any cheating in transaction ordering. If the transactions to be ordered are from different accounts, they need to be designed such that the second transaction in the ordering would fail to validate until after the first transaction is processed.

The fact that miners must run the smart-contract code of transactions they wish to include in a block, and that all full nodes must run the code of all transactions in mined blocks, regardless of which node mined the block, leads to a concern about security. Code is run in a safe manner, usually on a virtual machine designed in the style of the Java virtual machine. Ethereum has its own virtual machine, called the EVM. Hyperledger executes code in Docker containers.

26.6.2 External Input

A smart contract may be defined in terms of external events. As a simple example, consider a crop-insurance smart contract for a farmer that pays the farmer an amount of money dependent on the amount of rainfall in the growing season. Since the amount of rainfall in any future season is not known when the smart contract is written, that value cannot be hard-coded. Instead, input must be taken from an external source that is trusted by all parties to the smart contract. Such an external source is called an **oracle**.⁶

Oracles are essential to smart contracts in many business applications. The fact that the oracle must be trusted is a compromise on the general trustlessness of a blockchain environment. However, this is not a serious compromise in the sense that only the parties to a contract need to agree on any oracles used and, once that agreement is made, the agreement is coded into the smart contract and is immutable from that point forward.

Corruption of an oracle after it is coded into an operating smart contract is a real problem. This issue could be left as an externality for the legal system but ideally, a process for settlement of future disputes would be coded into the contract in a variety of ways. For example, parties to the contract could be required to send the contract

⁶This term is rooted in ancient Greek culture and bears no relationship to the company by the same name.

certification messages periodically, and code could be written defining actions to be taken in case a party fails to recertify its approval of the oracle.

Direct external output from a smart contract is problematic since such output would have to occur during its execution and thus before the corresponding transaction is added to the blockchain. Ethereum, for example, deals with this by allowing a smart contract to *emit events* that are then logged in the blockchain. The public visibility of the blockchain then allows the actions of the smart contract to trigger activity external to the blockchain.

26.6.3 Autonomous Smart Contracts

In many blockchains, including Ethereum, smart contracts can be deployed as independent entities. Such smart contracts have their own account, balance, and storage. This allows users (or other smart contracts) to use services provided by a smart contract and to send or receive currency from a smart contract.

Depending on how a specific smart contract is coded, a user may be able, by design, to control the smart contract by sending it messages (transactions). A smart contract may be coded so that it operates indefinitely and autonomously. Such a contract is referred to as a **distributed autonomous organization** (DAO).⁷ DAOs, once established, are difficult to control and manage. There is no way to install bug fixes. In addition, there are many unanswered questions about legal and regulatory matters. However, the ability to create these entities that communicate, store data, and do business independent of any user is one of the most powerful features of the blockchain concept. In an enterprise setting, smart contracts operate under some form of control by an organization or a consortium.

A smart contract may be used to create a currency on top of another currency. Ethereum often serves as the base blockchain as this allows the rich existing ecosystem for Ethereum to be leveraged to provide underlying infrastructure. Such higher-level currency units are called tokens, and the process of creating such currencies is referred to as an **initial coin offering** (ICO). An important added benefit of using an existing blockchain as the basis for a token is that it is then possible to reuse key elements of the user infrastructure, most importantly the wallet software users need to store tokens. The ERC-20 Ethereum standard for tokens is widely used. More recent standards, including ERC-223, ERC-621, ERC-721, ERC-777, and ERC-827, are discussed in the references in the Further Reading section at the end of the chapter.

The relative ease of creating an ICO has made it an important method of funding new ventures, but this has also led to several scams, resulting in attempts by governments to regulate this fundraising methodology.

Beyond fundraising, an important application of smart contracts is to create independent, autonomous service providers whose operation is controlled not by humans

⁷The general use of DAO is distinct from a specific distributed autonomous organization called “The DAO”. The DAO was a crowd-funded venture-capital operation that failed due to a bug that enabled a major theft of funds (see Note 26.1 on page 1258).

but by source code, often open-source. In this way, trustless services that do not require their users to trust any person or organization can be created. As we noted earlier, a fully autonomous contract cannot be stopped or modified. Thus, bugs last forever, and the contract can continue as long as it can raise enough currency to support its operation (i.e., for Ethereum, earn enough ether to pay for gas). These risks suggest that some compromise on the concept of trustlessness may make sense in smart-contract design, such as giving the contract creator the ability to send a self-destruct message to the contract.

26.6.4 Cross-Chain Transactions

Up to this point, we have assumed implicitly that a blockchain transaction is limited to one specific blockchain. If one wished to transfer currency from an account on one blockchain to another account that is on a different blockchain, not only is there the issue that the currencies are not the same, but also there is the problem that the two blockchains have to agree on the state of this cross-chain transaction at each point in time.

We have seen a related problem for distributed databases. If a single organization controls the entire distributed system, then two-phase commit can be used. However, if the system is controlled by multiple organizations as in the federated systems discussed in Section 23.5.3, coordination is more difficult. In the blockchain setting, the high level of autonomy of each system and the requirement of immutability set an even higher barrier.

The simplest solution is to use a trusted intermediary organization that operates much like one that exchanges traditional fiat currencies.

If both users have accounts on both blockchains, a trustless transaction can be defined by creating transactions on each chain for the required funds transfer that are designed such that if one transaction is added to its blockchain its smart-contract code reveals a secret that ensures that other transactions cannot be canceled. Techniques used include the following, among others:

- Time-lock transactions that reverse after a certain period of time unless specific events occur.
- Cross-chain exchange of Merkle-tree headers for validation purposes.

A risk in these techniques is the possibility that a successfully mined transaction winds up on an orphaned fork, though there are ways to mitigate these risks. The details are system specific. See the Further Reading section at the end of the chapter.

A more general solution is to create a smart contract that implements a market similar conceptually to a stock exchange in which willing buyers and sellers are matched. Such a contract operates in the role of trusted intermediary rather than a human-run bank or brokerage as would be used for fiat currencies. The technical issues in cross-chain transactions remain an area of active research.

26.7 Performance Enhancement

At a high level, a blockchain system may be viewed as having three major components:

1. **Consensus management:** Proof-of-work, proof-of-stake, Byzantine consensus, or some hybrid approach. Transaction processing performance is dominated by the performance of consensus management.
2. **State-access management:** Access methods to retrieve current blockchain state, ranging from a simple index to locate transactions from a specific account-id or user ID, to key-value store systems, to a full SQL interface.
3. **Smart contract execution:** The environment that runs the (possibly compiled) smart-contract code, typically in a virtualized environment for security and safety.

The rate of transaction processing, referred to as throughput, in blockchain systems is significantly lower than in traditional database systems. Traditional database systems are able to process simple funds-transfer transactions at peak rates on the order of tens of thousands of transactions per second. Blockchain systems' rates are less; Bitcoin processes less than 10 per second, and Ethereum, at present, only slightly more than 10 per second.⁸ The reason is that techniques such as proof-of-work limit the number of blocks that can be added to the chain per unit time, with Bitcoin targeting one block every 10 minutes. A block may contain multiple transactions, so the transaction processing rate is significantly more than 1 in 10 minutes, but is nevertheless limited.

In most applications, transaction throughput is not the only performance metric. A second and often more important metric is transaction latency, or response time. Here, the distributed consensus required by blockchain systems presents a serious problem. As an example, we consider Bitcoin's design in which the mining rate is maintained close to 1 block every 10 minutes. That alone creates significant latency, but added to that is the need to wait for several subsequent blocks to be mined so as to reduce the probability that a fork will cause the transaction's block to be orphaned. Using the usual recommendation of waiting for 6 blocks, we get a true latency of 1 hour. Such response times are unacceptable for interactive, real-time transaction processing. In contrast, traditional database systems commit individual transactions and can easily achieve millisecond response time.

These transaction processing performance issues are primarily issues due to consensus overhead with public blockchains. Permissioned blockchains are able to use faster message-based Byzantine consensus algorithms, but other performance issues still remain, and are continuing to be addressed.

⁸At the time of publication, Ethereum's architects are contemplating advocating a fork to allow faster, lower-overhead mining.

26.7.1 Enhancing Consensus Performance

There are two primary approaches to improve the performance of blockchain consensus:

1. **Sharding:** distributing the task of mining new blocks to enable parallelism among nodes.
2. **Off-chain transaction processing:** Trusted systems that process transactions internally without putting them on the blockchain. These transactions are grouped into a single transaction that is then placed on the blockchain. This grouping may occur with some agreed-upon periodicity or occur only at the termination of the agreement.

Sharding is the partitioning of the accounts in a blockchain into shards that are mined separately in parallel. In the case where a transaction spans shards, a separate transaction is run on each shard with a special system-internal cross-shard transaction recorded to ensure that both parts of the given transaction are committed. The overhead of the cross-shard transaction is low. There are some risks resulting from the fact that splitting the mining nodes up by shard results in smaller sets of miners that are then more vulnerable to attack since the cost to attack a smaller set of miners is less. However, there are ways to mitigate this risk.

Off-chain transactions require deployment of a separate system to manage those transactions. The best known of these is the Lightning network, which not only speeds blockchain transactions via off-chain processing but also can process certain cross-chain transactions. Lightning promises transaction throughput and latency at traditional database-system rates, but provides this at the cost of some degree of anonymity and immutability (i.e., transactions that commit off-chain, but are rejected at the blockchain). By increasing the frequency of transaction confirmations to the blockchain, one can decrease the loss of immutability at the price of reduced performance improvement.

26.7.2 Enhancing Query Performance

Some blockchain systems offer little more than an index on user or account identifiers to facilitate looking up unspent transactions. This suffices for a simple funds-transfer transaction. Complex smart contracts, however, may need to execute general-purpose queries against the stored current state of the blockchain. Such queries may perform the equivalent of join queries, whose optimization we studied at length in Chapter 16. However, the structure of blockchain systems, in which state-access management may be separate from the execution engine may limit the use of database-style query optimization. Furthermore, the data structures used for state representation, such as the Merkle-Patricia tree structure we saw in Section 26.5.2, may limit the choice of algorithms to implement join-style queries.

Blockchain systems built on a traditional or a NoSQL database keep state information within that database and allow smart contracts to run higher-level database-style queries against that state. Those advantages come at the cost of using a database-storage format that may lack the rigorous cryptographic protection of a true blockchain. A good compromise is for the database to be hosted by a trusted provider with updates going not only to the database but also to the blockchain, thus enabling any user who so wishes to validate the database against the secure blockchain.

26.7.3 Fault-Tolerance and Scalability

Performance in the presence of failures is a critical aspect of a blockchain system. In traditional database systems, this is measured by the performance of the recovery manager and, as we saw in Section 19.9, the ARIES recovery algorithm is designed to optimize recovery time. A blockchain system, in contrast, uses a consensus mechanism and a replication strategy designed for continuous operation during failures and malicious attacks, though perhaps with lower performance during such periods. Therefore, besides measuring throughput and latency, one must also measure how these performance statistics change in times of failure or attack.

Scalability is a performance concern in any distributed system as we saw in Chapter 20. The architectural differences between blockchain systems and parallel or distributed database systems introduce challenges in both the measure of scaleup and its optimization. We illustrate the differences by considering the relative scalability of 2PC and Byzantine consensus. In 2PC, a transaction accessing a fixed number of nodes, say five, needs only the agreement of these five nodes, regardless of the number of nodes in the system. If we scale the system up to more nodes, that transaction still needs only those five nodes to agree (unless the scaling added a replica site). Under Byzantine consensus, every transaction needs the agreement of a majority of the non-failed nodes, and so, the number of nodes that must agree not only starts much larger but also grows faster as the network scales.

The Further Reading section at the end of the chapter provides references that deal with the emerging issue of blockchain performance measurement and optimization.

26.8 Emerging Applications

Having seen how blockchains work and the benefits they offer, we can look at areas where blockchain technology is currently in use or may be used in the near future.

Applications most likely to benefit from the use of a blockchain are those that have high-value data, including possibly historical data, that need to be kept safe from malicious modification. Updates would consist mostly of appends in such applications. Another class of applications that are likely to benefit are those that involve multiple cooperating parties, who trust each other to some extent, but not fully, and desire to have a shared record of transactions that are digitally signed, and are kept safe from tampering. In this latter case, the cooperating parties could include the general public.

Below, we provide a list of several application domains along with a short explanation of the value provided by a blockchain implementation of the application. In some cases, the value added by a blockchain is a novel capability; in others, the value added is the ability to do something that could have been done previously only at prohibitive cost.

- **Academic certificates and transcripts:** Universities can put student certificates and transcripts on a public blockchain secured by the student's public key and signed digitally by the university. Only the student can read the records, but the student can then authorize access to those records. As a result, students can obtain certificates and transcripts for future study or for prospective employers in a secure manner from a public source. This approach was prototyped by MIT in 2017.
- **Accounting and audit:** Double-entry bookkeeping is a fundamental principle of accounting that helps ensure accurate and auditable records. A similar benefit can be gained from cryptographically signed blockchain entries in a digital distributed ledger. In particular, the use of a blockchain ensures that the ledger is tamperproof, even against insider attacks and hackers who may gain control of the database. Also, if the enterprise's auditor is a participant, then ledger entries can become visible immediately to auditors, enabling a continuous rather than periodic audit.
- **Asset management:** Tracking ownership records on a blockchain enables verifiable access to ownership records and secure, signed updates. As an example, real-estate ownership records, a matter of public record, could be made accessible to the public on a blockchain, while updates to those records could be made only by transactions signed by the parties to the transaction. A similar approach can be applied to ownership of financial assets such as stocks and bonds. While stock exchanges manage trading of stocks and bonds, long term records are kept by depositories that users must trust. Blockchain can help track such assets without having to trust a depository.
- **E-government:** A single government blockchain would eliminate agency duplication of records and create a common, authoritative information source. A highly notable user of this approach is the government of Estonia, which uses its blockchain for taxation, voting, health, and an innovative "e-Residency" program.
- **Foreign-currency exchange:** International financial transactions are often slow and costly. Use of an intermediary cryptocurrency can enable blockchain-based foreign-currency exchange at a relative rapid pace with full, irrefutable traceability. Ripple is offering such capability using the XRP currency.
- **Health care:** Health records are notorious for their nonavailability across health-care providers, their inconsistency, and their inaccuracy even with the increased use of electronic health records. Data are added from a large number of sources and the provenance of materials used may not be well documented (see discussion of supply chains below). A unified blockchain is suitable for distributed update,

and cryptographic data protection, unlockable by the patient's private key, would enable access to a patient's full health record anytime, anywhere in an emergency. The actual records may be kept offchain, but the blockchain acts as the trusted mechanism for accessing the data.

- **Insurance claims:** The processing of insurance claims is a complex workflow of data from the scene of the claim, various contractors involved in repairs, statements from witnesses, etc. A blockchain's ability to capture data from many sources and distribute it rapidly, accurately, and securely, promises efficiency and accuracy gains in the insurance industry.
- **Internet of things:** The Internet of Things (IoT) is a term that refers to systems of many interacting devices ("things"), including within smart buildings, smart cities, self-monitoring civil infrastructure, and so on. These devices could act as nodes that can pass blockchain transactions into the network without having to ensure the transmission of data reaches a central server. In the late 2010s, research is underway to see if this data-collection approach can be effective in lowering costs and increasing performance. Adding so many entries to a blockchain in a short period of time may suggest a replacement of the chain data structure with a directed, acyclic graph. The Iota blockchain is an example of one such system, where the graph structure is called a *tangle*.
- **Loyalty programs and aggregation of transactions:** There are a variety of situations where a customer or user makes multiple purchases from the same vendor, such as within a theme park, inside a video game, or from a large online retailer. These vendors could create internal cryptocurrencies in a proprietary, permissioned blockchain, with currency value pegged to a fiat currency like the dollar. The vendor gains by replacing credit-card transactions with vendor-internal transactions. This saves credit-card fees and allows the vendor to capture more of the valuable customer data coming from these transactions. The same concept can apply to retail loyalty points, exemplified by airline frequent-flyer miles. It is costly for vendors to maintain these systems and coordinate with partner vendors in the program. A blockchain-based system allows the hosting vendor to distribute the workload among the partners and allows transactions to be posted in a decentralized manner, relieving the vendor from have to run its own online transaction processing system. In the late 2010s, business strategies were being tested around these concepts.
- **Supply chain:** Blockchain enables every participant in a supply chain to log every action. This facilitates tracking the movement of every item in the chain rather than only aggregates like crates, shipments, etc. In the event of a recall, the set of affected products can be pinpointed to a smaller set of products and done so quickly. When a quality issue suggests a recall, some supply-chain members may be tempted to cover up their role, but the immutability of the blockchain prevents record falsification after the fact.

- **Tickets for events:** Suppose a person *A* has bought tickets for an event, but now wishes to sell them, and *B* buys the ticket from *A*. Given that tickets are all sold online, *B* would need to trust that the ticket given by *A* is genuine, and *A* has not already sold the ticket, that is, the ticket has not been double-spent. If ticket transactions are carried out on a blockchain, double-spending can be detected easily. Tickets can be verified if they are signed digitally by the event organizer (whether or not they are on a blockchain).
- **Trade finance:** Companies often depend on loans from banks, issued through letters of credits, to finance purchases. Such letters of credit are issued against goods based on bills of lading indicating that the goods are ready for shipment. The ownership of the goods (title) is then transferred to the buyer. These transactions involve multiple parties including the seller, buyer, the buyer's bank, the seller's bank, a shipping company and so forth, which trust each other to some extent, but not fully. Traditionally, these processes were based on physical documents that have to be signed and shipped between parties that may be anywhere on the globe, resulting in significant delays in these processes. Blockchain technology can be used to keep these documents in a digital form, and automate these processes in a way that is highly secure yet very fast (at least compared to processing of physical documents).

Other applications beyond those we have listed continue to emerge.

26.9 Summary

- Blockchains provide a degree of privacy, anonymity, and decentralization that is hard to achieve with a traditional database.
- Public blockchains are accessibly openly on the internet. Permissioned blockchains are managed by an organization and usually serve a specific enterprise or group of enterprises.
- The main consensus mechanisms for public blockchains are proof-of-work and proof-of-stake. Miners compete to add the next block to the blockchain in exchange for a reward of blockchain currency.
- Many permissioned blockchains use a Byzantine consensus algorithm to choose the node to add the next block to the chain.
- Nodes adding a block to the chain first validate the block. Then all full nodes maintaining a replica of the chain validate the new block.
- Key blockchain properties include irrefutability and tamper resistance.
- Cryptographic hash functions must exhibit collision resistance, irreversibility, and puzzle friendliness.

- Public-key encryption is based on a user having both a public and private key to enable both the encryption of data and the digital signature of documents.
- Proof-of-work requires a large amount of computation to guess a successful nonce that allows the hash target to be met. Proof-of-stake is based on ownership of blockchain currency. Hybrid schemes are possible.
- Smart contracts are executable pieces of code in a blockchain. In some chains, they may operate as independent entities with their own data and account. Smart contracts may encode complex business agreements and they may provide ongoing services to nodes participating in the blockchain.
- Smart contracts get input from the outside world via trusted oracles that serve as a real-time data source.
- Blockchains that retain state can serve in a manner similar to a database system and may benefit from the use of database indexing methods and access optimization, but the blockchain structure may place limits on this.

Review Terms

- | | |
|--------------------------------------|---------------------------|
| • Public and permissioned blockchain | • Double spend |
| • Cryptographic hash | • Orphaned block |
| • Mining | • Nonce |
| • Light and full nodes | • Block validation |
| • Proof-of-work | • Merkle tree |
| • Proof-of-stake | • Patricia tree |
| • Byzantine consensus | • Bitcoin |
| • Tamper resistance | • Ethereum |
| • Collision resistance | • Gas |
| • Irreversibility | • Smart contract |
| • Public-key encryption | • Oracles |
| • Digital signature | • Cross-chain transaction |
| • Irrefutability | • Sharding |
| • Forks: hard and soft | • Off-chain processing |

Practice Exercises

- 26.1** What is a blockchain fork? List the two types of fork and explain their differences.

- 26.2** Consider a hash function $h(x) = x \bmod 2^{256}$, that is, the hash function returns the last 256 bits of x .
Does this function have
- collision resistance
 - irreversibility
 - puzzle friendliness
- Why or why not?
- 26.3** If you were designing a new public blockchain, why might you choose proof-of-stake rather than proof-of-work?
- 26.4** If you were designing a new public blockchain, why might you choose proof-of-work rather than proof-of-stake?
- 26.5** Explain the distinction between a public and a permissioned blockchain and when each would be more desirable.
- 26.6** Data stored in a blockchain are protected by the tamper-resistance property of a blockchain. In what way is this tamper resistance more secure in practice than the security provided by a traditional enterprise database system?
- 26.7** In a public blockchain, how might someone determine the real-world identity that corresponds to a given user ID?
- 26.8** What is the purpose of *gas* in Ethereum?
- 26.9** Suppose we are in an environment where users can be assumed not to be malicious. In that case, what advantages, if any, does Byzantine consensus have over 2PC?
- 26.10** Explain the benefits and potential risks of sharding.
- 26.11** Why do enterprise blockchains often incorporate database-style access?

Exercises

- 26.12** In what order are blockchain transactions serialized?
- 26.13** Since blockchains are immutable, how is a transaction abort implemented so as not to violate immutability?
- 26.14** Since pointers in a blockchain include a cryptographic hash of the previous block, why is there the additional need for replication of the blockchain to ensure immutability?
- 26.15** Suppose a user forgets or loses her or his private key? How is the user affected?

- 26.16** How is the difficulty of proof-of-work mining adjusted as more nodes join the network, thus increasing the total computational power of the network? Describe the process in detail.
- 26.17** Why is Byzantine consensus a poor consensus mechanism in a public blockchain?
- 26.18** Explain how off-chain transaction processing can enhance throughput. What are the trade-offs for this benefit?
- 26.19** Choose an enterprise of personal interest to you and explain how blockchain technology could be employed usefully in that business.

Tools

One can download blockchain software to create a full node for public blockchains such as Bitcoin (bitcoin.org) and Ethereum (www.ethereum.org) and begin mining, though the economic return for the investment of power may be questionable. Tools exist also to join mining pools. Browsing tools exist to view the contents of public blockchains. For some blockchains, notably Ethereum, it is possible to install a private copy of the blockchain software managing a private blockchain as an educational tool. Ethereum also offers a public test network where smart contracts can be debugged without the expense of gas on the real network.

Hyperledger (www.hyperledger.org) which is supported by a large consortium of companies, provides a wide variety of open source blockchain platforms and tools. Corda (www.corda.net) and BigchainDB (www.bigchaindb.com) are two other blockchain platforms, with BigchainDB having a specific focus on blockchain databases.

Blockchain based systems for supporting academic certificates and medical records, such as Blockcert and Medrec (both from MIT), and several other applications are available online. The set of tools for blockchain are evolving rapidly. Due to the rapid rate of change and development, as of late 2018 we are unable to identify a best set of tools, beyond the few mentioned above, that we can recommend. We recommend you perform a web search for the latest tools.

Further Reading

The newness of blockchain technology and applications means that, unlike the more established technical topics elsewhere in this text, there are fewer references in the academic literature and fewer textbooks. Many of the key papers are published only on the website of a particular blockchain. The URLs for those references are likely to change often. Thus, web searches for key topics are a highly important source for further reading. Here, we cite some classic references as well as URLs current as of the publication date.

The original Bitcoin paper [Nakamoto (2008)] is authored under a pseudonym, with the identity of the author or authors still the subject of speculation. The original Ethereum paper [Buterin (2013)] has been superseded by newer Ethereum white papers (see ethereum.org), but the original work by Ethereum's creator, Vitalik Buterin, remains interesting reading. Solidity, the primary programming language for Ethereum smart contracts, is discussed in solidity.readthedocs.io. The ERC-20 standard is described in [Vogelsteller and Buterin (2013)] and the proposed (as of the publication date of this text) Casper upgrade to the performance of Ethereum's consensus mechanism appears in [Buterin and Griffith (2017)]. Another approach to using proof-of-stake is used by the Cardano blockchain (www.cardano.org).

Many of the theoretical results that make blockchain possible were first developed in the 20th century. The concepts behind cryptographic hash functions and public-key encryption were introduced in [Diffie and Hellman (1976)] and [Rivest et al. (1978)]. A good reference for cryptography is [Katz and Lindell (2014)]. [Narayanan et al. (2016)] is a good reference for the basics of cryptocurrency, though its focus is mainly on Bitcoin. There is a large body of literature on Byzantine consensus. Early papers that laid the foundation for this work include [Pease et al. (1980)] and [Lamport et al. (1982)]. Practical Byzantine fault tolerance ([Castro and Liskov (1999)]) serves as the basis for much of the current blockchain Byzantine consensus algorithms. [Mazières (2016)] describes in detail a consensus protocol specifically designed to allow for open, rather than permissioned, membership in the consensus group. References pertaining to Merkle trees appears in Chapter 23. Patricia trees were introduced in [Morrison (1968)].

A benchmarking framework for permissioned blockchains appears in [Dinh et al. (2017)]. A detailed comparison of blockchain systems appears in [Dinh et al. (2018)]. ForkBase, a storage system designed for improved blockchain performance, is discussed in [Wang et al. (2018)].

The Lightning network(lightning.network) aims to accelerate Bitcoin transactions and provide some degree of cross-chain transactions. Ripple (ripple.com) provides a network for international fiat currency exchange using the XRP token. Loopring (loopring.org) is a cryptocurrency exchange platform that allows users to retain control of their currency without having to surrender control to the exchange.

Many of the blockchains discussed in the chapter have their best descriptions on their respective web sites. These include Corda (docs.corda.net), Iota (iota.org), and Hyperledger (www.hyperledger.org). Many financial firms are creating their own blockchains, and some of those are publicly available, including J.P. Morgan's Quorum (www.jpmorgan.com/global/Quorum).

Bibliography

[Buterin (2013)] V. Buterin, "Ethereum: The Ultimate Smart Contract and Decentralized Application Platform", Technical report (2013).

- [Buterin and Griffith (2017)] V. Buterin and V. Griffith, “Casper the Friendly Finality Gadget”, Technical report (2017).
- [Castro and Liskov (1999)] M. Castro and B. Liskov, “Practical Byzantine Fault Tolerance”, In *Symp. on Operating Systems Design and Implementation (OSDI)*, USENIX (1999).
- [Diffie and Hellman (1976)] W. Diffie and M. E. Hellman, “New Directions in Cryptography”, *IEEE Transactions on Information Theory*, Volume 22, Number 6 (1976).
- [Dinh et al. (2017)] T. T. A. Dinh, J. Wang, G. Chen, R. Liu, B. C. Ooi, and K.-L. Tan, “BLOCKBENCH: A Framework for Analyzing Private Blockchains”, In *Proc. of the ACM SIGMOD Conf. on Management of Data* (2017), pages 1085–1100.
- [Dinh et al. (2018)] T. T. A. Dinh, R. Liu, M. H. Zhang, G. Chen, B. C. Ooi, and J. Wang, “Untangling Blockchain: A Data Processing View of Blockchain Systems”, volume 30 (2018), pages 1366–1385.
- [Katz and Lindell (2014)] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 3rd edition, Chapman and Hall/CRC (2014).
- [Lamport et al. (1982)] L. Lamport, R. Shostak, and M. Pease, “The Byzantine Generals Problem”, *ACM Transactions on Programming Languages and Systems*, Volume 4, Number 3 (1982), pages 382–401.
- [Mazières (2016)] D. Mazières, “The Stellar Consensus Protocol”, Technical report (2016).
- [Morrison (1968)] D. Morrison, “Practical Algorithm To Retrieve Information Coded in Alphanumeric”, *Journal of the ACM*, Volume 15, Number 4 (1968), pages 514–534.
- [Nakamoto (2008)] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System”, Technical report, Bitcoin.org (2008).
- [Narayanan et al. (2016)] A. Narayanan, J. Bonneau, E. Felten, A. Miller, and S. Goldfeder, *Bitcoin and Cryptocurrency Technologies*, Princeton University Press (2016).
- [Pease et al. (1980)] M. Pease, R. Shostak, and L. Lamport, “Reaching Agreement in the Presence of Faults”, *Journal of the ACM*, Volume 27, Number 2 (1980), pages 228–234.
- [Rivest et al. (1978)] R. L. Rivest, A. Shamir, and L. Adleman, “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems”, *Communications of the ACM*, Volume 21, Number 2 (1978), pages 120–126.
- [Vogelsteller and Buterin (2013)] F. Vogelsteller and V. Buterin, “ERC-20 Token Standard”, Technical report (2013).
- [Wang et al. (2018)] S. Wang, T. T. A. Dihn, Q. Lin, Z. Xie, M. Zhang, Q. Cai, G. Chen, B. C. Ooi, and P. Ruan, “ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications”, In *Proc. of the International Conf. on Very Large Databases* (2018), pages 1085–1100.

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.